



I. Tóm tắt bài thực hành

1. Yêu cầu lý thuyết

Sinh viên đã được trang bị các kiến thức về phát triển ứng dụng web trên framework Flutter, gồm:

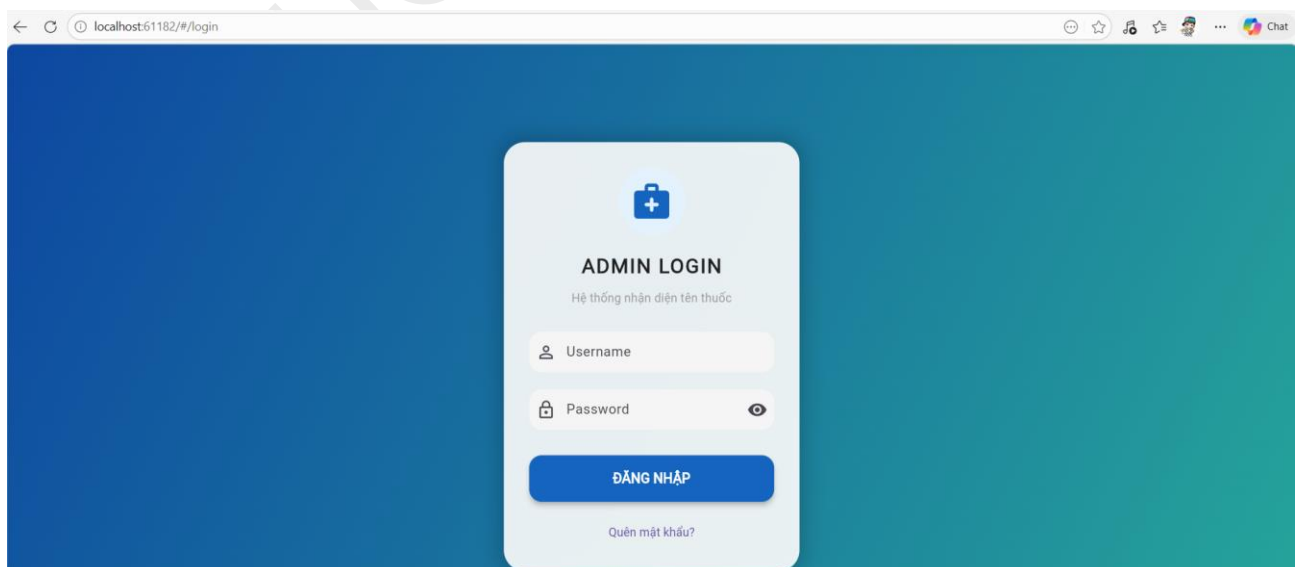
- Ngôn ngữ lập trình Dart
- Giới thiệu Flutter framework
- Widget trong Flutter
- Xử lý tương tác người dùng với Gesture
- Xây dựng Form và xác thực dữ liệu người dùng
- Điều hướng và định tuyến giữa các màn hình
- Quản lý trạng thái trong Flutter
- Hiệu ứng chuyển động và Animation
- Kết nối API và xử lý dữ liệu JSON
- Lưu trữ và quản lý dữ liệu trong thiết bị và trên môi trường cloud

2. Nội dung chính bài thực hành

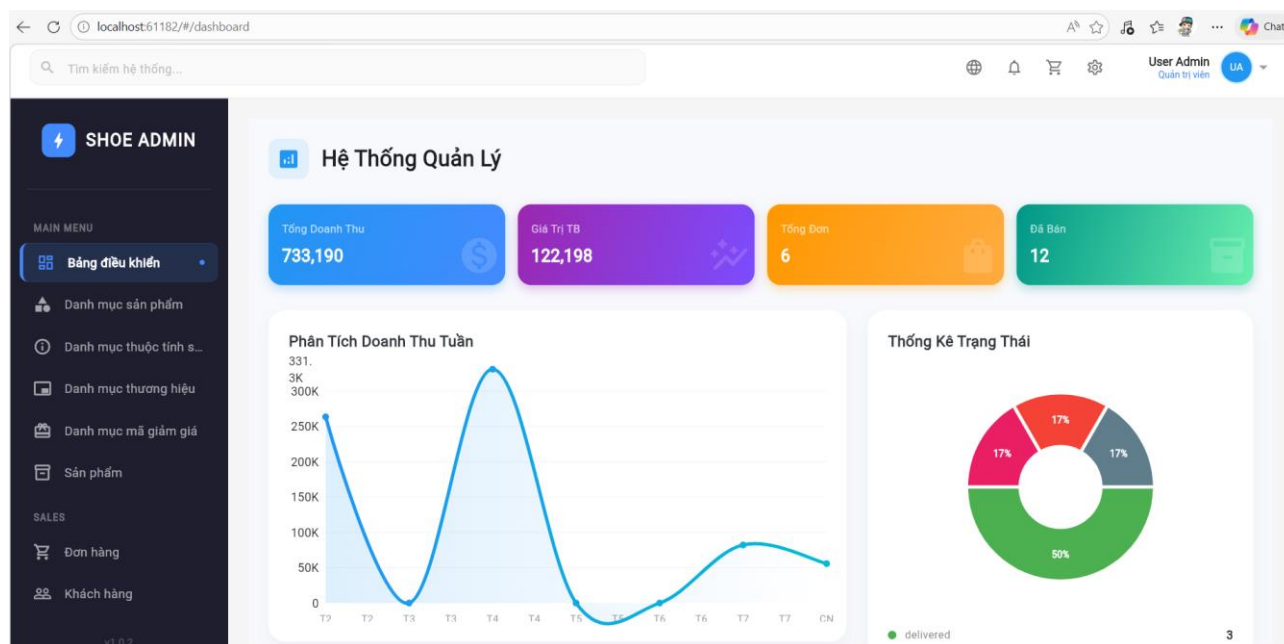
Trong bài thực hành này, sinh viên sẽ thiết kế và phát triển một ứng dụng web hỗ trợ cho người dùng quản trị các nội dung cho hệ thống mua bán sản phẩm trực tuyến

Danh sách các màn hình và chức năng tương ứng trong ứng dụng:

- Màn hình đăng nhập cho quản trị viên hệ thống



- Màn hình bảng tổng hợp thông tin (dashboard) cho quản trị viên hệ thống

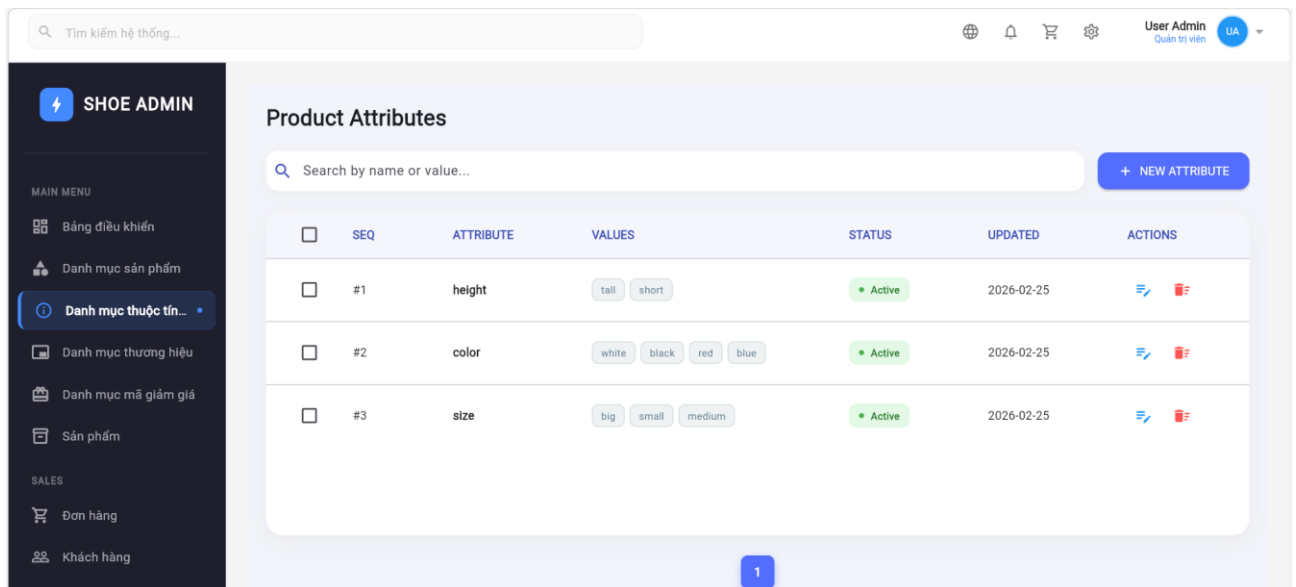


- Màn hình quản lý danh mục sản phẩm

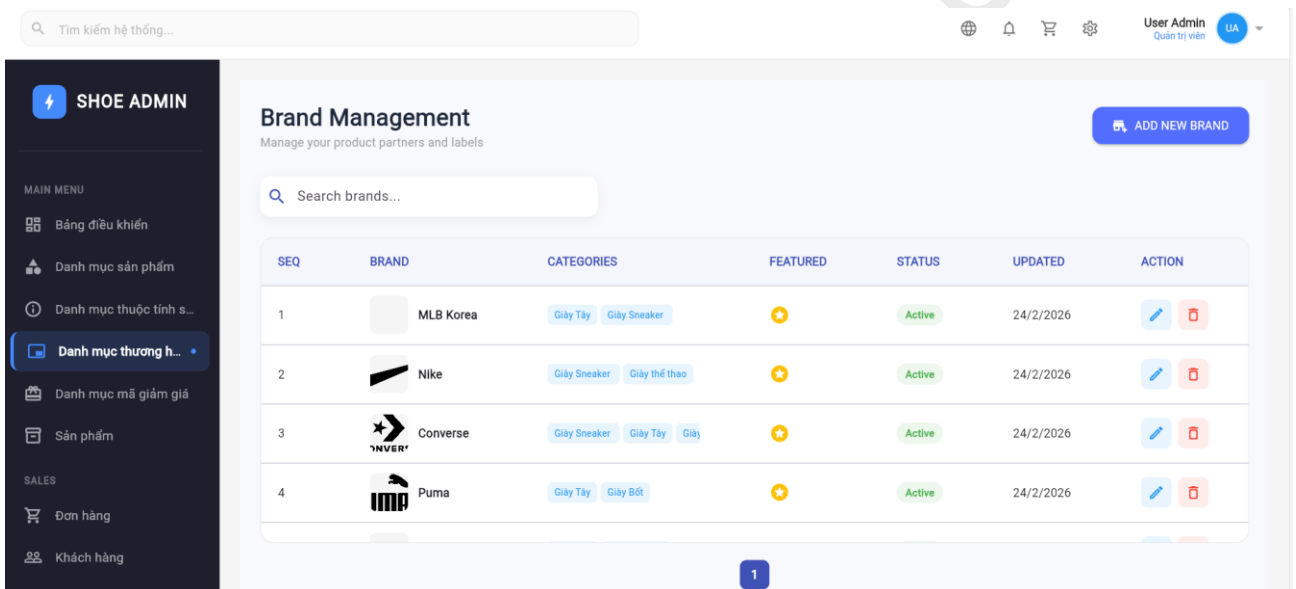
The page shows a table of product categories with the following data:

SEQ	CATEGORY	FEATURED	STATUS	UPDATED	ACTIONS
1	Giày Tây	★	Active	24/02/2026	[Edit] [Delete]
2	Giày Sneaker	★	Active	24/02/2026	[Edit] [Delete]
3	Giày thể thao	★	Active	24/02/2026	[Edit] [Delete]
4	Giày Bốt	★	Active	24/02/2026	[Edit] [Delete]
5	Giày Sandal & Dép	★	Active	24/02/2026	[Edit] [Delete]

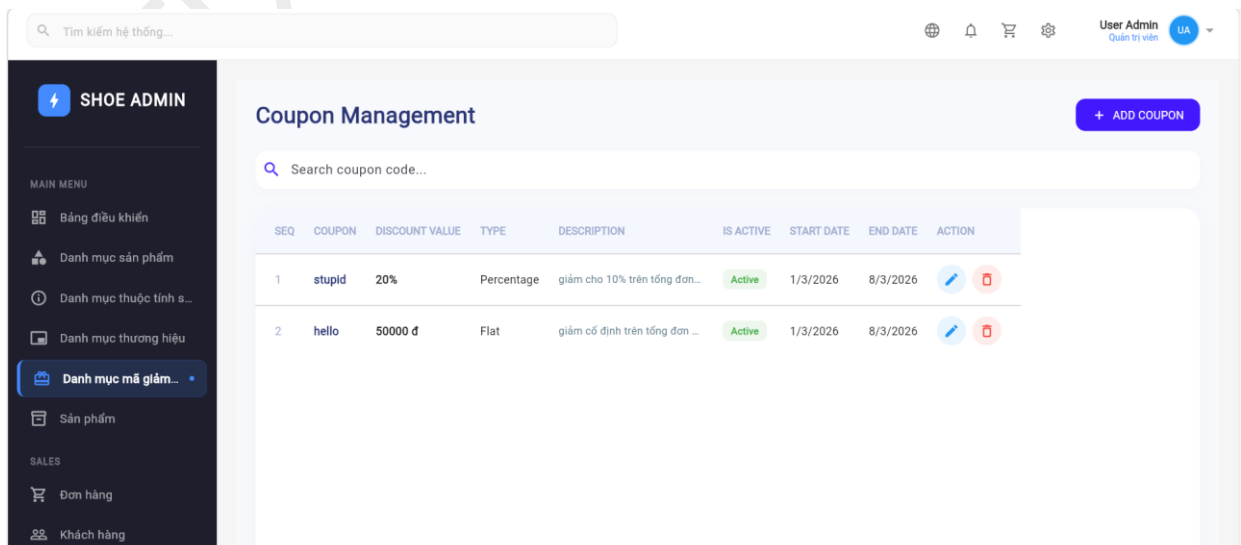
- Màn hình quản lý danh mục thuộc tính sản phẩm



- Màn hình quản lý danh mục thương hiệu sản phẩm



- Màn hình quản lý danh mục mã giảm giá cho đơn hàng



- Màn hình quản lý thông tin sản phẩm

SHOE ADMIN

MAIN MENU

- Bảng điều khiển
- Danh mục sản phẩm
- Danh mục thuộc tính s...
- Danh mục thương hiệu
- Danh mục mã giảm giá
- Sản phẩm**

SALES

- Đơn hàng
- Khách hàng

v1.0.2

Product Inventory

Search products...

+ ADD PRODUCT

SEQ	PRODUCT	PRICE	TYPE	STOCK	CÒN HÀNG	VISIBLE	STATUS	ACTION
1	New Balance Fresh F...	\$1400	simple	5 In Stock	✓	Published	✓	Edit Delete
2	New Balance Fresh F...	\$24000	simple	5 In Stock	✓	Published	✓	Edit Delete
3	Giày Thời Trang Unise...	\$1450	simple	74 In Stock	✓	Published	✓	Edit Delete
4	Nike Giannis Freak 6 ...	\$25000	simple	1300 In Stock	✓	Published	✓	Edit Delete
5	New Balance Fresh F...	\$15000	simple	5000 In Stock	✓	Published	✓	Edit Delete
6	ASICS GEL-GAME 9 - ...	\$79000	simple	1500 In Stock	✓	Published	✓	Edit Delete
7	New Balance Fresh F...	\$15000	simple	2000 In Stock	✓	Published	✓	Edit Delete

< Page 1 of 1 >

- Màn hình quản lý thông tin đơn hàng của khách hàng

SHOE ADMIN

MAIN MENU

- Bảng điều khiển
- Danh mục sản phẩm
- Danh mục thuộc tính s...
- Danh mục thương hiệu
- Danh mục mã giảm giá
- Sản phẩm

SALES

- Đơn hàng**
- Khách hàng

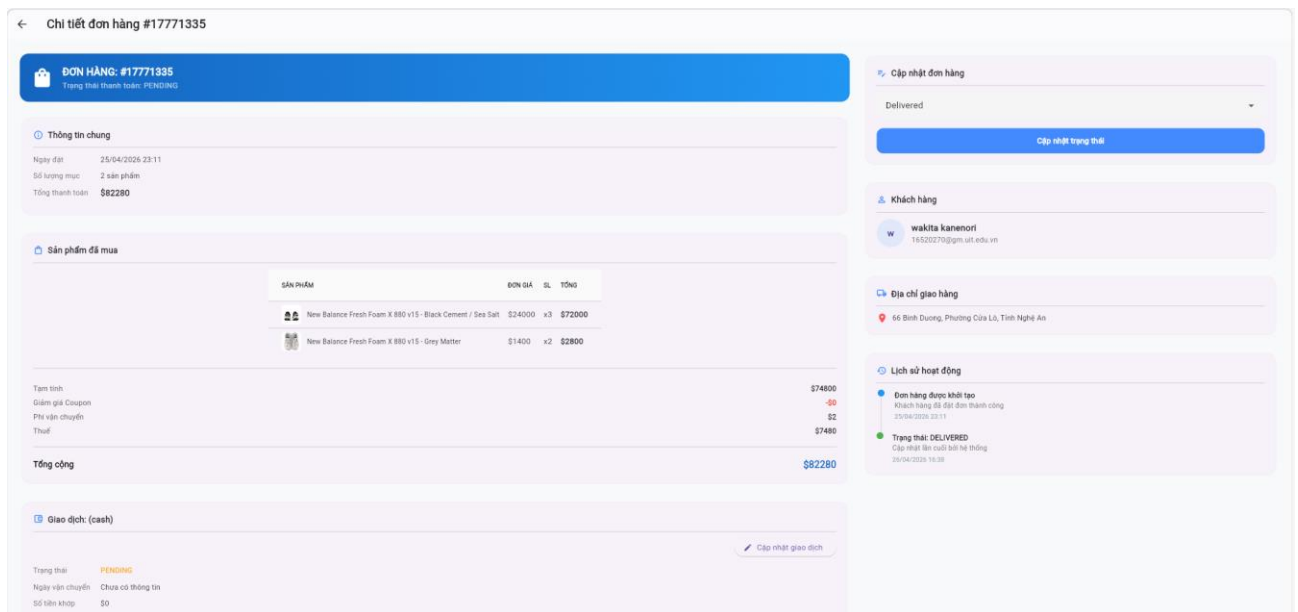
v1.0.2

Order Management

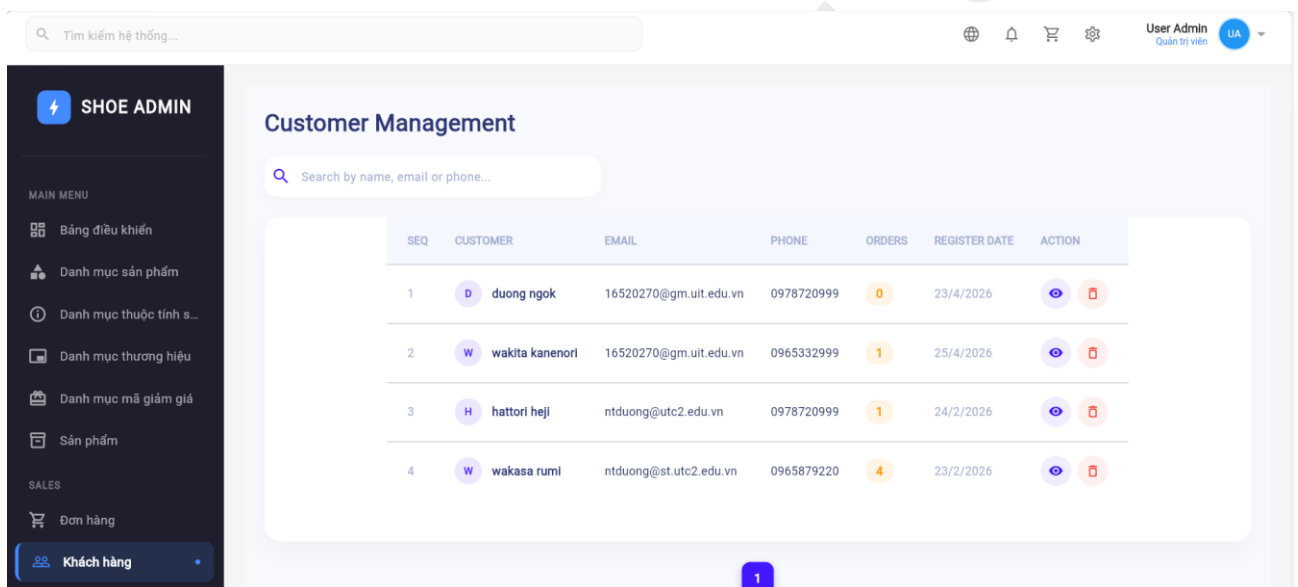
Search by order ID, customer name...

Làm mới

Seq	Order ID	Customer	Item	Order Status	Payment	Amount	Date	Action
1	#1777133500571	w wakita kanenori	2 sp	DELIVERED	PENDING	\$82280	25/04/2026	View Delete
2	#1773224036027	h hattori heji	2 sp	DELIVERED	PENDING	\$206800	11/03/2026	View Delete
3	#1772625372751	w wakasa rumi	2 sp	DELIVERED	PENDING	\$83820	04/03/2026	View Delete
4	#1772622188000	w wakasa rumi	2 sp	REFUNDED	PAID	\$40640	04/03/2026	View Delete
5	#1772459760970	w wakasa rumi	2 sp	CANCELLED	PENDING	\$263780	02/03/2026	View Delete



- Màn hình quản lý thông tin khách hàng



II. Chi tiết bài thực hành

1. Chuẩn bị cấu trúc thư mục trong project

SV tạo 1 project theo framework flutter và chuẩn bị cấu trúc thư mục libs như sau:

```
libs
|   firebase_options.dart
|   main.dart
|
|---controllers
|   attribute_controller.dart
|   auth_controller.dart
|   brand_controller.dart
|   category_controller.dart
|   coupon_controller.dart
```

```
|      customer_controller.dart
|      order_controller.dart
|      product_controller.dart
|      product_review_controller.dart
|
|-----data
|   |-----models
|   |      address_model.dart
|   |      attribute_model.dart
|   |      brand_category_model.dart
|   |      brand_model.dart
|   |      category_model.dart
|   |      coupon_model.dart
|   |      customer_model.dart
|   |      order_model.dart
|   |      product_model.dart
|   |      product_review_model.dart
|   |
|   |-----services
|   |      attribute_service.dart
|   |      auth_service.dart
|   |      brand_service.dart
|   |      category_service.dart
|   |      coupon_service.dart
|   |      customer_service.dart
|   |      order_service.dart
|   |      product_review_service.dart
|   |      product_service.dart
|
|-----routes
|      app_routes.dart
|
|-----views
|   |-----attributes
|   |      attribute_add_edit_page.dart
|   |      attribute_page.dart
|   |
|   |-----auth
|   |      login_page.dart
|   |
|   |-----brands
|   |      brands_page.dart
```

```
|
|
|---categories
|    category_page.dart
|
|---coupons
|    coupons_page.dart
|
|---customers
|    customers_page.dart
|    customer_detail_page.dart
|
|---dashboard
|    dashboard_page.dart
|
|---layout
|    admin_appbar.dart
|    admin_layout.dart
|    sidebar.dart
|
|---orders
|    orders_page.dart
|    order_detail_page.dart
|
|---products
|    product_form_page.dart
|    product_list_page.dart
|
|---product_review
|    all_review_screen.dart
```

2. Tạo kết nối đến CSDL lưu trữ trên Firebase

SV xem lại kiến thức trong chương 11 (Lưu trữ và quản lý dữ liệu trong thiết bị và trên môi trường cloud), thực hiện các nội dung sau:

- Truy cập Firebase Console, đăng nhập tài khoản Google và tạo một project mới (đặt tên, chọn khu vực, tắt/bật Google Analytics nếu cần)
- Trong project Firebase vừa tạo, thực hiện thêm ứng dụng trên nền tảng Flutter và thực hiện các cài đặt cấu hình cơ bản (package name, app name nếu có)
- Cài đặt công cụ FlutterFire CLI trên máy (nếu chưa có), sau đó đăng nhập Firebase bằng lệnh: `firebase login`

- Tại thư mục project Flutter trong Visual Studio Code hoặc Android Studio, chạy lệnh: `flutterfire configure` → Chọn project Firebase đã tạo để liên kết
- Sau khi cấu hình thành công, kiểm tra trong thư mục `lib/` của project Flutter đã xuất hiện file: `firebase_options.dart`
- Mở file `firebase_options.dart` để kiểm tra các thông tin cấu hình Firebase đã được tạo tự động (API key, project ID, app ID,...), nội dung file sẽ như sau:

```
// File generated by FlutterFire CLI.
// ignore_for_file: type=lint
import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
import 'package:flutter/foundation.dart'
    show defaultTargetPlatform, kIsWeb, TargetPlatform;

/// Default [FirebaseOptions] for use with your Firebase apps.
///
/// Example:
/// ```dart
/// import 'firebase_options.dart';
/// // ...
/// await Firebase.initializeApp(
///   options: DefaultFirebaseOptions.currentPlatform,
/// );
/// ```
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) {
      return web;
    }
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        return ios;
      case TargetPlatform.macOS:
        return macos;
      case TargetPlatform.windows:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for windows - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      case TargetPlatform.linux:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for linux - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      default:
        throw UnsupportedError(
```



```
        'DefaultFirebaseOptions are not supported for this platform.',
    );
}
}

static const FirebaseOptions web = FirebaseOptions(
  apiKey: 'AIzaSyCtQyzvaXcahnluQcUA-SfLfQa4I_BelVU',
  appId: '1:233797330677:web:1144538c1e2448281547bc',
  messagingSenderId: '233797330677',
  projectId: 'map-lab-utc2-e-commerce',
  authDomain: 'map-lab-utc2-e-commerce.firebaseio.com',
  databaseURL: 'https://map-lab-utc2-e-commerce-default-
rtdb.firebaseio.com',
  storageBucket: 'map-lab-utc2-e-commerce.firebaseio.com',
);

static const FirebaseOptions android = FirebaseOptions(
  apiKey: 'AIzaSyA0QeNGSqyEFdGnBA7RGj1eX7eg8DsZWcW',
  appId: '1:233797330677:android:7a2085ef1aae5efd1547bc',
  messagingSenderId: '233797330677',
  projectId: 'map-lab-utc2-e-commerce',
  databaseURL: 'https://map-lab-utc2-e-commerce-default-
rtdb.firebaseio.com',
  storageBucket: 'map-lab-utc2-e-commerce.firebaseio.com',
);

static const FirebaseOptions ios = FirebaseOptions(
  apiKey: 'AIzaSyCXZUSrEE1I3At-sYipZmJ61BDY1EVVGA0',
  appId: '1:233797330677:ios:f64bcee6160714301547bc',
  messagingSenderId: '233797330677',
  projectId: 'map-lab-utc2-e-commerce',
  databaseURL: 'https://map-lab-utc2-e-commerce-default-
rtdb.firebaseio.com',
  storageBucket: 'map-lab-utc2-e-commerce.firebaseio.com',
  iosBundleId: 'com.example.drafProjectWeb',
);

static const FirebaseOptions macos = FirebaseOptions(
  apiKey: 'AIzaSyCXZUSrEE1I3At-sYipZmJ61BDY1EVVGA0',
  appId: '1:233797330677:ios:f64bcee6160714301547bc',
  messagingSenderId: '233797330677',
  projectId: 'map-lab-utc2-e-commerce',
  databaseURL: 'https://map-lab-utc2-e-commerce-default-
rtdb.firebaseio.com',
  storageBucket: 'map-lab-utc2-e-commerce.firebaseio.com',
  iosBundleId: 'com.example.drafProjectWeb',
);
}
```

3. Thiết kế và phát triển nhóm chức năng đăng nhập cho quản trị viên hệ thống

Trong file `libs/views/auth/login_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/auth_controller.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final usernameController = TextEditingController();
  final passwordController = TextEditingController();
  bool _isObscure = true;

  @override
  Widget build(BuildContext context) {
    final auth = context.read<AuthController>();

    return Scaffold(
      body: Container(
        // Tạo nền Gradient chuyển màu sinh động
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [Colors.blue.shade900, Colors.teal.shade400],
          ),
        ),
      child: Center(
        child: SingleChildScrollView(
          child: TweenAnimationBuilder(
            duration: const Duration(milliseconds: 800),
            tween: Tween<double>(begin: 0, end: 1),
            builder: (context, double value, child) {
              return Opacity(
                opacity: value,
                child: Transform.translate(
                  offset: Offset(0, 50 * (1 - value)),
                  child: child,
                ),
              );
            },
          ),
        child: Container(
```

```
width: 350,
padding: const EdgeInsets.all(30),
decoration: BoxDecoration(
  color: Colors.white.withOpacity(0.9),
  borderRadius: BorderRadius.circular(25),
  boxShadow: [
    BoxShadow(
      color: Colors.black.withOpacity(0.2),
      blurRadius: 20,
      spreadRadius: 5,
    ),
  ],
),
child: Column(
  mainAxisAlignment: MainAxisAlignment.min,
  children: [
    // Icon đại diện cho ngành Y tế/OCR
    Container(
      padding: const EdgeInsets.all(15),
      decoration: BoxDecoration(
        color: Colors.blue.shade50,
        shape: BoxShape.circle,
      ),
      child: Icon(
        Icons.medical_services_rounded,
        size: 50,
        color: Colors.blue.shade800,
      ),
    ),
    const SizedBox(height: 20),
    const Text(
      "ADMIN LOGIN",
      style: TextStyle(
        fontSize: 24,
        fontWeight: FontWeight.bold,
        letterSpacing: 1.5,
        color: Colors.black87,
      ),
    ),
    const SizedBox(height: 10),
    const Text(
      "Hệ thống nhận diện tên thuốc",
      style: TextStyle(color: Colors.grey),
    ),
    const SizedBox(height: 30),
    // TextField Tên đăng nhập
    TextField(
      controller: usernameController,
      decoration: InputDecoration(
```

```
    prefixIcon: const Icon(Icons.person_outline),
    labelText: "Username",
    filled: true,
    fillColor: Colors.grey.shade100,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(15),
      borderSide: BorderSide.none,
    ),
  ),
),
const SizedBox(height: 20),

// TextField Mật khẩu
TextField(
  controller: passwordController,
  obscureText: _isObscure,
  decoration: InputDecoration(
    prefixIcon: const Icon(Icons.lock_outline),
    suffixIcon: IconButton(
      icon: Icon(
        _isObscure
          ? Icons.visibility
          : Icons.visibility_off,
      ),
      onPressed: () =>
        setState(() => _isObscure = !_isObscure),
    ),
    labelText: "Password",
    filled: true,
    fillColor: Colors.grey.shade100,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(15),
      borderSide: BorderSide.none,
    ),
  ),
),
const SizedBox(height: 30),

// Nút Login với hiệu ứng Gradient
SizedBox(
  width: double.infinity,
  height: 55,
  child: ElevatedButton(
    onPressed: () async {
      final success = await auth.login(
        usernameController.text,
        passwordController.text,
      );

      if (!success) {
```

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    backgroundColor: Colors.redAccent,
    behavior: SnackBarBehavior.floating,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ),
    content: const Row(
      children: [
        Icon(
          Icons.error_outline,
          color: Colors.white,
        ),
        SizedBox(width: 10),
        Text("Tài khoản hoặc mật khẩu không  
đúng"),
      ],
    ),
  ),
);
},
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.blue.shade800,
  foregroundColor: Colors.white,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(15),
  ),
  elevation: 5,
),
child: const Text(
  "ĐĂNG NHẬP",
  style: TextStyle(
    fontSize: 16,
    fontWeight: FontWeight.bold,
  ),
),
),
const SizedBox(height: 20),
TextButton(
  onPressed: () {},
  child: const Text("Quên mật khẩu?"),
),
],
),
),
),
),
),
```

```
    ),  
  );  
}  
}
```

Trong terminal của project, sinh viên chạy lệnh sau để thêm vào thư viện cần thiết:

```
flutter pub add provider
```

Trong file `libs/controllers/auth_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';  
import '../data/services/auth_service.dart';  
  
class AuthController extends ChangeNotifier {  
  final AuthService _service = AuthService();  
  
  bool _isLoggedIn = false;  
  bool get isLoggedIn => _isLoggedIn;  
  
  Future<void> checkLogin() async {  
    _isLoggedIn = await _service.isLoggedIn();  
    notifyListeners();  
  }  
  
  Future<bool> login(String username, String password) async {  
    final success = await _service.login(username, password);  
    _isLoggedIn = success;  
    notifyListeners();  
    return success;  
  }  
  
  Future<void> logout() async {  
    await _service.logout();  
    _isLoggedIn = false;  
    notifyListeners();  
  }  
}
```

Trong file `libs/data/services/auth_service.dart`, soạn mã nguồn có nội dung như sau *(tài khoản đăng nhập vào hệ thống cho admin được thiết lập mặc định là admin/123@456)*:

```
import 'package:shared_preferences/shared_preferences.dart';  
  
class AuthService {  
  static const _key = "is_logged_in";  
  
  Future<bool> login(String username, String password) async {
```

```
    if (username == "admin" && password == "123@456") {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setBool(_key, true);
      return true;
    }
    return false;
  }

Future<void> logout() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.remove(_key);
}

Future<bool> isLoggedIn() async {
  final prefs = await SharedPreferences.getInstance();
  return prefs.getBool(_key) ?? false;}}}
```

Trong terminal của project, sinh viên chạy lệnh sau để thêm vào thư viện cần thiết:

- flutter pub add shared_preferences
- flutter pub add flutter_localizations --sdk=flutter
- flutter pub add flutter_quill

Trong file `libs/main.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_quill/flutter_quill.dart';

import 'package:provider/provider.dart';

import 'firebase_options.dart';

import 'controllers/auth_controller.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
```

```

        ChangeNotifierProvider(create: (_) =>
AuthController()..checkLogin()),
    ],
    child: Builder(
      builder: (context) {
        final auth = context.watch<AuthController>();

        return MaterialApp.router(
          debugShowCheckedModeBanner: false,

          // ADD THIS BLOCK
          localizationsDelegates: const [
            FlutterQuillLocalizations.delegate, // FIX LỖI
            GlobalMaterialLocalizations.delegate,
            GlobalWidgetsLocalizations.delegate,
            GlobalCupertinoLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en'), Locale('vi')],
        );
      },
    ),
  );
}

```

Sau khi người dùng đăng nhập thành công, hệ thống sẽ điều hướng người dùng sang màn hình quản trị chung, chúng ta sẽ chuẩn bị mã nguồn giao diện tạm thời gồm các file mã nguồn sau:

- Trong file `lib/views/layout/admin_appbar.dart`, soạn mã nguồn có nội dung như sau:

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/auth_controller.dart';

class AdminAppBar extends StatelessWidget implements PreferredSizeWidget {
  const AdminAppBar({super.key});

  @override
  Widget build(BuildContext context) {
    final auth = context.read<AuthController>();

    return AppBar(
      backgroundColor: Colors.white,
      elevation: 0, // Bỏ bóng đổ mặc định
      automaticallyImplyLeading: false,
      titleSpacing: 24,
      // Thêm một đường kẻ mảnh ở dưới AppBar
      bottom: PreferredSize(

```



```

        preferredSize: const Size.fromHeight(1.0),
        child: Container(color: Colors.grey.withOpacity(0.2), height:
1.0),
    ),
    title: Row(
      children: [
        /// SEARCH BAR - Làm bo tròn và chuyên nghiệp hơn
        Expanded(
          flex: 2,
          child: Container(
            height: 42,
            decoration: BoxDecoration(
              color: Colors.grey[50],
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color: Colors.grey[200]!),
            ),
            child: TextField(
              decoration: InputDecoration(
                hintText: "Tìm kiếm hệ thống...",
                hintStyle: TextStyle(color: Colors.grey[400]),
fontSize: 14),
                prefixIcon: Icon(
                  Icons.search,
                  color: Colors.grey[400],
                  size: 20,
                ),
                border: InputBorder.none,
                contentPadding: const EdgeInsets.symmetric(vertical:
10),
              ),
            ),
          ),
        ),
      ],
    ),
    const Spacer(flex: 1), // Tạo khoảng trống giữa search và
icons
    ///ACTIONS GROUP
    _buildActionButton(Icons.language, "Ngôn ngữ", () {}),
    _buildActionButton(
      Icons.notifications_none_outlined,
      "Thông báo",
      () {},
    ),
    _buildActionButton(Icons.shopping_cart_outlined, "Đơn hàng",
() {}),
    _buildActionButton(Icons.settings_outlined, "Cài đặt", () {}),

    const VerticalDivider(indent: 15, endIndent: 15, width: 40),

    /// USER INFO SECTION

```

```
PopupMenuButton<String>(  
  offset: const Offset(0, 55),  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(12),  
  ),  
  child: MouseRegion(  
    cursor: SystemMouseCursors.click,  
    child: Row(  
      children: [  
        Column(  
          crossAxisAlignment: CrossAxisAlignment.end,  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: const [  
            Text(  
              "User Admin",  
              style: TextStyle(  
                fontWeight: FontWeight.bold,  
                fontSize: 14,  
                color: Colors.black87,  
              ),  
            ),  
            Text(  
              "Quản trị viên",  
              style: TextStyle(  
                fontSize: 11,  
                color: Colors.blueAccent,  
              ),  
            ),  
          ],  
        ),  
        const SizedBox(width: 12),  
        Container(  
          decoration: BoxDecoration(  
            shape: BoxShape.circle,  
            border: Border.all(  
              color: Colors.blue.withOpacity(0.2),  
              width: 2,  
            ),  
          ),  
          child: const CircleAvatar(  
            radius: 18,  
            backgroundColor: Colors.blue,  
            child: Text(  
              "UA",  
              style: TextStyle(color: Colors.white, fontSize:  
12),  
            ),  
          ),  
        ),  
      ],  
    ),  
    const Icon(Icons.arrow_drop_down, color: Colors.grey),  
  ),  
),
```

```
        ],
      ),
    ),
    itemBuilder: (context) => [
      _buildPopupItem(
        "profile",
        Icons.person_outline,
        "Thông tin cá nhân",
      ),
      _buildPopupItem("settings", Icons.security, "Bảo mật"),
      const PopupMenuDivider(),
      _buildPopupItem(
        "logout",
        Icons.logout,
        "Đăng xuất",
        color: Colors.red,
      ),
    ],
    onSelect: (value) {
      if (value == "logout") {
        auth.logout();
      }
    },
  ),
],
),
);
}

/// Widget hỗ trợ tạo Icon Button đẹp hơn
Widget _buildActionButton(IconData icon, String tooltip, VoidCallback
onTap) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 4),
    child: Tooltip(
      message: tooltip,
      child: IconButton(
        onPressed: onTap,
        icon: Icon(icon, color: Colors.black54, size: 22),
        hoverColor: Colors.blue.withOpacity(0.05),
        splashRadius: 22,
      ),
    ),
  );
}

/// Widget hỗ trợ tạo Item Menu đẹp hơn
PopupMenuItem<String> _buildPopupItem(
  String value,
  IconData icon,
```

```
String title, {
  Color? color,
}) {
  return PopupMenuItem(
    value: value,
    child: Row(
      children: [
        Icon(icon, size: 18, color: color ?? Colors.black54),
        const SizedBox(width: 12),
        Text(title, style: TextStyle(color: color, fontSize: 14)),
      ],
    ),
  );
}

@override
Size get preferredSize => const Size.fromHeight(65); // Tăng chiều cao
lên một chút cho thoáng
}
```

- Trong file `lib/views/layout/admin_layout.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'sidebar.dart';
import 'admin_appbar.dart';

class AdminLayout extends StatefulWidget {
  final Widget child;
  final Function(String) onNavigate;
  final String currentRoute;
  const AdminLayout({
    super.key,
    required this.child,
    required this.onNavigate,
    required this.currentRoute,
  });
  @override
  State<AdminLayout> createState() => _AdminLayoutState();
}

class _AdminLayoutState extends State<AdminLayout> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: const AdminAppBar(),
      body: Row(
        children: [
          Sidebar(
```

```
        onNavigate: widget.onNavigate,  
        currentRoute: widget.currentRoute,  
      ),  
      Expanded(  
        child: Container(  
          color: Colors.grey[100],  
          padding: const EdgeInsets.all(24),  
          child: widget.child,  
        ),  
      ),  
    ],  
  ),  
);  
}
```

- Trong file `lib/views/layout/sidebar.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';  
  
class Sidebar extends StatefulWidget {  
  final Function(String) onNavigate;  
  final String currentRoute;  
  
  const Sidebar({  
    super.key,  
    required this.onNavigate,  
    this.currentRoute = '/dashboard',  
  });  
  
  @override  
  State<Sidebar> createState() => _SidebarState();  
}  
  
class _SidebarState extends State<Sidebar> {  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: 260,  
      decoration: BoxDecoration(  
        color: const Color(0xFF1E1E2D), // Slate Navy chuyên nghiệp  
        boxShadow: [  
          BoxShadow(  
            color: Colors.black.withOpacity(0.1),  
            blurRadius: 10,  
            offset: const Offset(2, 0),  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```

    ),
    child: Column(
      children: [
        /// LOGO SECTION
        Container(
          padding: const EdgeInsets.symmetric(vertical: 30,
horizontal: 20),
          alignment: Alignment.center,
          child: Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Container(
                padding: const EdgeInsets.all(8),
                decoration: BoxDecoration(
                  color: Colors.blueAccent,
                  borderRadius: BorderRadius.circular(10),
                ),
                child: const Icon(Icons.bolt, color: Colors.white,
size: 24),
              ),
              const SizedBox(width: 12),
              const Flexible(
                child: Text(
                  "SHOE ADMIN",
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 20,
                    fontWeight: FontWeight.w800,
                    letterSpacing: 1.2,
                  ),
                overflow: TextOverflow.ellipsis,
              ),
            ],
          ),
        ),
        const Padding(
          padding: EdgeInsets.symmetric(horizontal: 20),
          child: Divider(color: Colors.white10, thickness: 1),
        ),
        const SizedBox(height: 10),

        /// MENU ITEMS
        Expanded(
          child: ListView(
            padding: const EdgeInsets.symmetric(horizontal: 12),
            children: [
              _buildSectionTitle("MAIN MENU"),
              _item(
                Icons.dashboard_outlined,

```

```

        "Bảng điều khiển",
        "/dashboard",
    ),
    _item(Icons.category, "Danh mục sản phẩm",
"/categories"),
    _item(
        Icons.info_outline,
        "Danh mục thuộc tính sản phẩm",
        "/attributes",
    ),
    _item(
        Icons.branding_watermark_outlined,
        "Danh mục thương hiệu",
        "/brands",
    ),
    _item(Icons.card_giftcard, "Danh mục mã giảm giá",
"/coupons"),
    _item(Icons.inventory_2_outlined, "Sản phẩm",
"/products"),

    _buildSectionTitle("SALES"),
    _item(Icons.shopping_cart_outlined, "Đơn hàng",
"/orders"),
    _item(Icons.people_outline, "Khách hàng", "/customers"),
    _item(Icons.star_rate_outlined, "Đánh giá", "/reviews"),

    _buildSectionTitle("SYSTEM"),
    _item(Icons.settings_outlined, "Cài đặt cá nhân",
"/settings"),
    ],
    ),
    ),
    ),

    /// FOOTER
    const Padding(
        padding: EdgeInsets.all(20),
        child: Text(
            "v1.0.2",
            style: TextStyle(color: Colors.white24, fontSize: 12),
        ),
    ),
    ],
    ),
    );
}

/// Widget tiêu đề nhóm menu
Widget _buildSectionTitle(String title) {
    return Padding(
        padding: const EdgeInsets.only(left: 16, top: 20, bottom: 8),
    );
}

```

```

        child: Text(
          title,
          style: const TextStyle(
            color: Colors.white38,
            fontSize: 11,
            fontWeight: FontWeight.bold,
            letterSpacing: 1.1,
          ),
        ),
      );
    }

    /// Widget Item Menu với hiệu ứng Hover và Active
    Widget _item(IconData icon, String title, String route) {
      final bool isActive = widget.currentRoute == route;

      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 2),
        child: InkWell(
          onTap: () => widget.onNavigate(route),
          borderRadius: BorderRadius.circular(10),
          child: AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical:
12),
            decoration: BoxDecoration(
              color: isActive
                ? Colors.blueAccent.withOpacity(0.15)
                : Colors.transparent,
              borderRadius: BorderRadius.circular(10),
              border: isActive
                ? const Border(
                    left: BorderSide(color: Colors.blueAccent, width:
4),
                    : null,
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  color: isActive ? Colors.blueAccent : Colors.white60,
                  size: 22,
                ),
                const SizedBox(width: 15),
                // FIX LỖI OVERFLOW Ở ĐÂY: Dùng Expanded để Text không đẩy
                Row ra ngoài
                Expanded(
                  child: Text(
                    title,

```



```

        style: TextStyle(
          color: isActive ? Colors.white : Colors.white60,
          fontSize: 15,
          fontWeight: isActive ? FontWeight.w600 :
FontWeight.w400,
        ),
        maxLines: 1,
        overflow: TextOverflow.ellipsis, // Hiện dấu ... nếu
quá dài
      ),
    ),
    if (isActive) ...[
      const SizedBox(width: 10),
      Container(
        width: 6,
        height: 6,
        decoration: const BoxDecoration(
          color: Colors.blueAccent,
          shape: BoxShape.circle,
        ),
      ),
    ],
  ],
),
),
),
);
}
}

```

- Trong file `lib/routes/app_routes.dart`, soạn mã nguồn có nội dung như sau:

```

import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../controllers/auth_controller.dart';

class AppRouterDelegate extends RouterDelegate<String>
    with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
  final GlobalKey<NavigatorState> navigatorKey;
  final AuthController authController;
  String _currentPath = "/login";
  AppRouterDelegate(this.authController)
    : navigatorKey = GlobalKey<NavigatorState>() {
    authController.addListener(notifyListeners);
  }
  @override
  String? get currentConfiguration => _currentPath;
  @override
  Widget build(BuildContext context) {

```

```
if (!authController.isLoggedIn) {
    return Navigator(
        key: navigatorKey,
        pages: const [MaterialPage(child: LoginPage())],
        onPopPage: (route, result) => route.didPop(result),
    );
}
Widget page;
switch (_currentPath) {
    default:
        page = const Center(child: Text("Dashboard Page"));
}
return Navigator(
    key: navigatorKey,
    pages: [
        MaterialPage(
            child: AdminLayout(
                currentRoute: _currentPath, // truyền route hiện tại
                onNavigate: (path) {
                    _currentPath = path;
                    notifyListeners();
                },
                child: page,
            ),
        ),
    ],
    onPopPage: (route, result) => route.didPop(result),
);
}

@override
Future<void> setNewRoutePath(String configuration) async {
    _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
    @override
    Future<String> parseRouteInformation(
        RouteInformation routeInformation,
    ) async {
        return routeInformation.location ?? "/login";
    }

    @override
    RouteInformation restoreRouteInformation(String configuration) {
        return RouteInformation(location: configuration);
    }
}
```

4. Thiết kế và phát triển nhóm chức năng quản lý danh mục sản phẩm

Trong file `lib/views/categories/category_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/category_controller.dart';
import '../data/models/category_model.dart';

class CategoriesPage extends StatelessWidget {
  const CategoriesPage({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => CategoryController()..fetchCategories(),
      child: Scaffold(
        backgroundColor: const Color(0xFFF8F9FA), // Màu nền nhẹ nhàng
        body: const _CategoriesView(),
      ),
    );
  }
}

class _CategoriesView extends StatelessWidget {
  const _CategoriesView();

  @override
  Widget build(BuildContext context) {
    final controller = context.watch<CategoryController>();

    return Padding(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Header Section
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const Text(
                "Product Categories",
                style: TextStyle(
                  fontSize: 28,
                  fontWeight: FontWeight.bold,
                  color: Color(0xFF2D3436),
                ),
              ),
              ElevatedButton.icon(
```

```
onPressed: () => _showDialog(context),
icon: const Icon(Icons.add, size: 18),
label: const Text("ADD CATEGORY"),
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.indigoAccent,
  foregroundColor: Colors.white,
  padding: const EdgeInsets.symmetric(
    horizontal: 20,
    vertical: 15,
  ),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(10),
  ),
  elevation: 2,
),
),
],
),
const SizedBox(height: 24),

// Search Bar Section
Container(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.05),
        blurRadius: 10,
      ),
    ],
  ),
  child: TextField(
    onChanged: controller.search,
    decoration: const InputDecoration(
      hintText: "Search categories by name...",
      prefixIcon: Icon(Icons.search, color: Colors.grey),
      border: InputBorder.none,
    ),
  ),
),
const SizedBox(height: 24),

// Table Content
Expanded(
  child: Container(
    width: double.infinity,
    decoration: BoxDecoration(
      color: Colors.white,
```

```

borderRadius: BorderRadius.circular(16),
boxShadow: [
  BoxShadow(
    color: Colors.black.withOpacity(0.05),
    blurRadius: 15,
  ),
],
),
child: controller.isLoading
  ? const Center(child: CircularProgressIndicator())
  : ClipRRect(
    borderRadius: BorderRadius.circular(16),
    child: SingleChildScrollView(
      child: DataTable(
        headingRowColor: MaterialStateProperty.all(
          const Color(0xFFF1F3F5),
        ),
        dataRowHeight: 70,
        horizontalMargin: 20,
        columns: const [
          DataColumn(
            label: Text(
              "SEQ",
              style: TextStyle(fontWeight:
FontWeight.bold),
            ),
          ),
          DataColumn(
            label: Text(
              "CATEGORY",
              style: TextStyle(fontWeight:
FontWeight.bold),
            ),
          ),
          DataColumn(
            label: Text(
              "FEATURED",
              style: TextStyle(fontWeight:
FontWeight.bold),
            ),
          ),
          DataColumn(
            label: Text(
              "STATUS",
              style: TextStyle(fontWeight:
FontWeight.bold),
            ),
          ),
          DataColumn(
            label: Text(

```

```

        "UPDATED",
        style: TextStyle(fontWeight:
Flutter DataTable
FontWeight.bold),
    ),
    ),
    DataColumn(
        label: Text(
            "ACTIONS",
            style: TextStyle(fontWeight:
FontWeight.bold),
        ),
    ),
    ],
    rows:
controller.paginatedData.asMap().entries.map((
    entry,
) {
    final index = entry.key;
    final c = entry.value;
    return DataRow(
        // Hiệu ứng rê chuột (Hover) mặc định của
Flutter DataTable
        onSelectChanged: (selected) {},
        cells: [
            DataColumn(
                Text(
                    "${(controller.currentPage - 1) *
controller.rowsPerPage + index + 1}",
                ),
            ),
            DataColumn(
                Row(
                    children: [
                        CircleAvatar(
                            radius: 20,
                            backgroundColor: Colors.grey[200],
                            backgroundImage:
c.imageUrl.isNotEmpty
                                ? NetworkImage(c.imageUrl)
                                : null,
                        child: c.imageUrl.isEmpty
                            ? const Icon(Icons.image, size:
20)
                            : null,
                    ),
                    const SizedBox(width: 12),
                    Text(
                        c.name,
                        style: const TextStyle(
                            fontWeight: FontWeight.w600,

```

```

        ),
      ),
    ],
  ),
),
DataCell(
  Icon(
    c.isFeatured
      ? Icons.star_rounded
      : Icons.star_outline_rounded,
    color: c.isFeatured
      ? Colors.amber
      : Colors.grey,
  ),
),
DataCell(_buildStatusBadge(c.isActive)),
DataCell(
  Text(
    c.updatedAt != null
      ?
"$${c.updatedAt!.day.toString().padLeft(2,
'0')}/${c.updatedAt!.month.toString().padLeft(2, '0')}/${c.updatedAt!.year}"
      : "-",
  ),
),
DataCell(
  Row(
    children: [
      _buildActionButton(
        Icons.edit_outlined,
        Colors.blue,
        () => _showDialog(context, category:
c),
      ),
      const SizedBox(width: 8),
      _buildActionButton(
        Icons.delete_outline,
        Colors.redAccent,
        () => _confirmDelete(context, c.id),
      ),
    ],
  ),
),
],
);
}).toList(),
),
),
),
),
),

```

```

    ),

    // Pagination Section
    const SizedBox(height: 20),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: List.generate(controller.totalPages, (index) {
        final page = index + 1;
        final isSelected = controller.currentPage == page;
        return AnimatedContainer(
          duration: const Duration(milliseconds: 300),
          margin: const EdgeInsets.symmetric(horizontal: 4),
          child: ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: isSelected
                ? Colors.indigoAccent
                : Colors.white,
              foregroundColor: isSelected ? Colors.white :
Colors.black87,
              elevation: isSelected ? 4 : 0,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(8),
                side: BorderSide(
                  color: isSelected
                    ? Colors.transparent
                    : Colors.grey[300]!,
                ),
              ),
            ),
            onPressed: () => controller.changePage(page),
            child: Text("$page"),
          ),
        ),
      ),
    ),
  ],
),
);
}

// Widget Badge Trạng thái
Widget _buildStatusBadge(bool isActive) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
    decoration: BoxDecoration(
      color: isActive
        ? Colors.green.withOpacity(0.1)
        : Colors.red.withOpacity(0.1),
      borderRadius: BorderRadius.circular(20),
      border: Border.all(

```



```
        color: isActive ? Colors.green : Colors.red,
        width: 0.5,
      ),
    ),
    child: Text(
      isActive ? "Active" : "Inactive",
      style: TextStyle(
        color: isActive ? Colors.green[700] : Colors.red[700],
        fontSize: 12,
        fontWeight: FontWeight.bold,
      ),
    ),
  );
}

// Widget Nút Action tròn
Widget _buildActionButton(IconData icon, Color color, VoidCallback onTap) {
  return InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(8),
    child: Container(
      padding: const EdgeInsets.all(8),
      decoration: BoxDecoration(
        color: color.withOpacity(0.1),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Icon(icon, size: 20, color: color),
    ),
  );
}

// Các hàm Dialog giữ nguyên logic nhưng bọc trong UI mới
void _confirmDelete(BuildContext context, String id) {
  showDialog(
    context: context,
    builder: (_) => AlertDialog(
      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(15)),
      title: const Text("Confirm Delete"),
      content: const Text("This action cannot be undone. Are you sure?"),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text("Cancel"),
        ),
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          onPressed: () async {
            await context.read<CategoryController>().delete(id);
            Navigator.pop(context);
          },
        ),
      ],
    ),
  );
}
```

```

        },
        child: const Text("Delete"),
      ),
    ],
  ),
);
}

void _showDialog(BuildContext context, {CategoryModel? category}) {
  final nameController = TextEditingController(text: category?.name ?? "");
  final imageController = TextEditingController(
    text: category?.imageUrl ?? "",
  );
  bool isActive = category?.isActive ?? true;
  bool isFeatured = category?.isFeatured ?? false;

  showDialog(
    context: context,
    builder: (_) => StatefulBuilder(
      builder: (context, setState) => AlertDialog(
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(20),
        ),
        title: Text(category == null ? "New Category" : "Edit Category"),
        content: SizedBox(
          width: 450,
          child: SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                TextField(
                  controller: nameController,
                  decoration: InputDecoration(
                    labelText: "Category Name",
                    prefixIcon: const Icon(Icons.label_outline),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(10),
                    ),
                  ),
                ),
                const SizedBox(height: 20),
                TextField(
                  controller: imageController,
                  onChanged: (_) => setState(() {}),
                  decoration: InputDecoration(
                    labelText: "Image URL",
                    prefixIcon: const Icon(Icons.link),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(10),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}

```

```

    ),
  ),
  const SizedBox(height: 15),
  if (imageController.text.isNotEmpty)
    ClipRRect(
      borderRadius: BorderRadius.circular(10),
      child: Image.network(
        imageController.text,
        height: 150,
        width: double.infinity,
        fit: BoxFit.cover,
        errorBuilder: (context, error, stackTrace) =>
          const Icon(Icons.broken_image, size: 50),
      ),
    ),
  const SizedBox(height: 10),
  SwitchListTile(
    value: isActive,
    title: const Text("Active Status"),
    activeColor: Colors.indigoAccent,
    onChanged: (v) => setState(() => isActive = v),
  ),
  SwitchListTile(
    value: isFeatured,
    title: const Text("Featured Category"),
    activeColor: Colors.orange,
    onChanged: (v) => setState(() => isFeatured = v),
  ),
],
),
),
),
actions: [
  TextButton(
    onPressed: () => Navigator.pop(context),
    child: const Text("Cancel"),
  ),
  ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.indigoAccent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(8),
      ),
    ),
    onPressed: () async {
      final newCategory = CategoryModel(
        id: category?.id ?? "",
        name: nameController.text,
        imageURL: imageController.text,
        isActive: isActive,

```

```
        isFeatured: isFeatured,
        priority: 0,
        numberOfProducts: 0,
        viewCount: 0,
        createdBy: "admin",
        updatedBy: "admin",
        createdAt: category?.createdAt ?? DateTime.now(),
        updatedAt: DateTime.now(),
    );
    if (category == null) {
      await context.read<CategoryController>().add(newCategory);
    } else {
      await
context.read<CategoryController>().update(newCategory);
    }
    Navigator.pop(context);
  },
  child: const Text(
    "Save Changes",
    style: TextStyle(color: Colors.white),
  ),
),
],
),
),
);
}
}
```

Trong file `lib/controllers/category_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/category_model.dart';
import '../data/services/category_service.dart';

class CategoryController extends ChangeNotifier {
  final CategoryService _service = CategoryService();

  List<CategoryModel> categories = [];
  List<CategoryModel> filtered = [];

  bool isLoading = false;

  Future<void> fetchCategories() async {
    isLoading = true;
    notifyListeners();

    categories = await _service.getCategories();
    filtered = categories;
  }
}
```

```
        isLoading = false;
        notifyListeners();
    }

    void search(String keyword) {
        if (keyword.isEmpty) {
            filtered = categories;
        } else {
            filtered = categories
                .where((c) => c.name.toLowerCase().contains(keyword.toLowerCase()))
                .toList();
        }
        notifyListeners();
    }

    Future<void> add(CategoryModel category) async {
        await _service.addCategory(category);
        await fetchCategories();
    }

    Future<void> update(CategoryModel category) async {
        await _service.updateCategory(category);
        await fetchCategories();
    }

    Future<void> delete(String id) async {
        await _service.deleteCategory(id);
        await fetchCategories();
    }

    int currentPage = 1;
    int rowsPerPage = 5;

    List<CategoryModel> get paginatedData {
        final start = (currentPage - 1) * rowsPerPage;
        final end = start + rowsPerPage;
        return filtered.sublist(
            start,
            end > filtered.length ? filtered.length : end,
        );
    }

    int get totalPages => (filtered.length / rowsPerPage).ceil();

    void changePage(int page) {
        currentPage = page;
        notifyListeners();
    }
}
```

Trong file `lib/data/models/category_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

class CategoryModel {
  String id;
  String name;
  String imageURL;
  bool isActive;
  bool isFeatured;
  int priority;
  DateTime? createdAt;
  DateTime? updatedAt;
  int numberOfProducts;
  int viewCount;
  String createdBy;
  String updatedBy;

  CategoryModel({
    required this.id,
    required this.name,
    required this.imageURL,
    required this.isActive,
    required this.isFeatured,
    required this.priority,
    this.createdAt,
    this.updatedAt,
    required this.numberOfProducts,
    required this.viewCount,
    required this.createdBy,
    required this.updatedBy,
  });

  factory CategoryModel.fromFirestore(DocumentSnapshot doc) {
    final data = doc.data() as Map<String, dynamic>;

    return CategoryModel(
      id: doc.id,
      name: data['name'] ?? '',
      imageURL: data['imageURL'] ?? '',
      isActive: data['isActive'] ?? true,
      isFeatured: data['isFeatured'] ?? false,
      priority: data['priority'] ?? 0,
      createdAt: (data['createdAt'] as Timestamp?)?.toDate(),
      updatedAt: (data['updatedAt'] as Timestamp?)?.toDate(),
      numberOfProducts: data['numberOfProducts'] ?? 0,
      viewCount: data['viewCount'] ?? 0,
      createdBy: data['createdBy'] ?? '',
    );
  }
}
```

```
        updatedBy: data['updatedBy'] ?? '',
    );
}

Map<String, dynamic> toMap() {
    return {
        'name': name,
        'imageUrl': imageUrl,
        'isActive': isActive,
        'isFeatured': isFeatured,
        'priority': priority,
        'createdAt': createdAt,
        'updatedAt': updatedAt,
        'numberOfProducts': numberOfProducts,
        'viewCount': viewCount,
        'createdBy': createdBy,
        'updatedBy': updatedBy,
    };
}
```

Trong terminal của project, sinh viên chạy lệnh sau để thêm vào thư viện cần thiết:

```
flutter pub add cloud_firestore
```

Trong file `lib/data/services/category_service.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/category_model.dart';

class CategoryService {
    final _firestore = FirebaseFirestore.instance;
    final _collection = "categories";

    Future<List<CategoryModel>> getCategories() async {
        final snapshot = await _firestore.collection(_collection).get();

        return snapshot.docs
            .map((doc) => CategoryModel.fromFirestore(doc))
            .toList();
    }

    Future<void> addCategory(CategoryModel category) async {
        await _firestore.collection(_collection).add(category.toMap());
    }

    Future<void> updateCategory(CategoryModel category) async {
        await _firestore
            .collection(_collection)
```

```
        .doc(category.id)
        .update(category.toMap());
    }

    Future<void> deleteCategory(String id) async {
        await _firestore.collection(_collection).doc(id).delete();
    }
}
```

Trong file `lib/main.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/controllers/category_controller.dart';
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_quill/flutter_quill.dart';
import 'package:provider/provider.dart';
import 'firebase_options.dart';
import 'controllers/auth_controller.dart';
import 'routes/app_routes.dart';

void main() async {
    WidgetsFlutterBinding.ensureInitialized();
    await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);

    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return MultiProvider(
            providers: [
                ChangeNotifierProvider(create: (_) =>
AuthController()..checkLogin()),
                ChangeNotifierProvider(create: (_) => CategoryController()),
            ],
            child: Builder(
                builder: (context) {
                    final auth = context.watch<AuthController>();

                    return MaterialApp.router(
                        debugShowCheckedModeBanner: false,

                        // ADD THIS BLOCK
                        localizationsDelegates: const [
                            FlutterQuillLocalizations.delegate, // FIX LỖI
```



```
        GlobalMaterialLocalizations.delegate,  
        GlobalWidgetsLocalizations.delegate,  
        GlobalCupertinoLocalizations.delegate,  
    ],  
    supportedLocales: const [Locale('en'), Locale('vi')],  
  
    routerDelegate: AppRouterDelegate(auth),  
    routeInformationParser: AppRouteParser(),  
  );  
},  
),  
);  
}
```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/cagetories/category_page.dart';  
import 'package:flutter/material.dart';  
import '../views/auth/login_page.dart';  
import '../views/layout/admin_layout.dart';  
import '../controllers/auth_controller.dart';  
  
class AppRouterDelegate extends RouterDelegate<String>  
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {  
  final GlobalKey<NavigatorState> navigatorKey;  
  final AuthController authController;  
  String _currentPath = "/login";  
  AppRouterDelegate(this.authController)  
    : navigatorKey = GlobalKey<NavigatorState>() {  
    authController.addListener(notifyListeners);  
  }  
  @override  
  String? get currentConfiguration => _currentPath;  
  @override  
  Widget build(BuildContext context) {  
    if (!authController.isLoggedIn) {  
      return Navigator(  
        key: navigatorKey,  
        pages: const [MaterialPage(child: LoginPage())],  
        onPopPage: (route, result) => route.didPop(result),  
      );  
    }  
    Widget page;  
    switch (_currentPath) {  
      case "/categories":  
        page = const CategoriesPage();  
        break;  
      default:
```

```
        page = const Center(child: Text("Dashboard Page"));
    }
    return Navigator(
      key: navigatorKey,
      pages: [
        MaterialPage(
          child: AdminLayout(
            currentRoute: _currentPath, // truyền route hiện tại
            onNavigate: (path) {
              _currentPath = path;
              notifyListeners();
            },
            child: page,
          ),
        ),
      ],
      onPopPage: (route, result) => route.didPop(result),
    );
  }

  @override
  Future<void> setNewRoutePath(String configuration) async {
    _currentPath = configuration;
  }
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

5. Thiết kế và phát triển nhóm chức năng quản lý danh mục thuộc tính sản phẩm

Trong file `lib/views/attributes/attribute_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/attribute_controller.dart';
import '../data/services/attribute_service.dart';
```

```
import 'attribute_add_edit_page.dart';

class AttributesPage extends StatelessWidget {
  const AttributesPage({super.key});

  @override
  Widget build(BuildContext context) {
    final service = AttributeService();

    return ChangeNotifierProvider(
      create: (_) => AttributeController(),
      child: Scaffold(
        backgroundColor: const Color(0xFFFF3F4F9), // Nền xám xanh nhẹ hiện
        đại
        body: Consumer<AttributeController>(
          builder: (context, controller, _) {
            return StreamBuilder(
              stream: service.getAll(),
              builder: (context, snapshot) {
                if (snapshot.hasData) {
                  controller.setData(snapshot.data!);
                }

                return Padding(
                  padding: const EdgeInsets.all(24.0),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      // --- HEADER ---
                      const Text(
                        "Product Attributes",
                        style: TextStyle(
                          fontSize: 26,
                          fontWeight: FontWeight.bold,
                          color: Color(0xFF1A1C24),
                        ),
                      ),
                      const SizedBox(height: 20),
                      Row(
                        children: [
                          Expanded(
                            child: Container(
                              decoration: BoxDecoration(
                                color: Colors.white,
                                borderRadius: BorderRadius.circular(15),
                                boxShadow: [
                                  BoxShadow(
                                    color: Colors.black.withOpacity(0.03),
                                    blurRadius: 10,
```

```
        offset: const Offset(0, 4),
      ),
    ],
  ),
  child: TextField(
    onChanged: controller.search,
    decoration: InputDecoration(
      hintText: "Search by name or value...",
      prefixIcon: const Icon(
        Icons.search_rounded,
        color: Colors.indigo,
      ),
      border: InputBorder.none,
      contentPadding: const EdgeInsets.symmetric(
        vertical: 15,
      ),
    ),
  ),
),
),
const SizedBox(width: 16),
ElevatedButton.icon(
  onPressed: () => Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => const AttributeFormPage(),
    ),
  ),
  icon: const Icon(
    Icons.add_rounded,
    color: Colors.white,
  ),
  label: const Text("NEW ATTRIBUTE"),
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.indigoAccent,
    foregroundColor: Colors.white,
    padding: const EdgeInsets.symmetric(
      horizontal: 24,
      vertical: 20,
    ),
  ),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(12),
  ),
  elevation: 2,
),
),
],
),

const SizedBox(height: 24),
```

```
// --- TABLE CONTAINER ---
Expanded(
  child: Container(
    width: double.infinity,
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(20),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.04),
          blurRadius: 20,
          offset: const Offset(0, 8),
        ),
      ],
    ),
  child: ClipRRect(
    borderRadius: BorderRadius.circular(20),
    child: SingleChildScrollView(
      child: DataTable(
        headingRowColor: MaterialStateProperty.all(
          Colors.indigo.withOpacity(0.05),
        ),
        dataRowHeight: 75,
        horizontalMargin: 24,
        columns: const [
          DataColumn(
            label: Text(
              "SEQ",
              style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
              ),
            ),
          ),
          DataColumn(
            label: Text(
              "ATTRIBUTE",
              style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
              ),
            ),
          ),
          DataColumn(
            label: Text(
              "VALUES",
              style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
              ),
            ),
          ),
        ],
      ),
    ),
  ),
)
```

```
        ),
    ),
),
DataColumn(
    label: Text(
        "STATUS",
        style: TextStyle(
            fontWeight: FontWeight.bold,
            color: Colors.indigo,
        ),
    ),
),
DataColumn(
    label: Text(
        "UPDATED",
        style: TextStyle(
            fontWeight: FontWeight.bold,
            color: Colors.indigo,
        ),
    ),
),
DataColumn(
    label: Text(
        "ACTIONS",
        style: TextStyle(
            fontWeight: FontWeight.bold,
            color: Colors.indigo,
        ),
    ),
),
],
rows:
List.generate(controller.paginatedData.length, (
    index,
) {
    final item =
controller.paginatedData[index];
    return DataRow(
        onSelectChanged:
            (_) {}, // Tạo hiệu ứng hover nhẹ
        cells: [
            DataCell(
                Text(
                    "#${index + 1} +
(controller.currentPage * controller.rowsPerPage)}",
                ),
            ),
            DataCell(
                Text(
                    item.name,
```

EdgeInsets.only(EdgeInsets.symmetric(BoxDecoration(ar(Colors.blueGrey , Colors.blueGrey,	<pre> style: const TextStyle(fontWeight: FontWeight.w600,),),), DataCell(SingleChildScrollView(scrollDirection: Axis.horizontal, child: Row(children: item.attributeValues .map((v) => Container(margin: const right: 6,), padding: const horizontal: 10, vertical: 4,), decoration: color: Colors.blueGrey .withOpacity(0.1), borderRadius: BorderRadius.circular 8,), border: Border.all(color: .withOpacity(0.2)),), child: Text(v, style: const TextStyle(fontSize: 12, color:),),),) .toList(), </pre>
---	---

g(

```
    ),
  ),
),
DataCell(
  _buildStatusBadge(item.isActive),
),
DataCell(
  Text(
    item.updatedAt?.toString().substin
    0,
    10,
  ) ??
  "-",
),
),
DataCell(
  Row(
    children: [
      _buildActionIcon(
        Icons.edit_note_rounded,
        Colors.blue,
      ) {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) =>
              AttributeFormPage(
                attribute: item,
              ),
          ),
        );
      },
    ],
  ),
  const SizedBox(width: 8),
  _buildActionIcon(
    Icons.delete_sweep_rounded,
    Colors.redAccent,
  ) {
    _showDeleteDialog(
      context,
      service,
      item.id,
    );
  },
),
],
),
),
],
),
],
```



```
);  
    }},  
  ),  
),  
),  
),  
  
const SizedBox(height: 24),  
  
// --- PAGINATION ---  
_buildPagination(controller),  
],  
),  
);  
},  
);  
},  
),  
),  
);  
}  
  
// Giao diện Badge Trạng thái  
Widget _buildStatusBadge(bool isActive) {  
  return Container(  
    padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),  
    decoration: BoxDecoration(  
      color: isActive ? const Color(0xFFE3F9E5) : const Color(0xFFFFEEBEB),  
      borderRadius: BorderRadius.circular(10),  
    ),  
    child: Row(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        Container(  
          width: 6,  
          height: 6,  
          decoration: BoxDecoration(  
            color: isActive ? Colors.green : Colors.red,  
            shape: BoxShape.circle,  
          ),  
        ),  
        const SizedBox(width: 8),  
        Text(  
          isActive ? "Active" : "Disabled",  
          style: TextStyle(  
            color: isActive ? Colors.green[800] : Colors.red[800],  
            fontWeight: FontWeight.bold,  
            fontSize: 12,  
          ),  
        ),  
      ],  
    ),  
  );  
}
```

```

        ),
      ],
    ),
  );
}

// Nút hành động (Edit/Delete) với hiệu ứng Hover
Widget _buildActionIcon(IconData icon, Color color, VoidCallback onPressed)
{
  return Material(
    color: Colors.transparent,
    child: InkWell(
      onTap: onPressed,
      borderRadius: BorderRadius.circular(8),
      hoverColor: color.withOpacity(0.1),
      child: Container(
        padding: const EdgeInsets.all(8),
        child: Icon(icon, color: color, size: 22),
      ),
    ),
  );
}

// Thanh phân trang thiết kế lại
Widget _buildPagination(AttributeController controller) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: List.generate(controller.totalPages, (index) {
      bool isCurrent = controller.currentPage == index;
      return AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        margin: const EdgeInsets.symmetric(horizontal: 4),
        child: InkWell(
          onTap: () => controller.currentPage = index,
          borderRadius: BorderRadius.circular(8),
          child: Container(
            width: 40,
            height: 40,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: isCurrent ? Colors.indigoAccent : Colors.white,
              borderRadius: BorderRadius.circular(8),
              border: Border.all(
                color: isCurrent ? Colors.indigoAccent : Colors.grey[300]!,
              ),
            ),
            boxShadow: isCurrent
              ? [
                  BoxShadow(
                    color: Colors.indigoAccent.withOpacity(0.4),
                    blurRadius: 8,

```

```
                offset: const Offset(0, 4),
            ),
        ],
        : [],
    ),
    child: Text(
        "${index + 1}",
        style: TextStyle(
            color: isCurrent ? Colors.white : Colors.black87,
            fontWeight: FontWeight.bold,
        ),
    ),
),
),
),
);
}),
);
}

void _showDeleteDialog(
  BuildContext context,
  AttributeService service,
  String id,
) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(20)),
      title: const Row(
        children: [
          Icon(Icons.warning_amber_rounded, color: Colors.red),
          SizedBox(width: 10),
          Text("Confirm Delete"),
        ],
      ),
      content: const Text(
        "This attribute will be permanently removed. Continue?",
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text("CANCEL"),
        ),
        ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.redAccent,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(8),
            ),
          ),

```

```
        ),
        onPressed: () async {
          await service.delete(id);
          Navigator.pop(context);
        },
        child: const Text("DELETE", style: TextStyle(color:
Colors.white)),
      ),
    ],
  ),
);
}
```

Trong file `lib/views/attributes/attribute_add_edit_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/attribute_model.dart';
import '../data/services/attribute_service.dart';

class AttributeFormPage extends StatefulWidget {
  final AttributeModel? attribute;

  const AttributeFormPage({super.key, this.attribute});

  @override
  State<AttributeFormPage> createState() => _AttributeFormPageState();
}

class _AttributeFormPageState extends State<AttributeFormPage> {
  final _formKey = GlobalKey<FormState>();
  final nameController = TextEditingController();
  final valueController = TextEditingController();

  bool isActive = true;
  bool isSearchable = false;
  bool isFilterable = false;
  bool isColorAttribute = false;

  final service = AttributeService();

  @override
  void initState() {
    super.initState();
    if (widget.attribute != null) {
      final a = widget.attribute!;
      nameController.text = a.name;
      valueController.text = a.attributeValues.join("|");
    }
  }
}
```

```
        isActive = a.isActive;
        isSearchable = widget.attribute?.isSearchable ?? false;
        isFilterable = a.isFilterable;
        isColorAttribute = a.isColorAttribute;
    }
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: Text(
                widget.attribute == null ? "Create Attribute" : "Update Attribute",
            ),
        ),
        body: Padding(
            padding: const EdgeInsets.all(20),
            child: Form(
                key: _formKey,
                child: Column(
                    children: [
                        TextFormField(
                            controller: nameController,
                            decoration: const InputDecoration(labelText: "Name"),
                        ),
                        TextFormField(
                            controller: valueController,
                            decoration: const InputDecoration(
                                labelText: "Values (small|medium|big)",
                            ),
                        ),
                        Row(
                            children: [
                                Expanded(
                                    child: CheckboxListTile(
                                        title: const Text("Active"),
                                        value: isActive,
                                        onChanged: (v) => setState(() => isActive = v!),
                                        controlAffinity: ListTileControlAffinity.leading,
                                    ),
                                ),
                            ],
                        ),
                        Row(
                            children: [
                                Expanded(
                                    child: CheckboxListTile(
                                        title: const Text("Searchable"),
                                        value: isSearchable,
```

```

        onChanged: (v) => setState(() => isSearchable = v!),
        controlAffinity: ListTileControlAffinity.leading,
      ),
    ),
    const SizedBox(width: 8),
    Tooltip(
      message:
        "Allow user to filter product based on this
attribute",
      child: const Icon(
        Icons.info_outline,
        size: 20,
        color: Colors.grey,
      ),
    ),
  ],
),
ElevatedButton(
  onPressed: () async {
    final values = valueController.text.split("|");

    final model = AttributeModel(
      id: widget.attribute?.id ?? "",
      name: nameController.text,
      attributeValues: values,
      isActive: isActive,
      isSearchable: isSearchable,
      isFilterable: isFilterable,
      isColorAttribute: isColorAttribute,
    );

    if (widget.attribute == null) {
      await service.create(model);
    } else {
      await service.update(model);
    }

    Navigator.pop(context);
  },
  child: const Text("Save"),
),
),
),
),
),
);
}
}

```

Trong file `lib/controllers/attribute_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/attribute_model.dart';

class AttributeController extends ChangeNotifier {
  List<AttributeModel> _allData = [];
  List<AttributeModel> _filteredData = [];

  int currentPage = 0;
  int rowsPerPage = 5;
  String _searchText = "";

  /// SET DATA từ stream
  void setData(List<AttributeModel> data) {
    _allData = data;
    _applyFilter();
  }

  /// SEARCH
  void search(String value) {
    _searchText = value.toLowerCase();
    currentPage = 0;
    _applyFilter();
  }

  /// FILTER LOGIC
  void _applyFilter() {
    if (_searchText.isEmpty) {
      _filteredData = _allData;
    } else {
      _filteredData = _allData.where((e) {
        return e.name.toLowerCase().contains(_searchText) ||
            e.attributeValues.join("|").toLowerCase().contains(_searchText);
      }).toList();
    }
    notifyListeners();
  }

  /// PAGINATION
  List<AttributeModel> get paginatedData {
    final start = currentPage * rowsPerPage;
    final end = start + rowsPerPage;
    return _filteredData.sublist(
      start,
      end > _filteredData.length ? _filteredData.length : end,
    );
  }
}
```

```
int get totalPages => (_filteredData.length / rowsPerPage).ceil();
}
```

Trong file `lib/data/models/attribute_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

class AttributeModel {
  String id;
  String name;
  List<String> attributeValues;
  bool isActive;
  bool isSearchable;
  bool isFilterable;
  bool isColorAttribute;
  DateTime? createdAt;
  DateTime? updatedAt;

  AttributeModel({
    required this.id,
    required this.name,
    required this.attributeValues,
    required this.isActive,
    required this.isSearchable,
    required this.isFilterable,
    required this.isColorAttribute,
    this.createdAt,
    this.updatedAt,
  });

  factory AttributeModel.fromMap(Map<String, dynamic> map, String id) {
    return AttributeModel(
      id: id,
      name: map['name'] ?? '',
      attributeValues: List<String>.from(map['attributeValues'] ?? []),
      isActive: map['isActive'] ?? true,
      isSearchable: map['isSearchable'] ?? false,
      isFilterable: map['isFilterable'] ?? false,
      isColorAttribute: map['isColorAttribute'] ?? false,
      createdAt: (map['createdAt'] as Timestamp?).toDate(),
      updatedAt: (map['updatedAt'] as Timestamp?).toDate(),
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'name': name,

```



```
'attributeValues': attributeValues,
'isActive': isActive,
'isSearchable': isSearchable,
'isFilterable': isFilterable,
'isColorAttribute': isColorAttribute,
'createdAt': createdAt ?? FieldValue.serverTimestamp(),
'updatedAt': FieldValue.serverTimestamp(),
};
}
}
```

Trong file `lib/data/services/attribute_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/attribute_model.dart';

class AttributeService {
  final _db = FirebaseFirestore.instance;
  final String collection = "attributes";

  Future<void> create(AttributeModel model) async {
    await _db.collection(collection).add(model.toMap());
  }

  Future<void> update(AttributeModel model) async {
    await _db.collection(collection).doc(model.id).update(model.toMap());
  }

  Future<void> delete(String id) async {
    await _db.collection(collection).doc(id).delete();
  }

  Stream<List<AttributeModel>> getAll() {
    return _db
      .collection(collection)
      .orderBy('updatedAt', descending: true)
      .snapshots()
      .map(
        (snapshot) => snapshot.docs
          .map((doc) => AttributeModel.fromMap(doc.data(), doc.id))
          .toList(),
      );
  }
}
```

Trong file `lib/main.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/controllers/attribute_controller.dart';
import 'package:draf_project_web/controllers/category_controller.dart';
import 'package:flutter/material.dart';
```

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_quill/flutter_quill.dart';

import 'package:provider/provider.dart';

import 'firebase_options.dart';

import 'controllers/auth_controller.dart';

import 'routes/app_routes.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) =>
AuthController()..checkLogin()),
        ChangeNotifierProvider(create: (_) => CategoryController()),
        ChangeNotifierProvider(create: (_) => AttributeController()),
      ],
      child: Builder(
        builder: (context) {
          final auth = context.watch<AuthController>();

          return MaterialApp.router(
            debugShowCheckedModeBanner: false,

            // ADD THIS BLOCK
            localizationsDelegates: const [
              FlutterQuillLocalizations.delegate, // FIX LỖI
              GlobalMaterialLocalizations.delegate,
              GlobalWidgetsLocalizations.delegate,
              GlobalCupertinoLocalizations.delegate,
            ],
            supportedLocales: const [Locale('en'), Locale('vi')],

            routerDelegate: AppRouterDelegate(auth),
            routeInformationParser: AppRouteParser(),
```

```
        );  
      },  
    ),  
  );  
}  
}
```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';  
import '../views/auth/login_page.dart';  
import '../views/layout/admin_layout.dart';  
import '../views/categories/category_page.dart';  
import '../controllers/auth_controller.dart';  
  
import '../views/attributes/attribute_page.dart';  
  
class AppRouterDelegate extends RouterDelegate<String>  
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {  
  final GlobalKey<NavigatorState> navigatorKey;  
  final AuthController authController;  
  String _currentPath = "/login";  
  AppRouterDelegate(this.authController)  
    : navigatorKey = GlobalKey<NavigatorState>() {  
    authController.addListener(notifyListeners);  
  }  
  @override  
  String? get currentConfiguration => _currentPath;  
  @override  
  Widget build(BuildContext context) {  
    if (!authController.isLoggedIn) {  
      return Navigator(  
        key: navigatorKey,  
        pages: const [MaterialPage(child: LoginPage())],  
        onPopPage: (route, result) => route.didPop(result),  
      );  
    }  
    Widget page;  
    switch (_currentPath) {  
      case "/attributes":  
        page = const AttributesPage();  
        break;  
  
      case "/categories":  
        page = const CategoriesPage();  
        break;  
  
      default:  
        page = const Center(child: Text("Dashboard Page"));  
    }  
  }  
}
```

```
return Navigator(  
  key: navigatorKey,  
  pages: [  
    MaterialPage(  
      child: AdminLayout(  
        currentRoute: _currentPath, // truyền route hiện tại  
        onNavigate: (path) {  
          _currentPath = path;  
          notifyListeners();  
        },  
        child: page,  
      ),  
    ),  
  ],  
  onPopPage: (route, result) => route.didPop(result),  
);  
  
@override  
Future<void> setNewRoutePath(String configuration) async {  
  _currentPath = configuration;  
}  
  
class AppRouteParser extends RouteInformationParser<String> {  
  @override  
  Future<String> parseRouteInformation(  
    RouteInformation routeInformation,  
  ) async {  
    return routeInformation.location ?? "/login";  
  }  
  
  @override  
  RouteInformation restoreRouteInformation(String configuration) {  
    return RouteInformation(location: configuration);  
  }  
}
```

6. Thiết kế và phát triển nhóm chức năng quản lý thương hiệu của sản phẩm

Trong file `lib/views/brands/brands_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import '../controllers/brand_controller.dart';  
import '../data/models/brand_model.dart';  
  
class BrandsPage extends StatelessWidget {  
  const BrandsPage({super.key});  
}
```

```
@override
Widget build(BuildContext context) {
  return ChangeNotifierProvider(
    create: (_) => BrandController()..loadBrands(),
    child: const Scaffold(
      backgroundColor: Color(0xFFF8F9FD), // Nền xám nhạt hiện đại
      body: _BrandsView(),
    ),
  );
}

class _BrandsView extends StatelessWidget {
  const _BrandsView();

  @override
  Widget build(BuildContext context) {
    final controller = context.watch<BrandController>();

    return Padding(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // --- HEADER SECTION ---
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    "Brand Management",
                    style: TextStyle(
                      fontSize: 28,
                      fontWeight: FontWeight.bold,
                      color: Color(0xFF1E293B),
                    ),
                  ),
                  Text(
                    "Manage your product partners and labels",
                    style: TextStyle(color: Colors.grey),
                  ),
                ],
              ),
              ElevatedButton.icon(
                onPressed: () => _showDialog(context),
                icon: const Icon(
                  Icons.add_business_rounded,

```

```
        color: Colors.white,
      ),
      label: const Text("ADD NEW BRAND"),
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.indigoAccent,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(
          horizontal: 24,
          vertical: 20,
        ),
      ),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      elevation: 2,
    ),
  ),
],
),

const SizedBox(height: 30),

// --- SEARCH BAR ---
Container(
  width: 400,
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.05),
        blurRadius: 10,
        offset: const Offset(0, 4),
      ),
    ],
  ),
),
child: TextField(
  decoration: const InputDecoration(
    hintText: "Search brands...",
    prefixIcon: Icon(Icons.search_rounded, color: Colors.indigo),
    border: InputBorder.none,
    contentPadding: EdgeInsets.symmetric(vertical: 15),
  ),
  onChanged: controller.search,
),
),

const SizedBox(height: 24),

// --- TABLE SECTION ---
controller.isLoading
```

```
? const Expanded(
  child: Center(child: CircularProgressIndicator()),
)
: Expanded(
  child: Container(
    width: double.infinity,
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color:
Colors.grey.withOpacity(0.2)),
    ),
    child: ClipRRect(
      borderRadius: BorderRadius.circular(16),
      child: SingleChildScrollView(
        child: DataTable(
          headingRowColor: MaterialStateProperty.all(
            Colors.indigo.withOpacity(0.05),
          ),
          dataRowHeight: 70,
          horizontalMargin: 20,
          columns: const [
            DataColumn(
              label: Text(
                "SEQ",
                style: TextStyle(
                  fontWeight: FontWeight.bold,
                  color: Colors.indigo,
                ),
            ),
            DataColumn(
              label: Text(
                "BRAND",
                style: TextStyle(
                  fontWeight: FontWeight.bold,
                  color: Colors.indigo,
                ),
            ),
            DataColumn(
              label: Text(
                "CATEGORIES",
                style: TextStyle(
                  fontWeight: FontWeight.bold,
                  color: Colors.indigo,
                ),
            ),
            DataColumn(
```

```
        label: Text(
            "FEATURED",
            style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
            ),
        ),
    ),
    DataColumn(
        label: Text(
            "STATUS",
            style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
            ),
        ),
    ),
    DataColumn(
        label: Text(
            "UPDATED",
            style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
            ),
        ),
    ),
    DataColumn(
        label: Text(
            "ACTION",
            style: TextStyle(
                fontWeight: FontWeight.bold,
                color: Colors.indigo,
            ),
        ),
    ),
],
rows:
controller.paginatedData.asMap().entries.map((
    entry,
) {
    final index = entry.key;
    final b = entry.value;
    return DataRow(
        cells: [
            DataCell(
                Text(
                    "${(controller.currentPage - 1) *
controller.rowsPerPage + index + 1}",
                ),
            ),
        ],
    ),
),
```



```
BorderRadius.circular(
DataCell(
  Row(
    children: [
      Container(
        width: 45,
        height: 45,
        decoration: BoxDecoration(
          borderRadius:
            8,
        ),
        color: Colors.grey[100],
        image: b.imageUrl.isNotEmpty
          ? DecorationImage(
              image: NetworkImage(
                b.imageUrl,
              ),
              fit: BoxFit.cover,
            )
          : null,
      ),
      child: b.imageUrl.isEmpty
        ? const Icon(Icons.business)
        : null,
    ],
    const SizedBox(width: 12),
    Text(
      b.name,
      style: const TextStyle(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
),
DataCell(
  SizedBox(
    width: 180,
    child: SingleChildScrollView(
      scrollDirection: Axis.horizontal,
      child: Row(
        children:
          (controller.brandCategoriesMap[b
            .id] ??
            [])
            .map(
              (cat) => Container(
                margin:
                  const
```

```

        right: 4,
      ),
      padding:
        const
        horizontal: 8,
        vertical: 4,
      ),
      decoration:
        color: Colors.blue
          .withOpacity(0.1),
        borderRadius:
          BorderRadius.circular
            6,
      ),
      child: Text(
        cat,
        style: const TextStyle(
          fontSize: 11,
          color: Colors.blue,
        ),
      ),
    ),
  ).toList(),
),
),
),
),
DataCell(
  Icon(
    b.isFeatured
      ? Icons.stars_rounded
      : Icons.star_outline_rounded,
    color: b.isFeatured
      ? Colors.amber
      : Colors.grey[400],
  ),
),
DataCell(_buildStatusBadge(b.isActive)),
DataCell(
  Text(
    b.updatedAt != null
      ?
        "${b.updatedAt!.day}/${b.updatedAt!.month}/${b.updatedAt!.year}"
        : "-",
  ),
),

```



```
        fontWeight: FontWeight.bold,
      ),
    ),
  );
}

Widget _buildIconButton(IconData icon, Color color, VoidCallback onTap) {
  return InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(8),
    child: Container(
      padding: const EdgeInsets.all(8),
      decoration: BoxDecoration(
        color: color.withOpacity(0.1),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Icon(icon, size: 20, color: color),
    ),
  );
}

Widget _buildPagination(BrandController controller) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: List.generate(controller.totalPages, (index) {
      final page = index + 1;
      bool isCurrent = controller.currentPage == page;
      return Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4),
        child: InkWell(
          onTap: () => controller.changePage(page),
          child: Container(
            width: 35,
            height: 35,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: isCurrent ? Colors.indigo : Colors.white,
              borderRadius: BorderRadius.circular(8),
              border: Border.all(
                color: isCurrent ? Colors.indigo : Colors.grey[300]!,
              ),
            ),
            child: Text(
              "$page",
              style: TextStyle(
                color: isCurrent ? Colors.white : Colors.black,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

```

        ),
    );
    }},
);
}

void _confirmDelete(BuildContext context, String id) {
    final controller = context.read<BrandController>();
    showDialog(
        context: context,
        builder: (_) => AlertDialog(
            shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(16)),
            title: const Text("Delete Brand?"),
            content: const Text("This action cannot be undone."),
            actions: [
                TextButton(
                    onPressed: () => Navigator.pop(context),
                    child: const Text("CANCEL"),
                ),
                ElevatedButton(
                    style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
                    onPressed: () async {
                        await controller.delete(id);
                        Navigator.pop(context);
                    },
                    child: const Text("DELETE", style: TextStyle(color:
Colors.white)),
                ),
            ],
        ),
    );
}

void _showDialog(BuildContext context, {BrandModel? brand}) async {
    final controller = context.read<BrandController>();
    await controller.fetchAllCategories();
    if (brand != null) {
        await controller.loadBrandCategories(brand.id);
    } else {
        controller.selectedCategoryIds = [];
    }

    final nameController = TextEditingController(text: brand?.name ?? "");
    final imageController = TextEditingController(text: brand?.imageUrl ??
"");
    bool isActive = brand?.isActive ?? true;
    bool isFeatured = brand?.isFeatured ?? false;

    showDialog(

```

```
context: context,
builder: (_) => StatefulBuilder(
  builder: (context, setState) {
    return AlertDialog(
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(20),
      ),
      title: Text(brand == null ? "🌟 New Brand" : "📝 Edit Brand"),
      content: SizedBox(
        width: 550,
        child: SingleChildScrollView(
          child: Column(
            children: [
              TextField(
                controller: nameController,
                decoration: InputDecoration(
                  labelText: "Brand Name",
                  prefixIcon: const Icon(Icons.label_outline),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: imageController,
                decoration: InputDecoration(
                  labelText: "Logo URL",
                  prefixIcon: const Icon(Icons.image_outlined),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
              onChanged: (_) => setState(() {}),
            ],
          ),
          const SizedBox(height: 16),
          if (imageController.text.isNotEmpty)
            ClipRRect(
              borderRadius: BorderRadius.circular(12),
              child: Image.network(
                imageController.text,
                height: 100,
                width: 100,
                fit: BoxFit.cover,
                errorBuilder: (context, error, stackTrace) =>
                  const Icon(
                    Icons.broken_image,
                    size: 50,
                    color: Colors.grey,
                  ),
            ),
        ),
      ),
    ),
  ),
),
```

```

    ),
  ),
  const SizedBox(height: 10),
  Row(
    children: [
      Expanded(
        child: SwitchListTile(
          title: const Text("Active"),
          value: isActive,
          onChanged: (v) => setState(() => isActive = v),
        ),
      ),
      Expanded(
        child: SwitchListTile(
          title: const Text("Featured"),
          value: isFeatured,
          onChanged: (v) => setState(() => isFeatured = v),
        ),
      ),
    ],
  ),
  const Divider(height: 32),
  const Align(
    alignment: Alignment.centerLeft,
    child: Text(
      "Categories Management",
      style: TextStyle(fontWeight: FontWeight.bold),
    ),
  ),
  const SizedBox(height: 12),
  Container(
    height: 200,
    decoration: BoxDecoration(
      color: Colors.grey[50],
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: Colors.grey[200]!),
    ),
    child: ListView(
      padding: const EdgeInsets.all(8),
      children: controller.allCategories.map((cat) {
        final isSelected = controller.selectedCategoryIds
          .contains(cat.id);
        return CheckboxListTile(
          value: isSelected,
          title: Text(cat.name),
          controlAffinity: ListTileControlAffinity.leading,
          onChanged: (checked) {
            setState(() {
              if (checked == true) {
                controller.selectedCategoryIds.add(cat.id);
              }
            });
          },
        );
      }).toList(),
    ),
  ),

```

```

        } else {
            controller.selectedCategoryIds.remove(cat.id);
        }
    });
},
);
}).toList(),
),
),
],
),
),
),
actions: [
    TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text("CANCEL"),
    ),
    ElevatedButton(
        style: ElevatedButton.styleFrom(
            backgroundColor: Colors.indigo,
            padding: const EdgeInsets.symmetric(
                horizontal: 30,
                vertical: 15,
            ),
        ),
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(10),
        ),
    ),
    onPressed: () async {
        final newBrand = BrandModel(
            id: brand?.id ?? "",
            name: nameController.text,
            imageURL: imageController.text,
            isFeatured: isFeatured,
            isActive: isActive,
            productsCount: brand?.productsCount ?? 0,
            viewCount: brand?.viewCount ?? 0,
            createdAt: brand?.createdAt ?? DateTime.now(),
            updatedAt: DateTime.now(),
        );
        if (brand == null)
            await controller.add(newBrand);
        else
            await controller.update(newBrand);
        if (brand != null) await
controller.saveRelations(brand.id);
        Navigator.pop(context);
    },

```



```
        child: const Text(  
          "SAVE BRAND",  
          style: TextStyle(color: Colors.white),  
        ),  
      ),  
    ],  
  );  
},  
),  
);  
}
```

Trong file `lib/controllers/brands/brand_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';  
import '../data/models/brand_model.dart';  
import '../data/services/brand_service.dart';  
import '../data/services/category_service.dart';  
import '../data/models/category_model.dart';  
  
class BrandController extends ChangeNotifier {  
  final BrandService _service = BrandService();  
  
  List<BrandModel> brands = [];  
  List<BrandModel> filtered = [];  
  
  bool isLoading = false;  
  
  int currentPage = 1;  
  int rowsPerPage = 5;  
  
  Future<void> fetchBrands() async {  
    isLoading = true;  
    notifyListeners();  
  
    brands = await _service.getBrands();  
    filtered = brands;  
  
    isLoading = false;  
    notifyListeners();  
  }  
  
  void search(String keyword) {  
    currentPage = 1;  
  
    if (keyword.isEmpty) {  
      filtered = brands;  
    }  
  }  
}
```

```
    } else {
        filtered = brands
            .where((b) => b.name.toLowerCase().contains(keyword.toLowerCase()))
            .toList();
    }
    notifyListeners();
}

List<BrandModel> get paginatedData {
    final start = (currentPage - 1) * rowsPerPage;
    final end = start + rowsPerPage;
    return filtered.sublist(
        start,
        end > filtered.length ? filtered.length : end,
    );
}

int get totalPages => (filtered.length / rowsPerPage).ceil();

void changePage(int page) {
    currentPage = page;
    notifyListeners();
}

Future<void> add(BrandModel brand) async {
    final id = await _service.addBrand(brand);

    await _service.saveBrandCategories(id, selectedCategoryIds);

    await loadBrands();
}

Future<void> update(BrandModel brand) async {
    await _service.updateBrand(brand);
    await fetchBrands();
}

Future<void> delete(String id) async {
    await _service.deleteBrand(id);
    await fetchBrands();
}

List<String> selectedCategoryIds = [];
Future<void> loadBrandCategories(String brandId) async {
    selectedCategoryIds = await _service.getCategoriesOfBrand(brandId);
    notifyListeners();
}

Future<void> saveRelations(String brandId) async {
    await _service.saveBrandCategories(brandId, selectedCategoryIds);
}
```

```
}

final CategoryService _categoryService = CategoryService();

List<CategoryModel> allCategories = [];

Future<void> fetchAllCategories() async {
  allCategories = await _categoryService.getCategories();
  notifyListeners();
}

Map<String, List<String>> brandCategoriesMap = {};

Future<void> loadBrands() async {
  isLoading = true;
  notifyListeners();

  brands = await _service.getAllBrands();

  final relations = await _service.getAllBrandCategoryRelations();
  final categories = await _service.getAllCategories();

  final categoryMap = {for (var c in categories) c.id: c.name};

  brandCategoriesMap.clear();

  for (var relation in relations) {
    brandCategoriesMap.putIfAbsent(relation.brandId, () => []);

    final categoryName = categoryMap[relation.categoryId];

    if (categoryName != null) {
      brandCategoriesMap[relation.brandId]!.add(categoryName);
    }
  }

  filtered = brands;
  currentPage = 1;

  isLoading = false;
  notifyListeners();
}
```

Trong file `lib/data/models/brand_category_model.dart`, soạn mã nguồn có nội dung như sau:

```
class BrandCategoryModel {
  final String brandId;
```

```
final String categoryId;

BrandCategoryModel({required this.brandId, required this.categoryId});

Map<String, dynamic> toMap() {
  return {'brandId': brandId, 'categoryId': categoryId};
}
}
```

Trong file `lib/data/models/brand_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

class BrandModel {
  String id;
  String name;
  String imageURL;
  bool isFeatured;
  bool isActive;
  int productsCount;
  int viewCount;
  DateTime? createdAt;
  DateTime? updatedAt;

  BrandModel({
    required this.id,
    required this.name,
    required this.imageURL,
    required this.isFeatured,
    required this.isActive,
    required this.productsCount,
    required this.viewCount,
    this.createdAt,
    this.updatedAt,
  });

  factory BrandModel.fromFirestore(DocumentSnapshot doc) {
    final data = doc.data() as Map<String, dynamic>;

    return BrandModel(
      id: doc.id,
      name: data['name'] ?? '',
      imageURL: data['imageURL'] ?? '',
      isFeatured: data['isFeatured'] ?? false,
      isActive: data['isActive'] ?? true,
      productsCount: data['productsCount'] ?? 0,
      viewCount: data['viewCount'] ?? 0,
      createdAt: (data['createdAt'] as Timestamp?).toDate(),
      updatedAt: (data['updatedAt'] as Timestamp?).toDate(),
    );
  }
}
```

```
    );  
  }  
  
  Map<String, dynamic> toMap() {  
    return {  
      'name': name,  
      'imageUrl': imageUrl,  
      'isFeatured': isFeatured,  
      'isActive': isActive,  
      'productsCount': productsCount,  
      'viewCount': viewCount,  
      'createdAt': createdAt,  
      'updatedAt': updatedAt,};  
  }
```

Trong file `lib/data/services/brand_service.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';  
import '../models/brand_model.dart';  
import '../models/category_model.dart';  
import '../models/brand_category_model.dart';  
  
class BrandService {  
  final _firestore = FirebaseFirestore.instance;  
  
  Future<List<BrandModel>> getBrands() async {  
    final snapshot = await _firestore.collection('brands').get();  
  
    return snapshot.docs.map((doc) =>  
      BrandModel.fromFirestore(doc)).toList();  
  }  
  
  Future<String> addBrand(BrandModel brand) async {  
    final doc = await _firestore.collection('brands').add(brand.toMap());  
    return doc.id;  
  }  
  
  Future<void> updateBrand(BrandModel brand) async {  
    await  
_firestore.collection('brands').doc(brand.id).update(brand.toMap());  
  }  
  
  Future<void> deleteBrand(String id) async {  
    await _firestore.collection('brands').doc(id).delete();  
  
    // Xóa relation  
    final relationSnapshot = await _firestore  
      .collection('brand_categories')  
      .where('brandId', isEqualTo: id)  
      .get();
```

```
        for (var doc in relationSnapshot.docs) {
            await doc.reference.delete();
        }
    }

Future<void> saveBrandCategories(
    String brandId,
    List<String> categoryIds,
) async {
    final batch = _firestore.batch();

    // xóa cũ
    final snapshot = await _firestore
        .collection('brand_categories')
        .where('brandId', isEqualTo: brandId)
        .get();

    for (var doc in snapshot.docs) {
        batch.delete(doc.reference);
    }

    // thêm mới
    for (var categoryId in categoryIds) {
        final ref = _firestore.collection('brand_categories').doc();
        batch.set(ref, {'brandId': brandId, 'categoryId': categoryId});
    }

    await batch.commit();
}

Future<List<String>> getCategoriesOfBrand(String brandId) async {
    final snapshot = await _firestore
        .collection('brand_categories')
        .where('brandId', isEqualTo: brandId)
        .get();

    return snapshot.docs.map((doc) => doc['categoryId'] as String).toList();
}

Future<List<BrandModel>> getAllBrands() async {
    final snapshot = await FirebaseFirestore.instance
        .collection('brands')
        .get();

    return snapshot.docs.map((doc) =>
BrandModel.fromFirestore(doc)).toList();
}

Future<List<String>> getCategoryNamesByBrand(String brandId) async {
```

```
final snapshot = await FirebaseFirestore.instance
    .collection('brand_categories')
    .where('brandId', isEqualTo: brandId)
    .get();

List<String> categoryIds = snapshot.docs
    .map((e) => e['categoryId'] as String)
    .toList();

if (categoryIds.isEmpty) return [];

final categorySnapshot = await FirebaseFirestore.instance
    .collection('categories')
    .where(FieldPath.documentId, whereIn: categoryIds)
    .get();

return categorySnapshot.docs.map((e) => e['name'] as String).toList();
}

Future<List<BrandCategoryModel>> getAllBrandCategoryRelations() async {
    final snapshot = await FirebaseFirestore.instance
        .collection('brand_categories')
        .get();

    return snapshot.docs.map((doc) {
        final data = doc.data();
        return BrandCategoryModel(
            brandId: data['brandId'],
            categoryId: data['categoryId'],
        );
    }).toList();
}

Future<List<CategoryModel>> getAllCategories() async {
    final snapshot = await FirebaseFirestore.instance
        .collection('categories')
        .get();

    return snapshot.docs
        .map((doc) => CategoryModel.fromFirestore(doc))
        .toList();
}

final String collection = "brands";

Stream<Map<String, String>> getBrandMap() {
    return _firestore.collection(collection).snapshots().map((snapshot) {
        final map = <String, String>{};
        for (var doc in snapshot.docs) {
            map[doc.id] = doc["name"] ?? "";
        }
    });
}
```

```
    }  
    return map;  
  });  
}  
}
```

Trong file `lib/main.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/controllers/brand_controller.dart';  
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'package:flutter_localizations/flutter_localizations.dart';  
import 'package:flutter_quill/flutter_quill.dart';  
  
import 'package:provider/provider.dart';  
  
import 'firebase_options.dart';  
  
import 'controllers/auth_controller.dart';  
  
import 'controllers/category_controller.dart';  
  
import 'controllers/attribute_controller.dart';  
  
import 'routes/app_routes.dart';  
  
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(options:  
DefaultFirebaseOptions.currentPlatform);  
  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MultiProvider(  
      providers: [  
        ChangeNotifierProvider(create: (_) =>  
AuthController()..checkLogin()),  
  
        ChangeNotifierProvider(create: (_) => CategoryController()),  
  
        ChangeNotifierProvider(create: (_) => AttributeController()),  
        ChangeNotifierProvider(create: (_) => BrandController()),  
      ],  
      child: Builder(  

```



```
builder: (context) {  
  final auth = context.watch<AuthController>();  
  
  return MaterialApp.router(  
    debugShowCheckedModeBanner: false,  
  
    // ADD THIS BLOCK  
    localizationsDelegates: const [  
      FlutterQuillLocalizations.delegate, // FIX LỖI  
      GlobalMaterialLocalizations.delegate,  
      GlobalWidgetsLocalizations.delegate,  
      GlobalCupertinoLocalizations.delegate,  
    ],  
    supportedLocales: const [Locale('en'), Locale('vi')],  
  
    routerDelegate: AppRouterDelegate(auth),  
    routeInformationParser: AppRouteParser(),  
  );  
},  
),  
);  
}
```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/brands/brands_page.dart';  
import 'package:flutter/material.dart';  
import '../views/auth/login_page.dart';  
import '../views/layout/admin_layout.dart';  
import '../views/cageteries/category_page.dart';  
import '../controllers/auth_controller.dart';  
  
import '../views/attributes/attribute_page.dart';  
  
class AppRouterDelegate extends RouterDelegate<String>  
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {  
  final GlobalKey<NavigatorState> navigatorKey;  
  final AuthController authController;  
  String _currentPath = "/login";  
  AppRouterDelegate(this.authController)  
    : navigatorKey = GlobalKey<NavigatorState>() {  
    authController.addListener(notifyListeners);  
  }  
  @override  
  String? get currentConfiguration => _currentPath;  
  @override  
  Widget build(BuildContext context) {  
    if (!authController.isLoggedIn) {  
      return Navigator(  

```

```

        key: navigatorKey,
        pages: const [MaterialPage(child: LoginPage())],
        onPopPage: (route, result) => route.didPop(result),
    );
}
Widget page;
switch (_currentPath) {
    case "/attributes":
        page = const AttributesPage();
        break;
    case "/brands":
        page = const BrandsPage();
        break;

    case "/categories":
        page = const CategoriesPage();
        break;

    default:
        page = const Center(child: Text("Dashboard Page"));
}
return Navigator(
    key: navigatorKey,
    pages: [
        MaterialPage(
            child: AdminLayout(
                currentRoute: _currentPath, // truyền route hiện tại
                onNavigate: (path) {
                    _currentPath = path;
                    notifyListeners();
                },
                child: page,
            ),
        ),
    ],
    onPopPage: (route, result) => route.didPop(result),
);
}

@override
Future<void> setNewRoutePath(String configuration) async {
    _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
    @override
    Future<String> parseRouteInformation(
        RouteInformation routeInformation,
    ) async {

```

```
        return routeInformation.location ?? "/login";
    }

    @override
    RouteInformation restoreRouteInformation(String configuration) {
        return RouteInformation(location: configuration);
    }
}
```

7. Thiết kế và phát triển nhóm chức năng quản lý mã giảm giá cho đơn hàng

Trong file `lib/views/coupons/coupons_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/coupon_controller.dart';
import '../data/models/coupon_model.dart';

class CouponsPage extends StatelessWidget {
    const CouponsPage({super.key});

    @override
    Widget build(BuildContext context) {
        return ChangeNotifierProvider(
            create: (_) => CouponController()..fetchCoupons(),
            child: const Scaffold(
                backgroundColor: Color(0xFFF8F9FD),
                body: _CouponsView(),
            ),
        );
    }
}

class _CouponsView extends StatelessWidget {
    const _CouponsView();

    @override
    Widget build(BuildContext context) {
        final controller = context.watch<CouponController>();

        return Padding(
            padding: const EdgeInsets.all(24),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    /// --- TOP BAR ---
                    Row(
                        mainAxisAlignment: MainAxisAlignment.spaceBetween,
                        children: [
```

```
const Text(
  "Coupon Management",
  style: TextStyle(
    fontSize: 28,
    fontWeight: FontWeight.bold,
    color: Color(0xFF2B3674),
  ),
),
ElevatedButton.icon(
  onPressed: () => _showDialog(context),
  icon: const Icon(Icons.add_rounded),
  label: const Text("ADD COUPON"),
  style: ElevatedButton.styleFrom(
    backgroundColor: const Color(0xFF4318FF),
    foregroundColor: Colors.white,
    padding: const EdgeInsets.symmetric(
      horizontal: 20,
      vertical: 18,
    ),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    elevation: 0,
  ),
),
],
),

const SizedBox(height: 24),

/// --- SEARCH FIELD ---
Container(
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(15),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.02),
        blurRadius: 10,
        offset: const Offset(0, 4),
      ),
    ],
  ),
  child: TextField(
    onChanged: controller.search,
    decoration: const InputDecoration(
      hintText: "Search coupon code...",
      prefixIcon: Icon(Icons.search, color: Color(0xFF4318FF)),
      border: InputBorder.none,
      contentPadding: EdgeInsets.symmetric(vertical: 15),
    ),
  ),
),
```

```

        ),
    ),
),

const SizedBox(height: 24),

/// --- DATA TABLE AREA ---
Expanded(
  child: Container(
    width: double.infinity,
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(20),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.03),
          blurRadius: 20,
          offset: const Offset(0, 10),
        ),
      ],
    ),
    child: controller.isLoading
      ? const Center(child: CircularProgressIndicator())
      : ClipRRect(
          borderRadius: BorderRadius.circular(20),
          child: SingleChildScrollView(
            scrollDirection: Axis.vertical,
            child: SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: DataTable(
                headingRowHeight: 56,
                dataRowMaxHeight: 64,
                columnSpacing: 24,
                headingRowColor: MaterialStateProperty.all(
                  const Color(0xFFFF4F7FE),
                ),
                columns: const [
                  DataColumn(label: _TableLabel("SEQ")),
                  DataColumn(label: _TableLabel("COUPON")),
                  DataColumn(label: _TableLabel("DISCOUNT
VALUE")),
                  DataColumn(label: _TableLabel("TYPE")),
                  DataColumn(label: _TableLabel("DESCRIPTION")),
                  DataColumn(label: _TableLabel("IS ACTIVE")),
                  DataColumn(label: _TableLabel("START DATE")),
                  DataColumn(label: _TableLabel("END DATE")),
                  DataColumn(label: _TableLabel("ACTION")),
                ],
                rows:
controller.paginatedData.asMap().entries.map((

```

```
        entry,
    ) {
        final index = entry.key;
        final c = entry.value;
        final seq =
            (controller.currentPage - 1) *
            controller.rowsPerPage +
            index +
            1;

        return DataRow(
            cells: [
                // SEQ
                DataCell(
                    Text(
                        "$seq",
                        style: const TextStyle(
                            color: Color(0xFFA3AED0),
                        ),
                    ),
                ),
                // COUPON
                DataCell(
                    Text(
                        c.code,
                        style: const TextStyle(
                            fontWeight: FontWeight.bold,
                            color: Color(0xFF2B3674),
                        ),
                    ),
                ),
                // DISCOUNT VALUE
                DataCell(
                    Text(
                        c.discountType ==
DiscountType.percentage
                        ? "${c.discountValue}%"
                        :
"${c.discountValue.toStringAsFixed(0)} đ",
                        style: const TextStyle(
                            fontWeight: FontWeight.w600,
                        ),
                    ),
                ),
                // TYPE
                DataCell(
                    Text(
```

```
DiscountType.percentage

        c.discountType ==
            ? "Percentage"
            : "Flat",
    ),
),

// DESCRIPTION
DataCell(
    SizedBox(
        width: 180,
        child: Text(
            c.description,
            overflow: TextOverflow.ellipsis,
            style: const TextStyle(
                fontSize: 13,
                color: Colors.blueGrey,
            ),
        ),
    ),
),

// IS ACTIVE
DataCell(_StatusBadge(isActive:
c.isActive)),

// START DATE
DataCell(
    Text(
        c.startDate != null
        ?
"${c.startDate!.day}/${c.startDate!.month}/${c.startDate!.year}"
        : "-",
    ),
),

// END DATE
DataCell(
    Text(
        c.endDate != null
        ?
"${c.endDate!.day}/${c.endDate!.month}/${c.endDate!.year}"
        : "-",
    ),
),

// ACTION
DataCell(
    Row(
        mainAxisAlignment: MainAxisAlignment.min,
```

```
c),  
        children: [  
          _CircularActionButton(  
            icon: Icons.edit_rounded,  
            color: Colors.blue,  
            onPressed: () =>  
              _showDialog(context, coupon:  
  
            ),  
          const SizedBox(width: 8),  
          _CircularActionButton(  
            icon: Icons.delete_outline_rounded,  
            color: Colors.red,  
            onPressed: () =>  
              controller.delete(c.id),  
  
          ),  
        ],  
      ),  
    ),  
  ],  
);  
}).toList(),  
),  
),  
),  
),  
),  
),  
),  
],  
),  
);  
}  
  
void _showDialog(BuildContext context, {CouponModel? coupon}) {  
  final controller = context.read<CouponController>();  
  
  final codeController = TextEditingController(text: coupon?.code ?? "");  
  final descController = TextEditingController(  
    text: coupon?.description ?? "",  
  );  
  final valueController = TextEditingController(  
    text: coupon?.discountValue.toString() ?? "",  
  );  
  
  DiscountType type = coupon?.discountType ?? DiscountType.percentage;  
  bool isActive = coupon?.isActive ?? true;  
  
  showDialog(  
    context: context,  
    builder: (_) => StatefulBuilder(  
      builder: (context, setState) {
```



```

return AlertDialog(
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(20),
  ),
  title: Text(coupon == null ? "New Coupon" : "Edit Coupon"),
  content: SizedBox(
    width: 400,
    child: SingleChildScrollView(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          _buildField(controller: codeController, label: "Code"),
          const SizedBox(height: 12),
          _buildField(
            controller: descController,
            label: "Description",
          ),
          const SizedBox(height: 12),
          DropdownButtonFormField<DiscountType>(
            value: type,
            decoration: const InputDecoration(
              labelText: "Discount Type",
              border: OutlineInputBorder(),
            ),
            items: const [
              DropdownMenuItem(
                value: DiscountType.percentage,
                child: Text("Percentage"),
              ),
              DropdownMenuItem(
                value: DiscountType.flat,
                child: Text("Flat"),
              ),
            ],
            onChanged: (v) => setState(() => type = v!),
          ),
          const SizedBox(height: 12),
          _buildField(
            controller: valueController,
            label: "Discount Value",
            isNumber: true,
          ),
          const SizedBox(height: 8),
          SwitchListTile(
            title: const Text(
              "Is Active",
              style: TextStyle(fontSize: 14),
            ),
            value: isActive,
            activeColor: const Color(0xFF4318FF),

```

```

        onChanged: (v) => setState(() => isActive = v),
      ),
    ],
  ),
),
actions: [
  TextButton(
    onPressed: () => Navigator.pop(context),
    child: const Text(
      "Cancel",
      style: TextStyle(color: Colors.grey),
    ),
  ),
  ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFF4318FF),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(8),
      ),
    ),
    onPressed: () async {
      final newCoupon = CouponModel(
        id: coupon?.id ?? "",
        code: codeController.text,
        description: descController.text,
        discountType: type,
        discountValue: double.tryParse(valueController.text) ??
0,
        startDate: DateTime.now(),
        endDate: DateTime.now().add(const Duration(days: 7)),
        usageLimit: 100,
        usageCount: coupon?.usageCount ?? 0,
        isActive: isActive,
        createdAt: coupon?.createdAt ?? DateTime.now(),
        updatedAt: DateTime.now(),
      );

      if (coupon == null)
        await controller.add(newCoupon);
      else
        await controller.update(newCoupon);

      Navigator.pop(context);
    },
    child: const Text(
      "SAVE",
      style: TextStyle(color: Colors.white),
    ),
  ),
],
),

```

```
        ],
      );
    },
  ),
);
}

Widget _buildField({
  required TextEditingController controller,
  required String label,
  bool isNumber = false,
}) {
  return TextField(
    controller: controller,
    keyboardType: isNumber ? TextInputType.number : TextInputType.text,
    decoration: InputDecoration(
      labelText: label,
      border: const OutlineInputBorder(),
      contentPadding: const EdgeInsets.symmetric(
        horizontal: 12,
        vertical: 12,
      ),
    ),
  );
}

/// --- PHẦN TRANG TRÍ RIÊNG ---

class _TableLabel extends StatelessWidget {
  final String label;
  const _TableLabel(this.label);

  @override
  Widget build(BuildContext context) {
    return Text(
      label,
      style: const TextStyle(
        color: Color(0xFFA3AED0),
        fontWeight: FontWeight.bold,
        fontSize: 13,
      ),
    );
  }
}

class _StatusBadge extends StatelessWidget {
  final bool isActive;
  const _StatusBadge({required this.isActive});
}
```

```
@override
Widget build(BuildContext context) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: isActive
        ? Colors.green.withOpacity(0.1)
        : Colors.red.withOpacity(0.1),
      borderRadius: BorderRadius.circular(8),
    ),
    child: Text(
      isActive ? "Active" : "Inactive",
      style: TextStyle(
        color: isActive ? Colors.green : Colors.red,
        fontWeight: FontWeight.bold,
        fontSize: 12,
      ),
    ),
  );
}

class _CircularActionButton extends StatelessWidget {
  final IconData icon;
  final Color color;
  final VoidCallback onPressed;

  const _CircularActionButton({
    required this.icon,
    required this.color,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return Material(
      color: color.withOpacity(0.1),
      shape: const CircleBorder(),
      child: IconButton(
        icon: Icon(icon, color: color, size: 20),
        onPressed: onPressed,
        constraints: const BoxConstraints(minWidth: 36, minHeight: 36),
        padding: EdgeInsets.zero,
      ),
    );
  }
}
```

Trong file `lib/controllers/coupon_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/coupon_model.dart';
import '../data/services/coupon_service.dart';

class CouponController extends ChangeNotifier {
  final CouponService _service = CouponService();

  List<CouponModel> coupons = [];
  List<CouponModel> filtered = [];

  bool isLoading = false;

  int currentPage = 1;
  int rowsPerPage = 5;

  Future<void> fetchCoupons() async {
    isLoading = true;
    notifyListeners();

    coupons = await _service.getCoupons();
    filtered = coupons;

    isLoading = false;
    notifyListeners();
  }

  void search(String keyword) {
    currentPage = 1;

    if (keyword.isEmpty) {
      filtered = coupons;
    } else {
      filtered = coupons
        .where((c) => c.code.toLowerCase().contains(keyword.toLowerCase()))
        .toList();
    }

    notifyListeners();
  }

  List<CouponModel> get paginatedData {
    final start = (currentPage - 1) * rowsPerPage;
    final end = start + rowsPerPage;
    return filtered.sublist(
      start,
      end > filtered.length ? filtered.length : end,
    );
  }
}
```

```
    );  
  }  
  
  int get totalPages => (filtered.length / rowsPerPage).ceil();  
  
  void changePage(int page) {  
    currentPage = page;  
    notifyListeners();  
  }  
  
  Future<void> add(CouponModel coupon) async {  
    await _service.addCoupon(coupon);  
    await fetchCoupons();  
  }  
  
  Future<void> update(CouponModel coupon) async {  
    await _service.updateCoupon(coupon);  
    await fetchCoupons();  
  }  
  
  Future<void> delete(String id) async {  
    await _service.deleteCoupon(id);  
    await fetchCoupons();  
  }  
}
```

Trong file `lib/data/models/coupon_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';  
  
enum DiscountType { percentage, flat }  
  
class CouponModel {  
  String id;  
  String code;  
  String description;  
  DiscountType discountType;  
  double discountValue;  
  DateTime? startDate;  
  DateTime? endDate;  
  int usageLimit;  
  int usageCount;  
  bool isActive;  
  DateTime? createdAt;  
  DateTime? updatedAt;  
  
  CouponModel({  
    required this.id,  
  }) {}  
}
```

```
required this.code,
required this.description,
required this.discountType,
required this.discountValue,
this.startDate,
this.endDate,
required this.usageLimit,
required this.usageCount,
required this.isActive,
this.createdAt,
this.updateAt,
});

factory CouponModel.fromFirestore(DocumentSnapshot doc) {
  final data = doc.data() as Map<String, dynamic>;

  return CouponModel(
    id: doc.id,
    code: data['code'] ?? '',
    description: data['description'] ?? '',
    discountType: data['discountType'] == 'flat'
      ? DiscountType.flat
      : DiscountType.percentage,
    discountValue: (data['discountValue'] ?? 0).toDouble(),
    startDate: (data['startDate'] as Timestamp?).toDate(),
    endDate: (data['endDate'] as Timestamp?).toDate(),
    usageLimit: data['usageLimit'] ?? 0,
    usageCount: data['usageCount'] ?? 0,
    isActive: data['isActive'] ?? true,
    createdAt: (data['createdAt'] as Timestamp?).toDate(),
    updateAt: (data['updateAt'] as Timestamp?).toDate(),
  );
}

Map<String, dynamic> toMap() {
  return {
    'code': code,
    'description': description,
    'discountType': discountType == DiscountType.flat ? 'flat' :
'percentage',
    'discountValue': discountValue,
    'startDate': startDate,
    'endDate': endDate,
    'usageLimit': usageLimit,
    'usageCount': usageCount,
    'isActive': isActive,
    'createdAt': createdAt,
    'updateAt': updateAt,
  };
}
```

Trong file `lib/data/services/coupon_service.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/coupon_model.dart';

class CouponService {
  final _firestore = FirebaseFirestore.instance;
  final String collection = "coupons";

  Future<List<CouponModel>> getCoupons() async {
    final snapshot = await _firestore.collection(collection).get();
    return snapshot.docs.map((doc) =>
CouponModel.fromFirestore(doc)).toList();
  }

  Future<String> addCoupon(CouponModel coupon) async {
    final doc = await _firestore.collection(collection).add(coupon.toMap());
    return doc.id;
  }

  Future<void> updateCoupon(CouponModel coupon) async {
    await _firestore
      .collection(collection)
      .doc(coupon.id)
      .update(coupon.toMap());
  }

  Future<void> deleteCoupon(String id) async {
    await _firestore.collection(collection).doc(id).delete();
  }
}
```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/brands/brands_page.dart';
import 'package:draf_project_web/views/coupons/coupons_page.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/cagetories/category_page.dart';
import '../controllers/auth_controller.dart';

import '../views/attributes/attribute_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
  final GlobalKey<NavigatorState> navigatorKey;
  final AuthController authController;
  String _currentPath = "/login";
```



```
AppRouterDelegate(this.authController)
  : navigatorKey = GlobalKey<NavigatorState>() {
  authController.addListener(notifyListeners);
}
@override
String? get currentConfiguration => _currentPath;
@override
Widget build(BuildContext context) {
  if (!authController.isLoggedIn) {
    return Navigator(
      key: navigatorKey,
      pages: const [MaterialPage(child: LoginPage())],
      onPopPage: (route, result) => route.didPop(result),
    );
  }
  Widget page;
  switch (_currentPath) {
    case "/attributes":
      page = const AttributesPage();
      break;
    case "/brands":
      page = const BrandsPage();
      break;
    case "/coupons":
      page = const CouponsPage();
      break;

    case "/categories":
      page = const CategoriesPage();
      break;

    default:
      page = const Center(child: Text("Dashboard Page"));
  }
  return Navigator(
    key: navigatorKey,
    pages: [
      MaterialPage(
        child: AdminLayout(
          currentRoute: _currentPath, // truyền route hiện tại
          onNavigate: (path) {
            _currentPath = path;
            notifyListeners();
          },
          child: page,
        ),
      ),
    ],
    onPopPage: (route, result) => route.didPop(result),
  );
}
```

```
}

@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

8. Thiết kế và phát triển nhóm chức năng quản lý thông tin sản phẩm

Trong file `lib/views/products/product_list_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/product_controller.dart';
import '../data/services/product_service.dart';
import 'product_form_page.dart';

class ProductListPage extends StatelessWidget {
  const ProductListPage({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => ProductController(),
      child: const Scaffold(
        backgroundColor: Color(0xFFF5F7FA),
        body: _ProductListView(),
      ),
    );
  }
}

class _ProductListView extends StatelessWidget {
  const _ProductListView();
```

```

@override
Widget build(BuildContext context) {
  final controller = context.watch<ProductController>();
  final service = ProductService();

  return Padding(
    padding: const EdgeInsets.all(24.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // --- HEADER ---
        const Text(
          "Product Inventory",
          style: TextStyle(
            fontSize: 28,
            fontWeight: FontWeight.bold,
            color: Color(0xFF1A1C1E),
          ),
        ),
        const SizedBox(height: 24),

        // --- TOOLBAR ---
        Row(
          children: [
            Expanded(
              child: Container(
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(12),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withOpacity(0.05),
                      blurRadius: 10,
                    ),
                  ],
                ),
            ),
            child: TextField(
              decoration: const InputDecoration(
                hintText: "Search products...",
                prefixIcon: Icon(Icons.search, color:
Colors.blueAccent),
                border: InputBorder.none,
              ),
              onChanged: controller.search,
            ),
          ],
        ),
        const SizedBox(width: 16),
        ElevatedButton.icon(

```

```
onPressed: () => Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const ProductFormPage()),
),
icon: const Icon(Icons.add, color: Colors.white),
label: const Text("ADD PRODUCT"),
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.blueAccent,
  foregroundColor: Colors.white,
  padding: const EdgeInsets.symmetric(
    horizontal: 20,
    vertical: 18,
  ),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(12),
  ),
),
),
],
),
),

const SizedBox(height: 24),

// --- TABLE SECTION (FIXED SCROLL) ---
Expanded(
  child: Container(
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: Colors.grey.shade200),
    ),
    child: ClipRRect(
      borderRadius: BorderRadius.circular(16),
      child: controller.filteredProducts.isEmpty
        ? const Center(child: Text("No products found"))
        : Scrollbar(
            thumbVisibility: true, // Luôn hiện thanh cuộn ngang
            thickness: 8,
            child: SingleChildScrollView(
              scrollDirection: Axis.horizontal, // Cuộn ngang
              child: SingleChildScrollView(
                scrollDirection: Axis.vertical, // Cuộn dọc
                child: ConstrainedBox(
                  constraints: const BoxConstraints(
                    minWidth: 1100,
                  ), // ÉP ĐỘ RỘNG BẢNG ĐỂ HIỆN HẾT CỘT
                  child: DataTable(
                    headingRowColor: WidgetStateProperty.all(
                      const Color(0xFF8FAFC),
                    ),
                  ),
                ),
              ),
            ),
          ),
        ),
    ),
  ),
),
```

=>

```
columnSpacing:
    20, // Thu hẹp khoảng cách giữa các cột
columns: const [
    DataColumn(label: Text("SEQ")),
    DataColumn(label: Text("PRODUCT")),
    DataColumn(label: Text("PRICE")),
    DataColumn(label: Text("TYPE")),
    DataColumn(label: Text("STOCK")),
    DataColumn(
        label: Text("CÒN HÀNG"),
    ), // ➡ THÊM
    DataColumn(label: Text("VISIBLE")),
    DataColumn(label: Text("STATUS")),
    DataColumn(label: Text("ACTION")),
],
rows: List.generate(
    controller.paginatedData.length,
    (index) {
        final item =
            controller.paginatedData[index];
        return DataRow(
            cells: [
                DataCell(Text("${index + 1}")),
                DataCell(
                    Row(
                        children: [
                            ClipRRect(
                                borderRadius:
                                    BorderRadius.circular(8),
                                child: Image.network(
                                    item.thumbnail,
                                    width: 40,
                                    height: 40,
                                    fit: BoxFit.cover,
                                    errorBuilder: (_, __, ___)
                                        =>
                                            const Icon(
                                                Icons.image,
                                                size: 40,
                                            ),
                                ),
                            const SizedBox(width: 10),
                            const SizedBox(
                                width: 150,
                                child: Text(
                                    item.title,
                                    overflow:
                                        TextOverflow.ellipsis,
                                ),
                            ),
                        ],
                    ),
                ],
            ),
        ),
    ],
);
```

```
        ),
      ],
    ),
  ),
  DataCell(
    Text(
      "\${item.price}",
      style: const TextStyle(
        fontWeight: FontWeight.bold,
        color: Colors.green,
      ),
    ),
  ),
  DataCell(Text(item.productType.name))

  DataCell(_buildStockBadge(item.stock))

  DataCell(
    Icon(
      (item.stock - item.soldQuantity)

      ? Icons.check_circle
      : Icons.cancel,
      color:
        (item.stock -
          item.soldQuantity) >
          0
          ? Colors.green
          : Colors.red,
      size: 20,
    ),
  ),
  DataCell(
    _buildVisibilityBadge(item.isDraft)
  ),
  DataCell(
    Icon(
      item.isActive
      ? Icons.check_circle
      : Icons.cancel,
      color: item.isActive
      ? Colors.green
      : Colors.red,
      size: 20,
    ),
  ),
  DataCell(
    Row(
      children: [
```



```
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          IconButton(
            icon: const Icon(Icons.chevron_left),
            onPressed: controller.previousPage,
          ),
          Text(
            "Page ${controller.currentPage + 1} of
${controller.totalPages}",
          ),
          IconButton(
            icon: const Icon(Icons.chevron_right),
            onPressed: controller.nextPage,
          ),
        ],
      ),
    ],
  ),
);
}

Widget _buildStockBadge(int stock) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
    decoration: BoxDecoration(
      color: stock > 0
        ? Colors.green.withOpacity(0.1)
        : Colors.red.withOpacity(0.1),
      borderRadius: BorderRadius.circular(6),
    ),
    child: Text(
      "$stock In Stock",
      style: TextStyle(
        color: stock > 0 ? Colors.green : Colors.red,
        fontSize: 12,
      ),
    ),
  );
}

Widget _buildVisibilityBadge(bool isDraft) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
    decoration: BoxDecoration(
      color: isDraft
        ? Colors.orange.withOpacity(0.1)
        : Colors.blue.withOpacity(0.1),
      borderRadius: BorderRadius.circular(6),
    ),
  ),
);
}
```



```
        child: Text(
          isDraft ? "Draft" : "Published",
          style: TextStyle(
            color: isDraft ? Colors.orange : Colors.blue,
            fontSize: 12,
          ),
        ),
      ),
    );
  }
}

Future<void> _showDeleteDialog(BuildContext context, VoidCallback onConfirm)
{
  return showDialog(
    context: context,
    builder: (_) => AlertDialog(
      title: const Text("Confirm Delete"),
      content: const Text("Are you sure?"),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text("Cancel"),
        ),
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          onPressed: () {
            onConfirm();
            Navigator.pop(context);
          },
          child: const Text("Delete", style: TextStyle(color: Colors.white)),
        ),
      ],
    ),
  );
}
```

Trong file lib/views/products/product_form_page.dart, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:flutter_quill/flutter_quill.dart';
import 'package:provider/provider.dart';
import 'dart:convert';

import '../controllers/product_controller.dart';
import '../data/models/product_model.dart';
import '../data/models/attribute_model.dart';

class ProductFormPage extends StatefulWidget {
  final ProductModel? product; // thêm dòng này
```

```
const ProductFormPage({
  super.key,
  this.product, // thêm dòng này
});

@override
State<ProductFormPage> createState() => _ProductFormPageState();
}

class _ProductFormPageState extends State<ProductFormPage> {
  final _formKey = GlobalKey<FormState>();
  final List<TextEditingController> imageControllers = List.generate(
    4,
    (_) => TextEditingController(),
  );

  final titleController = TextEditingController();
  final skuController = TextEditingController();
  final priceController = TextEditingController();
  final saleController = TextEditingController();
  final stockController = TextEditingController();
  final thumbnailController = TextEditingController();
  final descriptionController = TextEditingController();
  final discountController = TextEditingController();
  final tagController = TextEditingController();

  late QuillController _quillController;

  ProductType productType = ProductType.simple;
  bool isDraft = false;

  String? selectedBrandId;
  List<String> selectedCategoryIds = [];
  List<Map<String, dynamic>> selectedAttributes = [];
  AttributeModel? currentAttribute;
  List<String> additionalImages = [];
  List<String> tempSelectedValues = [];

  @override
  void initState() {
    super.initState();
    final controller = context.read<ProductController>();
    controller.loadInitialData();

    _quillController = QuillController.basic();

    if (widget.product != null) {
      final p = widget.product!;
```

```

titleController.text = p.title;
skuController.text = p.sku ?? "";
priceController.text = p.price.toString();
saleController.text = p.salePrice?.toString() ?? "";
stockController.text = p.stock.toString();
thumbnailController.text = p.thumbnail;

productType = p.productType;
isDraft = p.isDraft;

selectedBrandId = p.brandId;
selectedCategoryIds = p.categoryIds ?? [];
selectedAttributes = p.attributes ?? [];

/// images
final imgs = p.images ?? [];
for (int i = 0; i < imgs.length && i < imageControllers.length; i++) {
  imageControllers[i].text = imgs[i];
}

/// tags
tagController.text = (p.tags ?? []).join(',');

/// description (QUILL)
try {
  if (p.description.isNotEmpty) {
    _quillController = QuillController(
      document: Document.fromJson(
        List<Map<String, dynamic>>.from(jsonDecode(p.description)),
      ),
      selection: const TextSelection.collapsed(offset: 0),
    );
  }
} catch (_) {}
}

@override
void dispose() {
  titleController.dispose();
  skuController.dispose();
  priceController.dispose();
  saleController.dispose();
  stockController.dispose();
  thumbnailController.dispose();
  _quillController.dispose();
  tagController.dispose();
  for (var c in imageControllers) {
    c.dispose();
  }
}

```

```

        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        return Consumer<ProductController>(
            builder: (context, controller, _) {
                if (controller.brands.isEmpty) {
                    return const Scaffold(
                        body: Center(child: CircularProgressIndicator()),
                    );
                }

                return Scaffold(
                    appBar: AppBar(title: const Text("Create Product")),
                    // GIẢI PHÁP: Bọc toàn bộ body bằng SingleChildScrollView
                    body: SingleChildScrollView(
                        padding: const EdgeInsets.all(20),
                        child: Form(
                            key: _formKey,
                            child: Row(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: [
                                    /// ===== LEFT COLUMN (Main Content)
                                    =====
                                    Expanded(
                                        flex: 2,
                                        child: Column(
                                            crossAxisAlignment: CrossAxisAlignment.stretch,
                                            children: [
                                                /// BASIC INFO
                                                Card(
                                                    child: Padding(
                                                        padding: const EdgeInsets.all(16),
                                                        child: Column(
                                                            crossAxisAlignment: CrossAxisAlignment.start,
                                                            children: [
                                                                const Text(
                                                                    "Basic Information",
                                                                    style: TextStyle(
                                                                        fontSize: 18,
                                                                        fontWeight: FontWeight.bold,
                                                                    ),
                                                                ),
                                                                const SizedBox(height: 15),
                                                                TextFormField(
                                                                    controller: titleController,
                                                                    decoration: const InputDecoration(
                                                                        labelText: "Product Title",
                                                                        border: OutlineInputBorder(),

```

FontWeight.bold),

```

    ),
    validator: (v) => v == null || v.isEmpty
      ? "Required"
      : null,
  ),
  const SizedBox(height: 20),
  const Text(
    "Description",
    style: TextStyle(fontWeight:
FontWeight.bold),
  ),
  const SizedBox(height: 8),
  QuillSimpleToolbar(
    controller: _quillController,
    config: const QuillSimpleToolbarConfig(),
  ),
  Container(
    height:
      300, // Tăng thêm không gian cho editor
    decoration: BoxDecoration(
      border: Border.all(
        color: Colors.grey.shade300,
      ),
      borderRadius: const BorderRadius.only(
        bottomLeft: Radius.circular(8),
        bottomRight: Radius.circular(8),
      ),
    ),
  ),
  child: QuillEditor(
    controller: _quillController,
    focusNode: FocusNode(),
    scrollController: ScrollController(),
    config: const QuillEditorConfig(
      padding: EdgeInsets.all(10),
      placeholder:
        "Enter product description...",
    ),
  ),
),
],
),
),
),
const SizedBox(height: 20),

/// PRODUCT CONFIGURATION & MANAGEMENT
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(

```

FontWeight.bold),

value!);

value!);

```
crossAxisAlignment: CrossAxisAlignment.start,
children: [
  const Text(
    "Product Configuration & Management",
    style: TextStyle(
      fontSize: 18,
      fontWeight: FontWeight.bold,
    ),
  ),
  const SizedBox(height: 15),

  /// PRODUCT TYPE
  const Text(
    "Product Type",
    style: TextStyle(fontWeight:
FontWeight.bold),
  ),
  Row(
    children: [
      Expanded(
        child: RadioListTile<ProductType>(
          value: ProductType.simple,
          groupValue: productType,
          onChanged: (value) {
            setState(() => productType =
value!);
          },
          title: const Text("Single"),
        ),
      ),
      Expanded(
        child: RadioListTile<ProductType>(
          value: ProductType.variable,
          groupValue: productType,
          onChanged: (value) {
            setState(() => productType =
value!);
          },
          title: const Text("Variable"),
        ),
      ),
    ],
  ),

  const SizedBox(height: 15),

  /// SKU
  TextFormField(
    controller: skuController,
    decoration: const InputDecoration(
```

```
        labelText: "SKU",
        border: OutlineInputBorder(),
    ),
),

const SizedBox(height: 15),

/// STOCK
TextFormField(
    controller: stockController,
    keyboardType: TextInputType.number,
    enabled: productType == ProductType.simple,
    decoration: const InputDecoration(
        labelText: "Stock",
        border: OutlineInputBorder(),
    ),
    validator: (v) {
        if (productType == ProductType.simple) {
            if (v == null || v.isEmpty)
                return "Stock required";
        }
        return null;
    },
),

const SizedBox(height: 15),

/// PRICE
TextFormField(
    controller: priceController,
    keyboardType:
        const TextInputType.numberWithOptions(
            decimal: true,
        ),
    enabled: productType == ProductType.simple,
    decoration: const InputDecoration(
        labelText: "Price",
        border: OutlineInputBorder(),
    ),
    validator: (v) {
        if (productType == ProductType.simple) {
            if (v == null || v.isEmpty)
                return "Price required";
        }
        return null;
    },
),

const SizedBox(height: 15),
```

```

/// DISCOUNT PRICE
TextFormField(
  controller: saleController,
  keyboardType:
    const TextInputType.numberWithOptions(
      decimal: true,
    ),
  enabled: productType == ProductType.simple,
  decoration: const InputDecoration(
    labelText: "Discount Price",
    border: OutlineInputBorder(),
  ),
  validator: (v) {
    if (v != null && v.isNotEmpty) {
      double? sale = double.tryParse(v);
      double? price = double.tryParse(
        priceController.text,
      );
      if (sale != null &&
        price != null &&
        sale > price) {
        return "Discount must be <= Price";
      }
    }
    return null;
  },
),
],
),
),
),
const SizedBox(height: 20),

Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          "Additional Images",
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 15),

        Wrap(
          spacing: 20,

```



```
runSpacing: 20,
children: List.generate(4, (index) {
  return SizedBox(
    width: 220,
    child: Column(
      crossAxisAlignment:
        CrossAxisAlignment.start,
      children: [
        Text("Image ${index + 1}"),

        const SizedBox(height: 8),

        /// IMAGE PREVIEW
        Container(
          height: 120,
          width: 220,
          decoration: BoxDecoration(
            border: Border.all(
              color: Colors.grey.shade300,
            ),
          ),
          child:
            imageControllers[index]
              .text
              .isEmpty
            ? const Center(
                child: Text("No Image"),
              )
            : Image.network(
                imageControllers[index]
                  .text,
                fit: BoxFit.cover,
                errorBuilder:
                  (_, __, ___) =>
                    const Center(
                      child: Text(
                        "Invalid
URL",
                      ),
                    ),
                ),
              ),
            ),
          ),

        const SizedBox(height: 8),

        /// URL INPUT
        TextFormField(
          controller:
            imageControllers[index],

          decoration: InputDecoration(
```

```

Icon(Icons.clear),

hintText: "Paste image URL",
border:
  const OutlineInputBorder(),
suffixIcon: IconButton(
  icon: const

  onPressed: () {
    imageControllers[index]
      .clear();
    setState(() {});
  },
),
),
onChanged: (_) => setState(()

{}),

),
],
),
);
}),
),
],
),
),
),
const SizedBox(height: 20),

Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          "Product Attributes",
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 15),

        /// ATTRIBUTE DROPDOWN
        DropdownButtonFormField<AttributeModel>(
          value: currentAttribute,
          decoration: const InputDecoration(
            labelText: "Select Attribute",
            border: OutlineInputBorder(),
          ),
          items: controller.attributes

```

```
.where(
  (attr) => !selectedAttributes.any(
    (a) => a["attributeId"] == attr.id,
  ),
)
.map(
  (attr) => DropdownMenuItem(
    value: attr,
    child: Text(attr.name),
  ),
)
.toList(),
onChanged: (attr) {
  setState(() {
    currentAttribute = attr;
  });
},
),

const SizedBox(height: 15),

/// VALUE CHECKBOX
if (currentAttribute != null)
  Column(
    crossAxisAlignment:
      CrossAxisAlignment.start,
    children: [
      Text(
        currentAttribute!.name,
        style: const TextStyle(
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 10),

      Wrap(
        spacing: 10,
        children: currentAttribute!
          .attributeValues
          .map(
            (val) => FilterChip(
              label: Text(val),
              selected: tempSelectedValues
                .contains(val),
              onSelected: (checked) {
                setState(() {
                  if (checked) {
                    tempSelectedValues.add(
                      val,
                    );
                  }
                });
              },
            ),
          ),
      ),
    ],
  ),
),
```

```

ve(
    } else {
        tempSelectedValues.remo
            val,
        );
    }
    });
    },
    ),
    )
    .toList(),
),

const SizedBox(height: 15),

ElevatedButton(
  onPressed: () {
    if (currentAttribute == null ||
        tempSelectedValues.isEmpty)
      return;

    setState(() {
      selectedAttributes.add({
        "attributeId":
          currentAttribute!.id,
        "name": currentAttribute!.name,
        "values": List.from(
          tempSelectedValues,
        ),
      });

      /// reset
      currentAttribute = null;
      tempSelectedValues.clear();
    });
  },
  child: const Text("Add Attribute"),
),
],
),

const SizedBox(height: 20),

/// SELECTED ATTRIBUTE LIST
...selectedAttributes.map(
  (attr) => Card(
    margin: const EdgeInsets.only(bottom:
10),

    child: ListTile(
      title: Text(attr["name"]),

```



```

        title: const Text("Publish"),
    ),
    RadiolistTile<bool>(
        value: true,
        groupValue: isDraft,
        onChanged: (v) =>
            setState(() => isDraft = v!),
        title: const Text("Draft"),
    ),
    const Divider(),
    const SizedBox(height: 10),
    TextFormField(
        controller: thumbnailController,
        decoration: const InputDecoration(
            labelText: "Thumbnail URL",
            border: OutlineInputBorder(),
        ),
        onChanged: (_) => setState(() {}),
    ),
    const SizedBox(height: 10),
    if (thumbnailController.text.isNotEmpty)
        ClipRRect(
            borderRadius: BorderRadius.circular(8),
            child: Image.network(
                thumbnailController.text,
                height: 150,
                width: double.infinity,
                fit: BoxFit.cover,
                errorBuilder:
                    (context, error, stackTrace) =>
                        const Icon(
                            Icons.broken_image,
                            size: 50,
                        ),
            ),
        ),
    ],
),
),
),
const SizedBox(height: 20),

/// BRAND
Card(
    child: Padding(
        padding: const EdgeInsets.all(16),
        child: DropdownButtonFormField<String>(
            value: selectedBrandId,
            items: controller.brands
                .map(

```

```

        (b) => DropdownMenuItem(
          value: b.id,
          child: Text(b.name),
        ),
      ),
      .toList(),
      onChanged: (v) =>
        setState(() => selectedBrandId = v),
      decoration: const InputDecoration(
        labelText: "Brand",
        border: OutlineInputBorder(),
      ),
      validator: (v) =>
        v == null ? "Select brand" : null,
    ),
  ),
),
const SizedBox(height: 20),

/// CATEGORIES
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          "Categories",
          style: TextStyle(fontWeight:
FontWeight.bold),
        ),
        const SizedBox(height: 10),
        ...controller.categories.map(
          (cat) => CheckboxListTile(
            controlAffinity:
              ListTileControlAffinity.leading,
            value:
selectedCategoryIds.contains(cat.id),
            title: Text(cat.name),
            onChanged: (checked) {
              setState(() {
                if (checked!) {
                  selectedCategoryIds.add(cat.id);
                } else {
                  selectedCategoryIds.remove(cat.id);
                }
              });
            },
          ),
        ),
      ],
    ),
  ),
),

```

```

    ],
  ),
),
const SizedBox(height: 20),

Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          "Product Tags",
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 15),

        TextFormField(
          controller: tagController,
          decoration: const InputDecoration(
            labelText: "Enter tags (a,b,c,...)",
            border: OutlineInputBorder(),
            hintText: "summer, tshirt, new",
          ),
          onChanged: (_) => setState(() {}),
        ),

        const SizedBox(height: 10),

        Wrap(
          spacing: 8,
          children: tagController.text
            .split(',')
            .map((e) => e.trim())
            .where((e) => e.isNotEmpty)
            .map((tag) => Chip(label: Text(tag)))
            .toList(),
        ),
      ],
    ),
  ),
),
const SizedBox(height: 30),

/// SAVE BUTTON
SizedBox(

```



```

width: double.infinity,
height: 50,
child: ElevatedButton(
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blueAccent,
    foregroundColor: Colors.white,
  ),
  onPressed: () async {
    if (!_formKey.currentState!.validate()) return;

    final product = ProductModel(
      id: widget.product?.id ?? "",
      title: titleController.text,

      price:
        double.tryParse(priceController.text) ??
0,

      salePrice:
double.tryParse(saleController.text),

      thumbnail: thumbnailController.text,
      productType: productType,

      stock: productType == ProductType.simple
        ? int.tryParse(stockController.text) ?? 0
        : 0,

      soldQuantity: widget.product?.soldQuantity ??
0,

      isActive: !isDraft,
      isDraft: isDraft,
      isDeleted: false,

      images: imageControllers
        .map((c) => c.text.trim())
        .where((url) => url.isNotEmpty)
        .toList(),

      sku: skuController.text,

      /// FIX QUILL
      description: jsonEncode(
        _quillController.document.toDelta().toJson(
),
),

      brandId: selectedBrandId,
      categoryIds: selectedCategoryIds,

```

```

tags: tagController.text
    .split(',')
    .map((e) => e.trim())
    .where((e) => e.isNotEmpty)
    .toList(),

attributes: selectedAttributes,

/// 🔄 default fields (important)
isFeatured: widget.product?.isFeatured ??

false,

0,

0,

0,

0,

0,

likes: widget.product?.likes ?? 0,
);

await controller.save(
  product,
  isUpdate: widget.product != null,
);

if (mounted) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text("Product Saved

Successfully"),

    ),
  );
  Navigator.pop(context);
}
},
child: const Text(
  "SAVE PRODUCT",
  style: TextStyle(fontWeight: FontWeight.bold),

```

```
        ),
      ),
    ),
  ],
),
),
],
),
),
),
);
},
);
}
```

Trong file `lib/controllers/product_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/product_model.dart';
import '../data/models/brand_model.dart';
import '../data/models/category_model.dart';
import '../data/models/attribute_model.dart';
import '../data/services/product_service.dart';
import '../data/services/brand_service.dart';
import '../data/services/category_service.dart';
import '../data/services/attribute_service.dart';
import 'dart:async';

class ProductController extends ChangeNotifier {
  final ProductService productService = ProductService();
  final BrandService brandService = BrandService();
  final CategoryService categoryService = CategoryService();
  final AttributeService attributeService = AttributeService();
  StreamSubscription? _subscription;

  List<BrandModel> brands = [];
  List<CategoryModel> categories = [];
  List<AttributeModel> attributes = [];

  Future<void> loadInitialData() async {
    brands = await brandService.getAllBrands();
    categories = await categoryService.getCategories();
    attributes = await attributeService.getAll().first;
    notifyListeners();
  }

  Future<void> save(ProductModel model, {bool isUpdate = false}) async {
```

```
        if (isUpdate) {
            await productService.update(model);
        } else {
            await productService.create(model);
        }
    }

    List<ProductModel> allProducts = [];
    List<ProductModel> filteredProducts = [];

    int currentPage = 0;
    int rowsPerPage = 10;

    void setData(List<ProductModel> data) {
        if (allProducts.length == data.length) return;

        allProducts = data.where((e) => e.isDeleted == false).toList();
        filteredProducts = allProducts;

        notifyListeners();
    }

    ProductController() {
        _listenProducts();
    }

    void _listenProducts() {
        _subscription = productService.getAll().listen((data) {
            allProducts = data.where((e) => !e.isDeleted).toList();
            filteredProducts = allProducts;
            notifyListeners();
        });
    }

    void search(String keyword) {
        if (keyword.isEmpty) {
            filteredProducts = allProducts;
        } else {
            filteredProducts = allProducts
                .where((e) =>
                    e.title.toLowerCase().contains(keyword.toLowerCase()))
                .toList();
        }
        currentPage = 0;
        notifyListeners();
    }

    List<ProductModel> get paginatedData {
        final start = currentPage * rowsPerPage;
        final end = start + rowsPerPage;
        return filteredProducts.sublist(
```

```
        start,
        end > filteredProducts.length ? filteredProducts.length : end,
    );
}

int get totalPages => filteredProducts.isEmpty
    ? 1
    : (filteredProducts.length / rowsPerPage).ceil();

void nextPage() {
    if (currentPage < totalPages - 1) {
        currentPage++;
        notifyListeners();
    }
}

void previousPage() {
    if (currentPage > 0) {
        currentPage--;
        notifyListeners();
    }
}

@override
void dispose() {
    _subscription?.cancel();
    super.dispose();
}
}
```

Trong file `lib/data/models/product_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

enum ProductType { simple, variable }

class ProductModel {
    final String id;
    final String title;
    final String lowerTitle;
    final String description;
    final String? sku;

    final double price;
    final double? salePrice;

    final String thumbnail;
    final List<String>? images;
}
```

```
final ProductType productType;

final int stock;
final bool? isOutOfStock;
final int soldQuantity;

final String? brandId;

final List<String>? categoryIds;
final List<String>? tags;

final List<Map<String, dynamic>>? attributes;
final List<Map<String, dynamic>>? variations;

final bool isRecommended;
final bool isFeatured;

final bool isActive;
final bool isDraft;
final bool isDeleted;

final bool? onSale;
final DateTime? saleStartDate;
final DateTime? saleEndDate;

final int views;
final double rating;
final int ratingCount;
final int reviewsCount;

final int fiveStarCount;
final int fourStarCount;
final int threeStarCount;
final int twoStarCount;
final int oneStarCount;

final int likes;

final String? createdBy;
final String? updatedBy;

final DateTime? createdAt;
final DateTime? updatedAt;

ProductModel({
  required this.id,
  required this.title,
  String? lowerTitle,
  this.description = '',
```

```
this.sku,  
required this.price,  
this.salePrice,  
required this.thumbnail,  
this.images,  
this.productType = ProductType.simple,  
this.stock = 0,  
this.isOutOfStock,  
this.soldQuantity = 0,  
this.brandId,  
this.tags,  
this.categoryIds,  
this.attributes,  
this.variations,  
this.isRecommended = false,  
this.isFeatured = false,  
this.isActive = true,  
this.isDraft = false,  
this.isDeleted = false,  
this.onSale,  
this.saleStartDate,  
this.saleEndDate,  
this.views = 0,  
this.rating = 0.0,  
this.ratingCount = 0,  
this.reviewsCount = 0,  
this.fiveStarCount = 0,  
this.fourStarCount = 0,  
this.threeStarCount = 0,  
this.twoStarCount = 0,  
this.oneStarCount = 0,  
this.likes = 0,  
this.createdBy,  
this.createdAt,  
this.updatedBy,  
this.updatedAt,  
}) : lowerTitle = lowerTitle ?? title.toLowerCase();  
  
// Convert to Firestore  
Map<String, dynamic> toMap() {  
  return {  
    "title": title,  
    "lowerTitle": lowerTitle,  
    "description": description,  
    "sku": sku,  
    "price": price,  
    "salePrice": salePrice,  
    "thumbnail": thumbnail,  
    "images": images,  
    "productType": productType.name,  
  }  
}
```

```
"stock": stock,
"isOutOfStock": isOutOfStock,
"soldQuantity": soldQuantity,
"brandId": brandId,
"categoryIds": categoryIds,
"tags": tags,
"attributes": attributes,
"variations": variations,
"isRecommended": isRecommended,
"isFeatured": isFeatured,
"isActive": isActive,
"isDraft": isDraft,
"isDeleted": isDeleted,
"onSale": onSale,
"saleStartDate": saleStartDate,
"saleEndDate": saleEndDate,
"views": views,
"rating": rating,
"ratingCount": ratingCount,
"reviewsCount": reviewsCount,
"fiveStarCount": fiveStarCount,
"fourStarCount": fourStarCount,
"threeStarCount": threeStarCount,
"twoStarCount": twoStarCount,
"oneStarCount": oneStarCount,
"likes": likes,
"createdBy": createdBy,
"updatedBy": updatedBy,
"createdAt": createdAt ?? FieldValue.serverTimestamp(),
"updatedAt": FieldValue.serverTimestamp(),
};
}

// From Firestore
factory ProductModel.fromMap(String id, Map<String, dynamic> map) {
  return ProductModel(
    id: id,
    title: map['title'] ?? '',
    lowerTitle: map['lowerTitle'],
    description: map['description'] ?? '',
    sku: map['sku'],
    price: (map['price'] ?? 0).toDouble(),
    salePrice: map['salePrice']?.toDouble(),
    thumbnail: map['thumbnail'] ?? '',
    images: List<String>.from(map['images'] ?? []),
    productType: ProductType.values.firstWhere(
      (e) => e.name == map['productType'],
      orElse: () => ProductType.simple,
    ),
    stock: map['stock'] ?? 0,
```



```

        isOutOfStock: map['isOutOfStock'],
        soldQuantity: map['soldQuantity'] ?? 0,
        brandId: map['brandId'],
        categoryIds: List<String>.from(map['categoryIds'] ?? []),
        tags: List<String>.from(map['tags'] ?? []),
        attributes: List<Map<String, dynamic>>.from(map['attributes'] ?? []),
        variations: List<Map<String, dynamic>>.from(map['variations'] ?? []),
        isRecommended: map['isRecommended'] ?? false,
        isFeatured: map['isFeatured'] ?? false,
        isActive: map['isActive'] ?? true,
        isDraft: map['isDraft'] ?? false,
        isDeleted: map['isDeleted'] ?? false,
        onSale: map['onSale'],
        saleStartDate: map['saleStartDate']?.toDate(),
        saleEndDate: map['saleEndDate']?.toDate(),
        views: map['views'] ?? 0,
        rating: (map['rating'] ?? 0).toDouble(),
        ratingCount: map['ratingCount'] ?? 0,
        reviewsCount: map['reviewsCount'] ?? 0,
        fiveStarCount: map['fiveStarCount'] ?? 0,
        fourStarCount: map['fourStarCount'] ?? 0,
        threeStarCount: map['threeStarCount'] ?? 0,
        twoStarCount: map['twoStarCount'] ?? 0,
        oneStarCount: map['oneStarCount'] ?? 0,
        likes: map['likes'] ?? 0,
        createdBy: map['createdBy'],
        updatedBy: map['updatedBy'],
        createdAt: map['createdAt']?.toDate(),
        updatedAt: map['updatedAt']?.toDate(),
    );
}
}

```

Trong file `lib/data/services/product_service.dart`, soạn mã nguồn có nội dung như sau:

```

import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/product_model.dart';

class ProductService {
    final _db = FirebaseFirestore.instance;
    final String collection = "products";

    /// CREATE
    Future<void> create(ProductModel model) async {
        final docRef = _db.collection(collection).doc(); // tự generate id
        final newModel = ProductModel(
            id: docRef.id,
            title: model.title,

```

```
        lowerTitle: model.lowerTitle,
        description: model.description,
        sku: model.sku,
        price: model.price,
        salePrice: model.salePrice,
        thumbnail: model.thumbnail,
        images: model.images,
        productType: model.productType,
        stock: model.stock,
        isOutOfStock: model.isOutOfStock,
        soldQuantity: model.soldQuantity,
        brandId: model.brandId,
        categoryIds: model.categoryIds,
        tags: model.tags,
        attributes: model.attributes,
        variations: model.variations,
        isRecommended: model.isRecommended,
        isFeatured: model.isFeatured,
        isActive: model.isActive,
        isDraft: model.isDraft,
        isDeleted: model.isDeleted,
        onSale: model.onSale,
        saleStartDate: model.saleStartDate,
        saleEndDate: model.saleEndDate,
        views: model.views,
        rating: model.rating,
        ratingCount: model.ratingCount,
        reviewsCount: model.reviewsCount,
        fiveStarCount: model.fiveStarCount,
        fourStarCount: model.fourStarCount,
        threeStarCount: model.threeStarCount,
        twoStarCount: model.twoStarCount,
        oneStarCount: model.oneStarCount,
        likes: model.likes,
        createdBy: model.createdBy,
        updatedBy: model.updatedBy,
        createdAt: model.createdAt,
        updatedAt: model.updatedAt,
    );

    await docRef.set(newModel.toMap());
}

/// UPDATE
Future<void> update(ProductModel model) async {
    await _db.collection(collection).doc(model.id).update(model.toMap());
}

/// GET ALL
Stream<List<ProductModel>> getAll() {
```

```
        return _db
            .collection(collection)
            .orderBy("updatedAt", descending: true)
            .snapshots()
            .map((snapshot) {
                return snapshot.docs.map((doc) {
                    return ProductModel.fromMap(doc.id, doc.data());
                }).toList();
            });
    }

    /// GET ACTIVE PRODUCTS
    Stream<List<ProductModel>> getActiveProducts() {
        return _db
            .collection(collection)
            .where("isActive", isEqualTo: true)
            .where("isDeleted", isEqualTo: false)
            .orderBy("updatedAt", descending: true)
            .snapshots()
            .map((snapshot) {
                return snapshot.docs
                    .map((doc) => ProductModel.fromMap(doc.id, doc.data()))
                    .toList();
            });
    }

    /// POPULAR PRODUCTS
    Stream<List<ProductModel>> getPopularProducts() {
        return _db
            .collection(collection)
            .where("isActive", isEqualTo: true)
            .orderBy("soldQuantity", descending: true)
            .limit(10)
            .snapshots()
            .map((snapshot) {
                return snapshot.docs
                    .map((doc) => ProductModel.fromMap(doc.id, doc.data()))
                    .toList();
            });
    }

    /// FEATURED PRODUCTS
    Stream<List<ProductModel>> getFeaturedProducts() {
        return _db
            .collection(collection)
            .where("isFeatured", isEqualTo: true)
            .where("isActive", isEqualTo: true)
            .snapshots()
            .map((snapshot) {
                return snapshot.docs
```

```
        .map((doc) => ProductModel.fromMap(doc.id, doc.data()))
        .toList();
    });
}

/// DELETE (soft delete)
Future<void> softDelete(String id) async {
    await _db.collection(collection).doc(id).update({
        "isDeleted": true,
        "updatedAt": FieldValue.serverTimestamp(),
    });
}

/// HARD DELETE
Future<void> delete(String id) async {
    await _db.collection(collection).doc(id).delete();
}
}
```

Trong file `lib/main.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/controllers/brand_controller.dart';
import 'package:draf_project_web/controllers/product_controller.dart';
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_quill/flutter_quill.dart';

import 'package:provider/provider.dart';

import 'firebase_options.dart';

import 'controllers/auth_controller.dart';

import 'controllers/category_controller.dart';

import 'controllers/attribute_controller.dart';

import 'routes/app_routes.dart';

void main() async {
    WidgetsFlutterBinding.ensureInitialized();
    await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);

    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({super.key});
```

```
@override
Widget build(BuildContext context) {
  return MultiProvider(
    providers: [
      ChangeNotifierProvider(create: (_) =>
AuthController()..checkLogin()),

      ChangeNotifierProvider(create: (_) => CategoryController()),
      ChangeNotifierProvider(create: (_) => ProductController()),

      ChangeNotifierProvider(create: (_) => AttributeController()),
      ChangeNotifierProvider(create: (_) => BrandController()),
    ],
    child: Builder(
      builder: (context) {
        final auth = context.watch<AuthController>();

        return MaterialApp.router(
          debugShowCheckedModeBanner: false,

          // ADD THIS BLOCK
          localizationsDelegates: const [
            FlutterQuillLocalizations.delegate, // FIX LỖI
            GlobalMaterialLocalizations.delegate,
            GlobalWidgetsLocalizations.delegate,
            GlobalCupertinoLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en'), Locale('vi')],

          routerDelegate: AppRouterDelegate(auth),
          routeInformationParser: AppRouteParser(),
        );
      },
    ),
  );
}
```

Trong file `lib/routes/app_routes.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/brands/brands_page.dart';
import 'package:draf_project_web/views/coupons/coupons_page.dart';
import 'package:draf_project_web/views/products/product_list_page.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/cagetories/category_page.dart';
import '../controllers/auth_controller.dart';
```

```
import '../views/attributes/attribute_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
  final GlobalKey<NavigatorState> navigatorKey;
  final AuthController authController;
  String _currentPath = "/login";
  AppRouterDelegate(this.authController)
    : navigatorKey = GlobalKey<NavigatorState>() {
    authController.addListener(notifyListeners);
  }
  @override
  String? get currentConfiguration => _currentPath;
  @override
  Widget build(BuildContext context) {
    if (!authController.isLoggedIn) {
      return Navigator(
        key: navigatorKey,
        pages: const [MaterialPage(child: LoginPage())],
        onPopPage: (route, result) => route.didPop(result),
      );
    }
    Widget page;
    switch (_currentPath) {
      case "/attributes":
        page = const AttributesPage();
        break;
      case "/brands":
        page = const BrandsPage();
        break;
      case "/coupons":
        page = const CouponsPage();
        break;
      case "/products":
        page = const ProductListPage();
        break;

      case "/categories":
        page = const CategoriesPage();
        break;

      default:
        page = const Center(child: Text("Dashboard Page"));
    }
    return Navigator(
      key: navigatorKey,
      pages: [
        MaterialPage(
          child: AdminLayout(
            currentRoute: _currentPath, // truyền route hiện tại
```

```
        onNavigate: (path) {
          _currentPath = path;
          notifyListeners();
        },
        child: page,
      ),
    ),
  ],
  onPopPage: (route, result) => route.didPop(result),
);
}

@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

9. Thiết kế và phát triển nhóm chức năng quản lý thông tin đơn hàng

Trong file `lib/views/orders/orders_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../controllers/order_controller.dart';
import '../data/models/order_model.dart';
import 'package:intl/intl.dart';
import 'order_detail_page.dart';

class OrderPage extends StatefulWidget {
  const OrderPage({super.key});

  @override
  State<OrderPage> createState() => _OrderPageState();
}
```

```
class _OrderPageState extends State<OrderPage> {
  final OrderController controller = OrderController();
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    loadData();
  }

  Future<void> loadData() async {
    await controller.fetchOrders();
    if (mounted) {
      setState(() {
        isLoading = false;
      });
    }
  }

  /// LOGIC MÀU SẮC CHO ORDER STATUS
  Color _getOrderStatusColor(String status) {
    switch (status.toLowerCase()) {
      case 'pending':
        return Colors.orange;
      case 'processing':
        return Colors.lightBlue;
      case 'shipped':
        return Colors.blueAccent;
      case 'delivered':
        return Colors.green;
      case 'canceled':
        return Colors.red;
      case 'returned':
        return Colors.purple;
      case 'refunded':
        return Colors.blueGrey;
      default:
        return Colors.grey;
    }
  }

  /// LOGIC MÀU SẮC CHO PAYMENT STATUS
  Color _getPaymentStatusColor(String status) {
    switch (status.toLowerCase()) {
      case 'paid':
        return Colors.green;
      case 'pending':
        return Colors.redAccent; // Để màu đỏ để nhắc nhở chưa thanh toán
      default:
        return Colors.grey;
    }
  }
}
```



```
    }  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      backgroundColor: const Color(0xFFF0F2F5),  
      body: isLoading  
        ? const Center(child: CircularProgressIndicator())  
        : Padding(  
          padding: const EdgeInsets.all(24),  
          child: Column(  
            crossAxisAlignment: CrossAxisAlignment.start,  
            children: [  
              /// HEADER  
              const Text(  
                "Order Management",  
                style: TextStyle(  
                  fontSize: 28,  
                  fontWeight: FontWeight.bold,  
                  color: Color(0xFF1A1C1E),  
                ),  
            ),  
              const SizedBox(height: 24),  
  
              /// SEARCH & REFRESH  
              Row(  
                children: [  
                  _buildSearchBar(),  
                  const SizedBox(width: 16),  
                  _buildRefreshButton(),  
                ],  
              ),  
  
              const SizedBox(height: 24),  
  
              /// DATA TABLE  
              Expanded(  
                child: Container(  
                  decoration: BoxDecoration(  
                    color: Colors.white,  
                    borderRadius: BorderRadius.circular(16),  
                    boxShadow: [  
                      BoxShadow(  
                        color: Colors.black.withOpacity(0.04),  
                        blurRadius: 24,  
                        offset: const Offset(0, 8),  
                      ),  
                    ],  
                  ),  
              ),  
            ],  
          ),  
        ),  
    );  
  }  
}
```

```

child: ClipRRect(
  borderRadius: BorderRadius.circular(16),
  child: SingleChildScrollView(
    scrollDirection: Axis.vertical,
    child: SingleChildScrollView(
      scrollDirection: Axis.horizontal,
      child: DataTable(
        headingRowColor: MaterialStateProperty.all(
          Colors.grey.withOpacity(0.05),
        ),
        dataRowHeight: 75,
        headingRowHeight: 60,
        columnSpacing: 35,
        columns: const [
          DataColumn(label: _HeaderLabel("Seq")),
          DataColumn(label: _HeaderLabel("Order ID")),
          DataColumn(label: _HeaderLabel("Customer")),
          DataColumn(label: _HeaderLabel("Item")),
          DataColumn(label: _HeaderLabel("Order
Status")),

          DataColumn(label: _HeaderLabel("Payment")),
          DataColumn(label: _HeaderLabel("Amount")),
          DataColumn(label: _HeaderLabel("Date")),
          DataColumn(label: _HeaderLabel("Action")),
        ],
        rows: controller.filteredOrders
          .asMap()
          .entries
          .map((entry) {
            return _buildDataRow(
              entry.value,
              entry.key,
            );
          })
          .toList(),
      ),
    ),
  ),
);
}

DataRow _buildDataRow(OrderModel order, int index) {
  return DataRow(
    cells: [

```

```

DataCell(Text("${index + 1}")),
DataCell(
  Text(
    "#${order.id}",
    style: const TextStyle(
      fontWeight: FontWeight.bold,
      color: Colors.blue,
    ),
  ),
),
DataCell(
  Row(
    children: [
      CircleAvatar(
        radius: 14,
        backgroundColor: Colors.blue.withOpacity(0.1),
        child: Text(
          order.customerName[0],
          style: const TextStyle(fontSize: 10),
        ),
      ),
      const SizedBox(width: 8),
      Text(order.customerName),
    ],
  ),
),
DataCell(Text("${order.itemCount} sp")),

/// ORDER STATUS COLUMN
DataCell(
  _buildBadge(
    order.orderStatus,
    _getOrderStatusColor(order.orderStatus),
  ),
),

/// PAYMENT STATUS COLUMN
DataCell(
  _buildBadge(
    order.paymentStatus,
    _getPaymentStatusColor(order.paymentStatus),
    isOutline: true,
  ),
),

DataCell(
  Text(
    "\${order.totalAmount.toStringAsFixed(0)}",
    style: const TextStyle(fontWeight: FontWeight.bold),
  ),
),

```

```

    ),
    DataCell(Text(DateFormat('dd/MM/yyyy').format(order.orderDate))),
    DataCell(
      Row(
        children: [
          _buildActionIcon(Icons.remove_red_eye, Colors.blue, () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (_) => OrderDetailPage(order: order),
              ),
            );
          }),
          const SizedBox(width: 8),
          _buildActionIcon(
            Icons.delete_sweep,
            Colors.red,
            () => _confirmDelete(order),
          ),
        ],
      ),
    ),
  ],
);
}

/// Widget dùng chung cho các loại Tag/Badge
Widget _buildBadge(String text, Color color, {bool isOutline = false}) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
    decoration: BoxDecoration(
      color: isOutline ? Colors.transparent : color.withOpacity(0.12),
      borderRadius: BorderRadius.circular(8),
      border: isOutline ? Border.all(color: color.withOpacity(0.5)) : null,
    ),
    child: Text(
      text.toUpperCase(),
      style: TextStyle(
        color: color,
        fontSize: 10,
        fontWeight: FontWeight.bold,
        letterSpacing: 0.5,
      ),
    ),
  );
}

Widget _buildSearchBar() {
  return Container(
    width: 320,

```

```

padding: const EdgeInsets.symmetric(horizontal: 16),
decoration: BoxDecoration(
  color: Colors.white,
  borderRadius: BorderRadius.circular(12),
  border: Border.all(color: Colors.grey.withOpacity(0.2)),
),
child: TextField(
  onChanged: (v) => setState(() => controller.searchOrder(v)),
  decoration: const InputDecoration(
    icon: Icon(Icons.search, color: Colors.grey),
    hintText: "Tìm mã đơn, tên khách...",
    border: InputBorder.none,
  ),
),
);
}

Widget _buildRefreshButton() {
  return ElevatedButton.icon(
    onPressed: loadData,
    icon: const Icon(Icons.refresh, size: 18),
    label: const Text("Làm mới"),
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.white,
      foregroundColor: Colors.blue,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 18),
      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
      elevation: 0,
      side: const BorderSide(color: Color(0xFFE0E0E0)),
    ),
  );
}

Widget _buildActionIcon(IconData icon, Color color, VoidCallback onTap) {
  return IconButton(
    onPressed: onTap,
    icon: Icon(icon, color: color, size: 20),
    constraints: const BoxConstraints(),
    padding: const EdgeInsets.all(8),
    splashRadius: 24,
  );
}

void _confirmDelete(OrderModel order) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text("Xác nhận xóa"),
      content: Text("Bạn có chắc chắn muốn xóa đơn hàng #${order.id}?"),
    ),
  );
}

```

```
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text("Hủy"),
          ),
          TextButton(
            onPressed: () async {
              await controller.deleteOrder(order.docId);
              Navigator.pop(context);
              loadData();
            },
            child: const Text("Xóa", style: TextStyle(color: Colors.red)),
          ),
        ],
      ),
    );
  }
}

class _HeaderLabel extends StatelessWidget {
  final String label;
  const _HeaderLabel(this.label);
  @override
  Widget build(BuildContext context) {
    return Text(
      label,
      style: const TextStyle(
        fontWeight: FontWeight.bold,
        color: Colors.black87,
      ),
    );
  }
}
```

Trong file `lib/views/orders/order_detail_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import '../data/models/order_model.dart';
import '../controllers/order_controller.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:intl/intl.dart';

class OrderDetailPage extends StatefulWidget {
  final OrderModel order;

  const OrderDetailPage({super.key, required this.order});

  @override
```

```
State<OrderDetailPage> createState() => _OrderDetailPageState();
}

class _OrderDetailPageState extends State<OrderDetailPage> {
  final OrderController controller = OrderController();
  String selectedStatus = "";
  Map<String, dynamic>? customerData;

  final List<String> statuses = [
    "Created",
    "Pending",
    "Processing",
    "Shipped",
    "Delivered",
    "Cancelled",
    "Returned",
    "Refunded",
  ];

  @override
  void initState() {
    super.initState();
    selectedStatus = widget.order.orderStatus.capitalize();
    fetchCustomer();
  }

  Future<void> fetchCustomer() async {
    final doc = await FirebaseFirestore.instance
      .collection('users')
      .doc(widget.order.userId)
      .get();

    if (mounted) {
      setState(() {
        customerData = doc.data();
      });
    }
  }

  // Lấy màu sắc theo trạng thái để đồng bộ UI
  Color _getStatusColor(String status) {
    switch (status.toLowerCase()) {
      case 'delivered':
        return Colors.green;
      case 'pending':
        return Colors.orange;
      case 'processing':
        return Colors.blue;
      case 'shipped':
        return Colors.indigo;
    }
  }
}
```

```

        case 'canceled':
            return Colors.red;
        default:
            return Colors.blueGrey;
    }
}

void _showTransactionDialog(OrderModel order) {
    final amountController = TextEditingController();
    DateTime? selectedDate;
    String selectedStatus = order.paymentStatus;

    showDialog(
        context: context,
        builder: (context) {
            return AlertDialog(
                title: const Text("Cập nhật giao dịch"),
                content: StatefulBuilder(
                    builder: (context, setStateDialog) {
                        return Column(
                            mainAxisAlignment: MainAxisAlignment.min,
                            children: [
                                /// Payment Status
                                DropdownButtonFormField<String>(
                                    value: selectedStatus,
                                    items: ["pending", "paid", "failed"]
                                        .map(
                                            (e) => DropdownMenuItem(
                                                value: e,
                                                child: Text(e.toUpperCase()),
                                            ),
                                        )
                                        .toList(),
                                    onChanged: (value) {
                                        selectedStatus = value!;
                                    },
                                ),

                                const SizedBox(height: 12),

                                /// Shipping Date
                                ElevatedButton(
                                    onPressed: () async {
                                        final picked = await showDatePicker(
                                            context: context,
                                            initialDate: DateTime.now(),
                                            firstDate: DateTime(2023),
                                            lastDate: DateTime(2100),
                                        );
                                    },
                                ),
                            ],
                        );
                    },
                ),
            );
        },
    );
}

```



```

        if (picked != null) {
            setStateDialog(() {
                selectedDate = picked;
            });
        }
    },
    child: Text(
        selectedDate == null
            ? "Chọn ngày giao hàng"
            : selectedDate.toString(),
    ),
),

const SizedBox(height: 12),

/// Amount Received
TextField(
    controller: amountController,
    keyboardType: TextInputType.number,
    decoration: const InputDecoration(
        labelText: "Số tiền đã nhận",
        border: OutlineInputBorder(),
    ),
),
],
);
},
),
actions: [
    TextButton(
        child: const Text("Huỷ"),
        onPressed: () => Navigator.pop(context),
    ),
    ElevatedButton(
        child: const Text("Cập nhật"),
        onPressed: () async {
            double amountReceived =
                double.tryParse(amountController.text) ?? 0;

            await controller.updateTransaction(
                order: order,
                amountReceived: amountReceived,
                shippingDate: selectedDate,
                paymentStatus: selectedStatus,
            );

            Navigator.pop(context);

            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text("Cập nhật thành công")),
            ),
        ),
    ),
],
),
),
);
}
}

```

```

        );
        setState(() {});
      },
    ),
  ],
);
},
);
}

@override
Widget build(BuildContext context) {
  final order = widget.order;

  return Scaffold(
    backgroundColor: const Color(0xFFF8F9FA),
    appBar: AppBar(
      title: Text(
        "Chi tiết đơn hàng #${order.id.characters.take(8)}",
        style: const TextStyle(
          fontWeight: FontWeight.bold,
          color: Colors.black87,
        ),
      ),
    ),
    backgroundColor: Colors.white,
    elevation: 0.5,
    leading: const BackButton(color: Colors.black87),
  ),
  body: SingleChildScrollView(
    child: Padding(
      padding: const EdgeInsets.all(24),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          /// LEFT COLUMN
          Expanded(
            flex: 8,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                _buildTopStatusBanner(order),
                const SizedBox(height: 24),

                _buildAnimatedFrame(
                  index: 1,
                  title: "Thông tin chung",
                  icon: Icons.info_outline,
                  child: Column(
                    children: [

```

```

        _infoRow(
            "Ngày đặt",
            DateFormat(
                'dd/MM/yyyy HH:mm',
            ).format(order.orderDate),
        ),
        _infoRow(
            "Số lượng mục",
            "${order.itemCount} sản phẩm",
        ),
        _infoRow(
            "Tổng thanh toán",
            "\${order.totalAmount.toStringAsFixed(0)}",
            isPrice: true,
        ),
    ],
),
),
const SizedBox(height: 24),

_buildAnimatedFrame(
    index: 2,
    title: "Sản phẩm đã mua",
    icon: Icons.shopping_bag_outlined,
    child: Column(
        children: [
            _buildProductTable(order),
            const Padding(
                padding: EdgeInsets.symmetric(vertical: 16),
                child: Divider(),
            ),
            _summaryRow(
                "Tạm tính",
                "\${order.subTotal.toStringAsFixed(0)}",
            ),
            _summaryRow(
                "Giảm giá Coupon",
                "-\${order.couponDiscountAmount}",
                color: Colors.red,
            ),
            _summaryRow(
                "Phí vận chuyển",
                "\${order.shippingAmount}",
            ),
            _summaryRow(
                "Thuế",
                "\${order.taxAmount.toStringAsFixed(0)}",
            ),
            const Divider(height: 32, thickness: 1),
            _summaryRow(

```

```

        "Tổng cộng",
        "\${order.totalAmount.toStringAsFixed(0)}",
        bold: true,
    ),
  ],
),
),
const SizedBox(height: 24),

_buildAnimatedFrame(
  index: 3,
  title: "Giao dịch: (${order.paymentMethod})",
  icon: Icons.account_balance_wallet_outlined,
  child: Column(
    children: [
      /// UPDATE BUTTON
      Align(
        alignment: Alignment.centerRight,
        child: ElevatedButton.icon(
          onPressed: () => _showTransactionDialog(order),
          icon: const Icon(Icons.edit, size: 16),
          label: const Text("Cập nhật giao dịch"),
        ),
      ),

      const SizedBox(height: 16),

      _infoRow(
        "Trạng thái",
        order.paymentStatus.toUpperCase(),
        color: order.paymentStatus == "paid"
          ? Colors.green
          : Colors.orange,
      ),

      _infoRow(
        "Ngày vận chuyển",
        order.shippingDate?.toString() ??
          "Chưa có thông tin",
      ),

      _infoRow(
        "Số tiền khớp",
        order.paymentStatus == "pending"
          ? "\$0"
          :
        "\${order.totalAmount.toStringAsFixed(0)}",
      ),
    ],
  ),
),
],

```



```

    ),
  ),
  const SizedBox(width: 12),
  Expanded(
    child: Column(
      crossAxisAlignment:
        CrossAxisAlignment.start,
      children: [
        Text(
          "${customerData!['firstName']}"
        ),
        Text(
          "${customerData!['lastName']}",
          style: const TextStyle(
            fontWeight: FontWeight.bold,
            fontSize: 16,
          ),
        ),
        Text(
          customerData!['email'],
          style: TextStyle(
            color: Colors.grey[600],
            fontSize: 13,
          ),
        ),
      ],
    ),
  ),
),
const SizedBox(height: 24),

// Shipping Address
_buildAnimatedFrame(
  index: 6,
  title: "Địa chỉ giao hàng",
  icon: Icons.local_shipping_outlined,
  child: Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Icon(
        Icons.location_on,
        color: Colors.redAccent,
        size: 20,
      ),
      const SizedBox(width: 8),
      Expanded(
        child: Text(
          "${order.shippingAddress['number']}
          ${order.shippingAddress['street']}, ${order.shippingAddress['ward']},
          ${order.shippingAddress['city']}",

```

```

style: const TextStyle(height: 1.5, fontSize:
13),
    ),
    ),
  ],
),
),
const SizedBox(height: 24),

/// --- PHẦN LỊCH SỬ HOẠT ĐỘNG (TRACKING STYLE) ---
_buildAnimatedFrame(
  index: 7,
  title: "Lịch sử hoạt động",
  icon: Icons.history,
  child: Column(
    children: [
      // 1. Mốc Khởi tạo (Cố định)
      _buildActivityItem(
        title: "Đơn hàng được khởi tạo",
        subtitle: "Khách hàng đã đặt đơn thành công",
        time: DateFormat(
          'dd/MM/yyyy HH:mm',
        ).format(order.orderDate),
        isLast: false,
        iconColor: Colors.blue,
      ),
      // 2. Mốc Cập nhật (Lần cuối)
      _buildActivityItem(
        title:
          "Trạng thái:
${order.orderStatus.toUpperCase()}",
        subtitle: "Cập nhật lần cuối bởi hệ thống",
        time: DateFormat(
          'dd/MM/yyyy HH:mm',
        ).format(order.updatedAt),
        isLast: true,
        iconColor: _getStatusColor(order.orderStatus),
      ),
    ],
  ),
),
),
),
),
),
),
),
),
),
),
);
}

```



```
// Widget vẽ từng dòng Timeline
Widget _buildActivityItem({
  required String title,
  required String subtitle,
  required String time,
  required bool isLast,
  required Color iconColor,
}) {
  return IntrinsicHeight(
    child: Row(
      children: [
        Column(
          children: [
            Container(
              width: 12,
              height: 12,
              decoration: BoxDecoration(
                color: iconColor,
                shape: BoxShape.circle,
                border: Border.all(
                  color: iconColor.withOpacity(0.2),
                  width: 3,
                ),
              ),
            ),
            if (!isLast)
              Expanded(
                child: VerticalDivider(thickness: 2, color:
Colors.grey[300]),
              ),
          ],
        ),
        const SizedBox(width: 16),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                title,
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                  fontSize: 13,
                ),
              ),
              Text(
                subtitle,
                style: TextStyle(color: Colors.grey[600], fontSize: 12),
              ),
              const SizedBox(height: 4),
            ],
          ),
        ),
      ],
    ),
  );
}
```

```

        Text(
          time,
          style: TextStyle(
            color: Colors.blueGrey[300],
            fontSize: 11,
            fontWeight: FontWeight.w500,
          ),
        ),
        if (!isLast) const SizedBox(height: 20),
      ],
    ),
  ],
),
);
}

// --- HÀM HELPER GIỮ NGUYÊN TỪ BẢN TRƯỚC ---

Widget _buildTopStatusBanner(OrderModel order) {
  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: [Colors.blue.shade800, Colors.blue.shade500],
      ),
      borderRadius: BorderRadius.circular(16),
    ),
    child: Row(
      children: [
        const Icon(Icons.shopping_bag, color: Colors.white, size: 40),
        const SizedBox(width: 16),
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              "ĐƠN HÀNG:
              ${order.id.characters.take(8).toString().toUpperCase()}",
              style: const TextStyle(
                color: Colors.white,
                fontWeight: FontWeight.bold,
                fontSize: 18,
              ),
            ),
            Text(
              "Trạng thái thanh toán:
              ${order.paymentStatus.toUpperCase()}",
              style: const TextStyle(color: Colors.white70),
            ),
          ],
        ),
      ],
    ),
  );
}

```

```

        ),
      ],
    ),
  );
}

Widget _buildAnimatedFrame({
  required int index,
  required String title,
  required IconData icon,
  required Widget child,
}) {
  return TweenAnimationBuilder(
    duration: Duration(milliseconds: 400 + (index * 100)),
    tween: Tween<double>(begin: 0, end: 1),
    builder: (context, double value, child) {
      return Opacity(
        opacity: value,
        child: Transform.translate(
          offset: Offset(0, 20 * (1 - value)),
          child: child,
        ),
      );
    },
    child: Card(
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: Colors.grey.shade200),
      ),
      child: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Icon(icon, size: 18, color: Colors.blueAccent),
                const SizedBox(width: 8),
                Text(
                  title,
                  style: const TextStyle(
                    fontWeight: FontWeight.bold,
                    fontSize: 15,
                  ),
                ),
              ],
            ),
            const Divider(height: 24),
            child,
          ],
        ),
      ),
    ),
  );
}

```

```
    ],
  ),
),
);
}

Widget _buildProductTable(OrderModel order) {
  return DataTable(
    headingRowColor: MaterialStateProperty.all(Colors.grey[50]),
    columnSpacing: 15,
    columns: const [
      DataColumn(
        label: Text(
          "SẢN PHẨM",
          style: TextStyle(fontSize: 11, fontWeight: FontWeight.bold),
        ),
      ),
      DataColumn(
        label: Text(
          "ĐƠN GIÁ",
          style: TextStyle(fontSize: 11, fontWeight: FontWeight.bold),
        ),
      ),
      DataColumn(
        label: Text(
          "SL",
          style: TextStyle(fontSize: 11, fontWeight: FontWeight.bold),
        ),
      ),
      DataColumn(
        label: Text(
          "TỔNG",
          style: TextStyle(fontSize: 11, fontWeight: FontWeight.bold),
        ),
      ),
    ],
    rows: order.products.map<DataRow>((item) {
      return DataRow(
        cells: [
          DataCell(
            Row(
              children: [
                ClipRRect(
                  borderRadius: BorderRadius.circular(6),
                  child: Image.network(
                    item['image'],
                    width: 35,
                    height: 35,
                    fit: BoxFit.cover,

```

```
        ),
      ),
      const SizedBox(width: 10),
      Expanded(
        child: Text(
          item['title'],
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontSize: 12),
        ),
      ),
    ],
  ),
),
DataCell(Text("\${item['price']}")),
DataCell(Text("x${item['quantity']}")),
DataCell(
  Text(
    "\${(item['price'] * item['quantity'])}",
    style: const TextStyle(fontWeight: FontWeight.bold),
  ),
),
],
);
}).toList(),
);
}

Widget _infoRow(
  String title,
  String value, {
  Color? color,
  bool isPrice = false,
}) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 10),
    child: Row(
      children: [
        SizedBox(
          width: 120,
          child: Text(
            title,
            style: TextStyle(color: Colors.grey[600], fontSize: 13),
          ),
        ),
        Expanded(
          child: Text(
            value,
            style: TextStyle(
              fontWeight: isPrice ? FontWeight.bold : FontWeight.w500,
              color: color ?? Colors.black87,
            ),
          ),
        ),
      ],
    ),
  );
}
```

```
        fontSize: isPrice ? 15 : 13,
      ),
    ),
  ],
),
);
}

Widget _summaryRow(
  String title,
  String value, {
  bool bold = false,
  Color? color,
}) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 3),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          title,
          style: TextStyle(
            fontWeight: bold ? FontWeight.bold : null,
            fontSize: bold ? 15 : 13,
            color: bold ? Colors.black : Colors.grey[700],
          ),
        ),
        Text(
          value,
          style: TextStyle(
            fontWeight: bold ? FontWeight.bold : FontWeight.w600,
            fontSize: bold ? 17 : 13,
            color: color ?? (bold ? Colors.blue.shade800 : Colors.black87),
          ),
        ),
      ],
    ),
  );
}

extension CapExtension on String {
  String capitalize() =>
    isEmpty ? "" : "${this[0].toUpperCase()}${substring(1)}";
}
```

Trong file `lib/controllers/order_controller.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

import '../data/models/order_model.dart';
import '../data/services/order_service.dart';

class OrderController {
  final OrderService _service = OrderService();

  List<OrderModel> orders = [];
  List<OrderModel> filteredOrders = [];

  Future<void> fetchOrders() async {
    orders = await _service.getAllOrders();

    for (var order in orders) {
      final user = await _service.getUserById(order.userId);

      if (user != null) {
        order.customerName =
          "${user['firstName'] ?? ''} ${user['lastName'] ?? ''}";
      }
    }

    filteredOrders = orders;
  }

  void searchOrder(String keyword) {
    if (keyword.isEmpty) {
      filteredOrders = orders;
    } else {
      filteredOrders = orders
        .where(
          (o) =>
            o.id.toLowerCase().contains(keyword.toLowerCase()) ||
            o.customerName.toLowerCase().contains(keyword.toLowerCase()),
        )
        .toList();
    }
  }

  Future<void> deleteOrder(String docId) async {
    await _service.deleteOrder(docId);
    orders.removeWhere((e) => e.docId == docId);
    filteredOrders = orders;
  }

  Future<void> updateOrderStatus(OrderModel order, String newStatus) async {
    final oldStatus = order.orderStatus.toLowerCase();
    final lowerNewStatus = newStatus.toLowerCase();
  }
}
```

```
const revertStatuses = ["canceled", "returned", "refunded"];

if (revertStatuses.contains(lowerNewStatus) &&
    !revertStatuses.contains(oldStatus)) {
  await _service.handleOrderRevertStock(order);
}

/// UPDATE ORDER
await _service.updateOrderStatus(order.docId, newStatus);

/// CREATE NOTIFICATION
await FirebaseFirestore.instance.collection('notifications').add({
  'userId': order.userId,
  'orderId': order.id,
  'orderStatus': newStatus,
  'message':
    'Đơn hàng ${order.id} của bạn đã được cập nhật trạng thái
$newStatus',
  'isRead': false,
  'createdAt': FieldValue.serverTimestamp(),
});
}

Future<void> updateTransaction({
  required OrderModel order,
  required double amountReceived,
  required DateTime? shippingDate,
  required String paymentStatus,
}) async {
  String finalStatus = paymentStatus;

  if (amountReceived >= order.totalAmount) {
    finalStatus = "paid";
  }

  await _service.updateTransaction(
    docId: order.docId,
    paymentStatus: finalStatus,
    shippingDate: shippingDate,
    amountCaptured: amountReceived,
  );
}
```

Trong file `lib/data/models/order_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
```



```
class OrderModel {
    final String docId;
    final String id;
    final String userId;

    final List<dynamic> products;

    final double subTotal;
    final double taxAmount;
    final double taxRate;
    final double shippingAmount;

    final double totalDiscountAmount;
    final double couponDiscountAmount;
    final double totalAmount;

    final String paymentStatus;
    final String orderStatus;

    final String paymentMethod;
    final String paymentMethodType;

    final int itemCount;

    final DateTime orderDate;
    final DateTime createdAt;
    final DateTime updatedAt;
    final DateTime? shippingDate;

    final Map<String, dynamic> shippingAddress;

    // 📌 Admin list helper fields
    String customerName;

    OrderModel({
        required this.docId,
        required this.id,
        required this.userId,
        required this.products,
        required this.subTotal,
        required this.taxAmount,
        required this.taxRate,
        required this.shippingAmount,
        required this.totalDiscountAmount,
        required this.couponDiscountAmount,
        required this.totalAmount,
        required this.paymentStatus,
        required this.orderStatus,
        required this.paymentMethod,
```

```
required this.paymentMethodType,
required this.itemCount,
required this.orderDate,
required this.createdAt,
required this.updatedAt,
required this.shippingAddress,
this.shippingDate,
this.customerName = '',
});

factory OrderModel.fromFirestore(DocumentSnapshot doc) {
  final data = doc.data() as Map<String, dynamic>;

  return OrderModel(
    docId: doc.id,
    id: data['id'] ?? '',
    userId: data['userId'] ?? '',
    products: data['products'] ?? [],
    subTotal: (data['subTotal'] ?? 0).toDouble(),
    taxAmount: (data['taxAmount'] ?? 0).toDouble(),
    taxRate: (data['taxRate'] ?? 0).toDouble(),
    shippingAmount: (data['shippingAmount'] ?? 0).toDouble(),
    totalDiscountAmount: (data['totalDiscountAmount'] ?? 0).toDouble(),
    couponDiscountAmount: (data['couponDiscountAmount'] ?? 0).toDouble(),
    totalAmount: (data['totalAmount'] ?? 0).toDouble(),
    paymentStatus: data['paymentStatus'] ?? '',
    orderStatus: data['orderStatus'] ?? '',
    paymentMethod: data['paymentMethod'] ?? '',
    paymentMethodType: data['paymentMethodType'] ?? '',
    itemCount: data['itemCount'] ?? 0,
    orderDate: (data['orderDate'] as Timestamp).toDate(),
    createdAt: (data['createdAt'] as Timestamp).toDate(),
    updatedAt: (data['updatedAt'] as Timestamp).toDate(),
    shippingDate: (data['shippingDate'] as Timestamp?)?.toDate(),
    shippingAddress: data['shippingAddress'] ?? {},
  );
}
```

Trong file `lib/data/services/order_service.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/order_model.dart';

class OrderService {
  final _db = FirebaseFirestore.instance;

  Future<List<OrderModel>> getAllOrders() async {
```

```
        final snapshot = await _db
            .collection('orders')
            .orderBy('createdAt', descending: true)
            .get();

        return snapshot.docs.map((e) => OrderModel.fromFirestore(e)).toList();
    }

    Future<void> deleteOrder(String docId) async {
        await _db.collection('orders').doc(docId).delete();
    }

    Future<Map<String, dynamic>?> getUserById(String userId) async {
        final doc = await _db.collection('users').doc(userId).get();
        return doc.data();
    }

    Future<void> updateOrderStatus(String docId, String newStatus) async {
        await _db.collection('orders').doc(docId).update({
            'orderStatus': newStatus.toLowerCase(),
            'updatedAt': FieldValue.serverTimestamp(),
        });
    }

    Future<void> updateTransaction({
        required String docId,
        required String paymentStatus,
        required DateTime? shippingDate,
        required double amountCaptured,
    }) async {
        await _db.collection('orders').doc(docId).update({
            'paymentStatus': paymentStatus.toLowerCase(),
            'shippingDate': shippingDate,
            'amountCaptured': amountCaptured,
            'updatedAt': FieldValue.serverTimestamp(),
        });
    }

    Future<void> handleOrderRevertStock(OrderModel order) async {
        final batch = _db.batch();

        for (var item in order.products) {
            final String productId = item['productId'];
            final int quantity = item['quantity'];

            final productRef = _db.collection('products').doc(productId);
            final productSnap = await productRef.get();

            if (!productSnap.exists) continue;
        }
    }
}
```

```
final data = productSnap.data()!;
final int currentSold = data['soldQuantity'] ?? 0;
final int stock = data['stock'] ?? 0;

/// ===== REVERT SOLD =====
int newSold = currentSold - quantity;
if (newSold < 0) newSold = 0;

/// ===== FIX LOGIC OUT OF STOCK =====
bool newOutOfStock;

if (newSold < stock) {
  newOutOfStock = false;
} else {
  newOutOfStock = true;
}

batch.update(productRef, {
  'soldQuantity': newSold,
  'isOutOfStock': newOutOfStock,
  'updatedAt': FieldValue.serverTimestamp(),
});

await batch.commit();
}
```

Trong file `lib/routes/app_routes.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/brands/brands_page.dart';
import 'package:draf_project_web/views/coupons/coupons_page.dart';
import 'package:draf_project_web/views/orders/orders_page.dart';
import 'package:draf_project_web/views/products/product_list_page.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/categories/category_page.dart';
import '../controllers/auth_controller.dart';

import '../views/attributes/attribute_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
  final GlobalKey<NavigatorState> navigatorKey;
  final AuthController authController;
  String _currentPath = "/login";
  AppRouterDelegate(this.authController)
    : navigatorKey = GlobalKey<NavigatorState>() {
    authController.addListener(notifyListeners);
  }
}
```

```
}
@override
String? get currentConfiguration => _currentPath;
@override
Widget build(BuildContext context) {
  if (!authController.isLoggedIn) {
    return Navigator(
      key: navigatorKey,
      pages: const [MaterialPage(child: LoginPage())],
      onPopPage: (route, result) => route.didPop(result),
    );
  }
  Widget page;
  switch (_currentPath) {
    case "/attributes":
      page = const AttributesPage();
      break;
    case "/orders":
      page = const OrderPage();
      break;
    case "/brands":
      page = const BrandsPage();
      break;
    case "/coupons":
      page = const CouponsPage();
      break;
    case "/products":
      page = const ProductListPage();
      break;

    case "/categories":
      page = const CategoriesPage();
      break;

    default:
      page = const Center(child: Text("Dashboard Page"));
  }
  return Navigator(
    key: navigatorKey,
    pages: [
      MaterialPage(
        child: AdminLayout(
          currentRoute: _currentPath, // truyền route hiện tại
          onNavigate: (path) {
            _currentPath = path;
            notifyListeners();
          },
        ),
        child: page,
      ),
    ],
  ),
}
```

```
    ],
    onPopPage: (route, result) => route.didPop(result),
  );
}

@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

10. Thiết kế và phát triển nhóm chức năng quản lý thông tin khách hàng

Trong file `lib/views/customers/customers_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../controllers/customer_controller.dart';
import '../data/models/customer_model.dart';
import 'customer_detail_page.dart';

class CustomersPage extends StatelessWidget {
  const CustomersPage({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => CustomerController()..fetchCustomers(),
      child: const Scaffold(
        backgroundColor: Color(0xFF8F9FD),
        body: _CustomersView(),
      ),
    );
  }
}
```

```
class _CustomersView extends StatelessWidget {
  const _CustomersView();

  @override
  Widget build(BuildContext context) {
    final controller = context.watch<CustomerController>();

    return Padding(
      padding: const EdgeInsets.all(24),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // ===== HEADER =====
          const Text(
            "Customer Management",
            style: TextStyle(
              fontSize: 28,
              fontWeight: FontWeight.bold,
              color: Color(0xFF2B3674),
            ),
          ),
          const SizedBox(height: 20),
          // ===== SEARCH BAR =====
          Container(
            width: 400,
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(15),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 10,
                  offset: const Offset(0, 4),
                ),
              ],
            ),
            child: TextField(
              decoration: const InputDecoration(
                hintText: "Search by name, email or phone...",
                hintStyle: TextStyle(color: Color(0xFFA3AED0), fontSize: 14),
                prefixIcon: Icon(
                  Icons.search_rounded,
                  color: Color(0xFF4318FF),
                ),
                border: InputBorder.none,
                contentPadding: EdgeInsets.symmetric(vertical: 15),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

```

        onChanged: (value) {
          controller.search(value);
        },
      ),
    ),

    const SizedBox(height: 24),

    // ===== TABLE AREA =====
    Expanded(
      child: Container(
        width: double.infinity,
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(20),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.03),
              blurRadius: 20,
              offset: const Offset(0, 10),
            ),
          ],
        ),
      child: controller.isLoading
        ? const Center(child: CircularProgressIndicator())
        : ClipRRect(
            borderRadius: BorderRadius.circular(20),
            child: Column(
              children: [
                Expanded(
                  child: SingleChildScrollView(
                    scrollDirection: Axis.vertical,
                    child: SingleChildScrollView(
                      scrollDirection: Axis.horizontal,
                      child: DataTable(
                        headingRowHeight: 56,
                        dataRowMaxHeight: 70,
                        columnSpacing: 32,
                        headingRowColor: MaterialStateProperty.all(
                          const Color(0xFF4F7FE),
                        ),
                        columns: const [
                          DataColumn(label: _TableLabel("SEQ")),
                          DataColumn(label:
                            _TableLabel("CUSTOMER")),

                          DataColumn(label: _TableLabel("EMAIL")),
                          DataColumn(label: _TableLabel("PHONE")),
                          DataColumn(label: _TableLabel("ORDERS")),
                          DataColumn(
                            label: _TableLabel("REGISTER DATE"),

```



```

        ),
        DataColumn(label: _TableLabel("ACTION")),
    ],
    rows:
controller.paginatedData.asMap().entries.map((
    entry,
) {
    final index = entry.key;
    final c = entry.value;
    final seq =
        (controller.currentPage - 1) *
        controller.rowsPerPage +
        index +
        1;

    return DataRow(
        cells: [
            // SEQ
            DataCell(
                Text(
                    "$seq",
                    style: const TextStyle(
                        color: Color(0xFFA3AED0),
                    ),
                ),
            ),
            // CUSTOMER (Name with Avatar)
            DataCell(
                Row(
                    mainAxisAlignment: MainAxisAlignment.min,
                    children: [
                        CircleAvatar(
                            radius: 16,
                            backgroundColor: const Color(
                                0xFF4318FF,
                            ).withOpacity(0.1),
                        ),
                        Text(
                            c.firstName[0].toUpperCase(

                                style: const TextStyle(
                                    color: Color(0xFF4318FF),
                                    fontSize: 12,
                                    fontWeight:
                                        FontWeight.bold,
                                ),
                            ),
                        ),
                    ],
                ),
            ),
            const SizedBox(width: 12),
            Text(

```

```
        "${c.firstName}
        style: const TextStyle(
            fontWeight:
                color: Color(0xFF2B3674),
        ),
    ],
),
),
// EMAIL
DataCell(
    Text(
        c.email,
        style: const TextStyle(
            color: Color(0xFF1B2559),
        ),
    ),
),
// PHONE
DataCell(
    Text(
        c.phone,
        style: const TextStyle(
            color: Color(0xFF1B2559),
        ),
    ),
),
// ORDERS
DataCell(
    Container(
        padding: const
            horizontal: 12,
            vertical: 4,
        ),
    decoration: BoxDecoration(
        color:
            0.1,
        ),
        borderRadius:
            BorderRadius.circular(12),
    ),
    child: Text(
```

```

d] ?? 0}",

                                "${controller.orderCountMap[c.i
                                style: const TextStyle(
                                    color: Colors.orange,
                                    fontWeight: FontWeight.bold,
                                ),
                                ),
                                ),
                                ),

                                // REGISTER DATE
                                DataCell(
                                    Text(
                                        c.createdAt != null
                                            ?
                                "${c.createdAt!.day}/${c.createdAt!.month}/${c.createdAt!.year}"
                                            : "-",
                                        style: const TextStyle(
                                            color: Color(0xFFA3AED0),
                                        ),
                                    ),
                                ),

                                // ACTION
                                DataCell(
                                    Row(
                                        mainAxisAlignment: MainAxisAlignment.min,
                                        children: [
                                            _CircularActionButton(
                                                icon:

                                Icons.visibility_rounded,

                                Color(0xFF4318FF),

                                                color: const

                                onPressed: () {
                                    Navigator.push(
                                        context,
                                        MaterialPageRoute(
                                            builder: (_) =>
                                                CustomerDetailPage(
                                                    customer: c,
                                                ),
                                            ),
                                        );
                                    },
                                ),
                                const SizedBox(width: 8),
                                _CircularActionButton(
                                    icon: Icons
                                        .delete_outline_rounded,
                                    color: Colors.red,

```



```

        offset: const Offset(0, 4),
      ),
    ],
    : [],
    border: Border.all(
      color: isCurrent
        ? const Color(0xFF4318FF)
        : const Color(0xFFE0E5F2),
    ),
  ),
  child: Text(
    "$page",
    style: TextStyle(
      color: isCurrent
        ? Colors.white
        : const Color(0xFF2B3674),
      fontWeight: FontWeight.bold,
    ),
  ),
),
),
),
),
);
}),
),
],
),
);
}

// ===== DETAIL DIALOG =====
void _showDetail(BuildContext context, CustomerModel customer) async {
  final controller = context.read<CustomerController>();
  final orders = await controller.getOrders(customer.id);

  showDialog(
    context: context,
    builder: (_) => AlertDialog(
      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(24)),
      title: Text(
        "Orders of ${customer.firstName}",
        style: const TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF2B3674),
        ),
      ),
      content: SizedBox(
        width: 500,
        child: orders.isEmpty
          ? const Padding(

```

```

        padding: EdgeInsets.symmetric(vertical: 20),
        child: Text("No orders found.", textAlign:
TextAlign.center),
    )
    : ListView.separated(
        shrinkWrap: true,
        itemCount: orders.length,
        separatorBuilder: (_, __) => const Divider(height: 1),
        itemBuilder: (_, i) {
            final order = orders[i];
            return ListTile(
                contentPadding: EdgeInsets.zero,
                leading: const CircleAvatar(
                    backgroundColor: Color(0xFFF4F7FE),
                    child: Icon(
                        Icons.receipt_long_rounded,
                        color: Color(0xFF4318FF),
                    ),
                ),
                title: Text(
                    "Order ID: ${order['id'] ?? ''}",
                    style: const TextStyle(
                        fontWeight: FontWeight.bold,
                        fontSize: 14,
                    ),
                ),
                subtitle: Text("Total: ${order['totalAmount'] ?? 0}"),
                trailing: const Icon(Icons.chevron_right_rounded),
            );
        },
    ),
    actionsPadding: const EdgeInsets.all(16),
    actions: [
        TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text(
                "Close",
                style: TextStyle(color: Color(0xFFA3AED0)),
            ),
        ),
    ],
);
}

// ===== DELETE CONFIRM =====
void _confirmDelete(BuildContext context, String id) {
    final controller = context.read<CustomerController>();

```

```

        showDialog(
          context: context,
          builder: (_) => AlertDialog(
            shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(20)),
            title: const Text("Delete Customer?"),
            content: const Text(
              "This action cannot be undone. Are you sure you want to remove this
customer?",
            ),
            actions: [
              TextButton(
                onPressed: () => Navigator.pop(context),
                child: const Text("Cancel", style: TextStyle(color:
Colors.grey)),
              ),
              ElevatedButton(
                style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.red,
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
                onPressed: () async {
                  await controller.delete(id);
                  Navigator.pop(context);
                },
                child: const Text("Delete", style: TextStyle(color:
Colors.white)),
              ),
            ],
          ),
        );
      }
    }

    /// --- SUPPORTING WIDGETS ---

class _TableLabel extends StatelessWidget {
  final String label;
  const _TableLabel(this.label);

  @override
  Widget build(BuildContext context) {
    return Text(
      label,
      style: const TextStyle(
        color: Color(0xFFA3AED0),
        fontWeight: FontWeight.bold,
        fontSize: 13,
      ),
    );
  }
}

```

```
    ),  
  );  
}  
}  
  
class _CircularActionButton extends StatelessWidget {  
  final IconData icon;  
  final Color color;  
  final VoidCallback onPressed;  
  
  const _CircularActionButton({  
    required this.icon,  
    required this.color,  
    required this.onPressed,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return Material(  
      color: color.withOpacity(0.08),  
      shape: const CircleBorder(),  
      child: IconButton(  
        icon: Icon(icon, color: color, size: 18),  
        onPressed: onPressed,  
        constraints: const BoxConstraints(minWidth: 36, minHeight: 36),  
        padding: EdgeInsets.zero,  
        hoverColor: color.withOpacity(0.15),  
      ),  
    );  
  }  
}
```

Trong file `lib/views/customers/customer_detail_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/data/models/address_model.dart';  
import 'package:draf_project_web/data/models/customer_model.dart';  
import 'package:draf_project_web/data/models/order_model.dart';  
import 'package:flutter/material.dart';  
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'package:intl/intl.dart';  
  
class CustomerDetailPage extends StatefulWidget {  
  final CustomerModel customer;  
  
  const CustomerDetailPage({super.key, required this.customer});  
  
  @override  
  State<CustomerDetailPage> createState() => _CustomerDetailPageState();  
}
```



```
}  
class _CustomerDetailPageState extends State<CustomerDetailPage> {  
  List<AddressModel> addresses = [];  
  List<OrderModel> orders = [];  
  bool isLoading = true;  
  
  @override  
  void initState() {  
    super.initState();  
    fetchData();  
  }  
  
  Future<void> fetchData() async {  
    final addressSnapshot = await FirebaseFirestore.instance  
      .collection('users')  
      .doc(widget.customer.id)  
      .collection('addresses')  
      .get();  
  
    final orderSnapshot = await FirebaseFirestore.instance  
      .collection('orders')  
      .where('userId', isEqualTo: widget.customer.id)  
      .get();  
  
    addresses = addressSnapshot.docs  
      .map((doc) => AddressModel.fromFirestore(doc))  
      .toList();  
  
    orders = orderSnapshot.docs  
      .map((doc) => OrderModel.fromFirestore(doc))  
      .toList();  
  
    setState(() {  
      isLoading = false;  
    });  
  }  
  
  double get totalSpent =>  
    orders.fold(0, (sum, item) => sum + item.totalAmount);  
  
  double get averageOrderValue =>  
    orders.isEmpty ? 0 : totalSpent / orders.length;  
  
  DateTime? get lastOrderDate {  
    if (orders.isEmpty) return null;  
    orders.sort(  
      (a, b) => (b.orderDate ?? DateTime(2000)).compareTo(  
        a.orderDate ?? DateTime(2000),  
      ),  
    );  
  }  
}
```

```
        return orders.first.orderDate;
    }

    String formatDate(DateTime? date) {
        if (date == null) return "-";
        return DateFormat('dd/MM/yyyy HH:mm').format(date);
    }

    @override
    Widget build(BuildContext context) {
        if (isLoading) {
            return const Scaffold(
                backgroundColor: Color(0xFFF8F9FD),
                body: Center(
                    child: CircularProgressIndicator(color: Color(0xFF4318FF)),
                ),
            );
        }

        return Scaffold(
            backgroundColor: const Color(0xFFF8F9FD),
            appBar: AppBar(
                backgroundColor: Colors.white,
                elevation: 0,
                leading: IconButton(
                    icon: const Icon(
                        Icons.arrow_back_ios_new_rounded,
                        color: Color(0xFF2B3674),
                        size: 20,
                    ),
                    onPressed: () => Navigator.pop(context),
                ),
                title: const Text(
                    "Customer Detail",
                    style: TextStyle(
                        color: Color(0xFF2B3674),
                        fontWeight: FontWeight.bold,
                    ),
                ),
            ),
            body: SingleChildScrollView(
                padding: const EdgeInsets.all(24),
                child: Row(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        /// --- LEFT SIDE: Profile & Address ---
                        Expanded(
                            flex: 4,
                            child: Column(
                                children: [
```

```

_buildSectionCard(
  title: "Customer Information",
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Center(
        child: CircleAvatar(
          radius: 40,
          backgroundColor: const Color(
            0xFF4318FF,
          ).withOpacity(0.1),
          child: Text(
            widget.customer.firstName[0].toUpperCase(),
            style: const TextStyle(
              fontSize: 32,
              fontWeight: FontWeight.bold,
              color: Color(0xFF4318FF),
            ),
          ),
        ),
      ),
      const SizedBox(height: 24),
      infoRow("FName", widget.customer.firstName),
      infoRow("LName", widget.customer.lastName),
      infoRow("Email", widget.customer.email),
      infoRow("Phone", widget.customer.phone ?? '-'),
      const Divider(height: 32),
      infoRow("Last Order", formatDate(lastOrderDate)),
      infoRow(
        "Average Order Value",
        averageOrderValue.toStringAsFixed(2),
      ),
      infoRow(
        "Register Date",
        formatDate(widget.customer.createdAt),
      ),
    ],
  ),
),
const SizedBox(height: 24),
_buildSectionCard(
  title: "Shipping Address",
  child: addresses.isEmpty
    ? const Text(
        "No address found",
        style: TextStyle(color: Color(0xFFA3AED0)),
      )
    : Column(
        children: addresses
          .map((address) => _buildAddressItem(address))

```

```

        .toList(),
    ),
  ),
],
),
),
),

const SizedBox(width: 24),

/// --- RIGHT SIDE: Orders Table ---
Expanded(
  flex: 8,
  child: Column(
    children: [
      _buildSectionCard(
        title: "Orders History",
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            SizedBox(
              width: double.infinity,
              child: DataTable(
                headingRowColor: MaterialStateProperty.all(
                  const Color(0xFFFF4F7FE),
                ),
                columnSpacing: 20,
                columns: const [
                  DataColumn(label: _TableLabel("Order ID")),
                  DataColumn(label: _TableLabel("Date")),
                  DataColumn(label: _TableLabel("Item")),
                  DataColumn(label: _TableLabel("Status")),
                  DataColumn(label: _TableLabel("Amount")),
                ],
                rows: orders
                  .map(
                    (order) => DataRow(
                      cells: [
                        DataCell(
                          Text(
                            order.id,
                            style: const TextStyle(
                              fontSize: 12,
                              color: Color(0xFF4318FF),
                            ),
                        ),
                        DataCell(
                          Text(formatDate(order.orderDate)),
                        ),
                        DataCell(

```

```

        Text(order.itemCount.toString()),
      ),
      DataCell(
        _buildStatusBadge(order.orderStatus),
      ),
      DataCell(
        Text(
          order.totalAmount.toStringAsFixed(2),
        ),
        style: const TextStyle(
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
)
.toList(),
),
),
const SizedBox(height: 24),
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: const Color(0xFFFF4F7FE),
    borderRadius: BorderRadius.circular(12),
  ),
  child: Row(
    mainAxisAlignment:
MainAxisAlignment.spaceBetween,
    children: [
      const Text(
        "Summary Report",
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF2B3674),
        ),
      ),
      Text(
        "Total spent ${totalSpent.toStringAsFixed(2)}
on ${orders.length} orders",
        style: const TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF4318FF),
        ),
      ),
    ],
  ),
),
],
),
],

```

```
        ),
      ),
    ],
  ),
),
],
),
),
);
}

Widget _buildSectionCard({required String title, required Widget child}) {
  return Container(
    padding: const EdgeInsets.all(24),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(20),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.03),
          blurRadius: 20,
          offset: const Offset(0, 10),
        ),
      ],
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          title,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
            color: Color(0xFF2B3674),
          ),
        ),
        const SizedBox(height: 20),
        child,
      ],
    ),
  );
}

Widget _buildAddressItem(AddressModel address) {
  return Container(
    margin: const EdgeInsets.only(bottom: 12),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(12),
      border: Border.all(
```

```

        color: address.isDefault
            ? const Color(0xFF4318FF)
            : const Color(0xFFE0E5F2),
    ),
    color: address.isDefault
        ? const Color(0xFF4318FF).withOpacity(0.02)
        : Colors.transparent,
    ),
    child: Row(
      children: [
        Icon(
          Icons.location_on_rounded,
          color: address.isDefault
              ? const Color(0xFF4318FF)
              : const Color(0xFFA3AED0),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              if (address.isDefault)
                const Text(
                  "DEFAULT ADDRESS",
                  style: TextStyle(
                    color: Color(0xFF4318FF),
                    fontSize: 10,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              Text(
                address.fullAddress,
                style: const TextStyle(
                  color: Color(0xFF2B3674),
                  fontSize: 13,
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
}

Widget _buildStatusBadge(String status) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: Colors.blue.withOpacity(0.1),

```

```
        borderRadius: BorderRadius.circular(8),
      ),
      child: Text(
        status,
        style: const TextStyle(
          color: Colors.blue,
          fontWeight: FontWeight.bold,
          fontSize: 12,
        ),
      ),
    );
  }

Widget infoRow(String title, String value) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 16),
    child: Row(
      children: [
        SizedBox(
          width: 140,
          child: Text(
            title,
            style: const TextStyle(
              color: Color(0xFFA3AED0),
              fontWeight: FontWeight.w500,
            ),
          ),
        ),
        Expanded(
          child: Text(
            value,
            style: const TextStyle(
              color: Color(0xFF2B3674),
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      ],
    ),
  );
}

class _TableLabel extends StatelessWidget {
  final String label;
  const _TableLabel(this.label);

  @override
  Widget build(BuildContext context) {
    return Text(
```



```
        label,  
        style: const TextStyle(  
          color: Color(0xFFA3AED0),  
          fontWeight: FontWeight.bold,  
          fontSize: 13,  
        ),  
      );  
    }  
  }
```

Trong file `lib/controllers/customer_controller_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:e_commerce_web_admin/data/models/address_model.dart';  
import 'package:e_commerce_web_admin/data/models/customer_model.dart';  
import 'package:e_commerce_web_admin/data/models/order_model.dart';  
import 'package:flutter/material.dart';  
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'package:intl/intl.dart'; // Đảm bảo bạn đã thêm intl vào pubspec.yaml  
  
class CustomerDetailPage extends StatefulWidget {  
  final CustomerModel customer;  
  
  const CustomerDetailPage({super.key, required this.customer});  
  
  @override  
  State<CustomerDetailPage> createState() => _CustomerDetailPageState();  
}  
  
class _CustomerDetailPageState extends State<CustomerDetailPage> {  
  List<AddressModel> addresses = [];  
  List<OrderModel> orders = [];  
  bool isLoading = true;  
  
  @override  
  void initState() {  
    super.initState();  
    fetchData();  
  }  
  
  Future<void> fetchData() async {  
    final addressSnapshot = await FirebaseFirestore.instance  
      .collection('users')  
      .doc(widget.customer.id)  
      .collection('addresses')  
      .get();  
  
    final orderSnapshot = await FirebaseFirestore.instance  
      .collection('orders')
```

```

        .where('userId', isEqualTo: widget.customer.id)
        .get();

addresses = addressSnapshot.docs
    .map((doc) => AddressModel.fromFirestore(doc))
    .toList();

orders = orderSnapshot.docs
    .map((doc) => OrderModel.fromFirestore(doc))
    .toList();

setState(() {
    isLoading = false;
});
}

double get totalSpent =>
    orders.fold(0, (sum, item) => sum + item.totalAmount);

double get averageOrderValue =>
    orders.isEmpty ? 0 : totalSpent / orders.length;

DateTime? get lastOrderDate {
    if (orders.isEmpty) return null;
    orders.sort(
        (a, b) => (b.orderDate ?? DateTime(2000)).compareTo(
            a.orderDate ?? DateTime(2000),
        ),
    );
    return orders.first.orderDate;
}

String formatDate(DateTime? date) {
    if (date == null) return "-";
    return DateFormat('dd/MM/yyyy HH:mm').format(date);
}

@override
Widget build(BuildContext context) {
    if (isLoading) {
        return const Scaffold(
            backgroundColor: Color(0xFFF8F9FD),
            body: Center(
                child: CircularProgressIndicator(color: Color(0xFF4318FF)),
            ),
        );
    }

    return Scaffold(
        backgroundColor: const Color(0xFFF8F9FD),

```

```
appBar: AppBar(  
  backgroundColor: Colors.white,  
  elevation: 0,  
  leading: IconButton(  
    icon: const Icon(  
      Icons.arrow_back_ios_new_rounded,  
      color: Color(0xFF2B3674),  
      size: 20,  
    ),  
    onPressed: () => Navigator.pop(context),  
  ),  
  title: const Text(  
    "Customer Detail",  
    style: TextStyle(  
      color: Color(0xFF2B3674),  
      fontWeight: FontWeight.bold,  
    ),  
  ),  
,  
body: SingleChildScrollView(  
  padding: const EdgeInsets.all(24),  
  child: Row(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      /// --- LEFT SIDE: Profile & Address ---  
      Expanded(  
        flex: 4,  
        child: Column(  
          children: [  
            _buildSectionCard(  
              title: "Customer Information",  
              child: Column(  
                crossAxisAlignment: CrossAxisAlignment.start,  
                children: [  
                  Center(  
                    child: CircleAvatar(  
                      radius: 40,  
                      backgroundColor: const Color(  
                        0xFF4318FF,  
                      ).withOpacity(0.1),  
                    child: Text(  
                      widget.customer.firstName[0].toUpperCase(),  
                      style: const TextStyle(  
                        fontSize: 32,  
                        fontWeight: FontWeight.bold,  
                        color: Color(0xFF4318FF),  
                      ),  
                    ),  
                  ),  
                ],  
              ),  
            ),  
          ],  
        ),  
      ),  
    ],  
  ),  
,  
)
```

```

const SizedBox(height: 24),
infoRow("FName", widget.customer.firstName),
infoRow("LName", widget.customer.lastName),
infoRow("Email", widget.customer.email),
infoRow("Phone", widget.customer.phone ?? '-'),
const Divider(height: 32),
infoRow("Last Order", formatDate(lastOrderDate)),
infoRow(
  "Average Order Value",
  averageOrderValue.toStringAsFixed(2),
),
infoRow(
  "Register Date",
  formatDate(widget.customer.createdAt),
),
],
),
),
const SizedBox(height: 24),
_buildSectionCard(
  title: "Shipping Address",
  child: addresses.isEmpty
    ? const Text(
        "No address found",
        style: TextStyle(color: Color(0xFFA3AED0)),
      )
    : Column(
        children: addresses
          .map((address) => _buildAddressItem(address))
          .toList(),
      ),
),
],
),
),
const SizedBox(width: 24),

/// --- RIGHT SIDE: Orders Table ---
Expanded(
  flex: 8,
  child: Column(
    children: [
      _buildSectionCard(
        title: "Orders History",
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            SizedBox(
              width: double.infinity,

```

```

child: DataTable(
  headingRowColor: MaterialStateProperty.all(
    const Color(0xFFFF4F7FE),
  ),
  columnSpacing: 20,
  columns: const [
    DataColumn(label: _TableLabel("Order ID")),
    DataColumn(label: _TableLabel("Date")),
    DataColumn(label: _TableLabel("Item")),
    DataColumn(label: _TableLabel("Status")),
    DataColumn(label: _TableLabel("Amount")),
  ],
  rows: orders
    .map(
      (order) => DataRow(
        cells: [
          DataCell(
            Text(
              order.id,
              style: const TextStyle(
                fontSize: 12,
                color: Color(0xFF4318FF),
              ),
            ),
          ),
          DataCell(
            Text(formatDate(order.orderDate)),
          ),
          DataCell(
            Text(order.itemCount.toString()),
          ),
          DataCell(
            _buildStatusBadge(order.orderStatus),
          ),
          DataCell(
            Text(
              order.totalAmount.toStringAsFixed(2),
              style: const TextStyle(
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ],
      ),
    )
    .toList(),
),
),
const SizedBox(height: 24),

```

```
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: const Color(0xFFFFF7FE),
    borderRadius: BorderRadius.circular(12),
  ),
  child: Row(
    mainAxisAlignment:
MainAxisAlignment.spaceBetween,
    children: [
      const Text(
        "Summary Report",
        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF2B3674),
        ),
      ),
      Text(
        "Total spent ${totalSpent.toStringAsFixed(2)}
on ${orders.length} orders",
        style: const TextStyle(
          fontWeight: FontWeight.bold,
          color: Color(0xFF4318FF),
        ),
      ),
    ],
  ),
),
],
),
),
),
),
],
),
),
],
),
);
}

Widget _buildSectionCard({required String title, required Widget child}) {
  return Container(
    padding: const EdgeInsets.all(24),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(20),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.03),
          blurRadius: 20,
```

```

        offset: const Offset(0, 10),
      ),
    ],
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        title,
        style: const TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.bold,
          color: Color(0xFF2B3674),
        ),
      ),
      const SizedBox(height: 20),
      child,
    ],
  ),
);
}

Widget _buildAddressItem(AddressModel address) {
  return Container(
    margin: const EdgeInsets.only(bottom: 12),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(12),
      border: Border.all(
        color: address.isDefault
          ? const Color(0xFF4318FF)
          : const Color(0xFFE0E5F2),
      ),
      color: address.isDefault
        ? const Color(0xFF4318FF).withOpacity(0.02)
        : Colors.transparent,
    ),
    child: Row(
      children: [
        Icon(
          Icons.location_on_rounded,
          color: address.isDefault
            ? const Color(0xFF4318FF)
            : const Color(0xFFA3AED0),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [

```

```
        if (address.isDefault)
            const Text(
                "DEFAULT ADDRESS",
                style: TextStyle(
                    color: Color(0xFF4318FF),
                    fontSize: 10,
                    fontWeight: FontWeight.bold,
                ),
            ),
        Text(
            address.fullAddress,
            style: const TextStyle(
                color: Color(0xFF2B3674),
                fontSize: 13,
            ),
        ),
    ],
),
),
],
),
);
}

Widget _buildStatusBadge(String status) {
    return Container(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
        decoration: BoxDecoration(
            color: Colors.blue.withOpacity(0.1),
            borderRadius: BorderRadius.circular(8),
        ),
        child: Text(
            status,
            style: const TextStyle(
                color: Colors.blue,
                fontWeight: FontWeight.bold,
                fontSize: 12,
            ),
        ),
    );
}

Widget infoRow(String title, String value) {
    return Padding(
        padding: const EdgeInsets.only(bottom: 16),
        child: Row(
            children: [
                SizedBox(
                    width: 140,
                    child: Text(
```



```
        title,
        style: const TextStyle(
          color: Color(0xFFA3AED0),
          fontWeight: FontWeight.w500,
        ),
      ),
    ),
    Expanded(
      child: Text(
        value,
        style: const TextStyle(
          color: Color(0xFF2B3674),
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ],
),
);
}
}
```

```
class _TableLabel extends StatelessWidget {
  final String label;
  const _TableLabel(this.label);

  @override
  Widget build(BuildContext context) {
    return Text(
      label,
      style: const TextStyle(
        color: Color(0xFFA3AED0),
        fontWeight: FontWeight.bold,
        fontSize: 13,
      ),
    );
  }
}
```

Trong file `lib/data/models/customer_model.dart`, soạn mã nguồn có nội dung như sau:

```
class CustomerModel {
  String id;
  String firstName;
  String lastName;
  String email;
  String phone;
  String username;
```

```
String gender;
DateTime? createdAt;

CustomerModel({
  required this.id,
  required this.firstName,
  required this.lastName,
  required this.email,
  required this.phone,
  required this.username,
  required this.gender,
  this.createdAt,
});

factory CustomerModel.fromMap(Map<String, dynamic> map) {
  return CustomerModel(
    id: map['id'] ?? '',
    firstName: map['firstName'] ?? '',
    lastName: map['lastName'] ?? '',
    email: map['email'] ?? '',
    phone: map['phone'] ?? '',
    username: map['username'] ?? '',
    gender: map['gender'] ?? 'Not set',
    createdAt: map['createdAt'] != null
      ? DateTime.tryParse(map['createdAt'])
      : null,
  );
}

String get fullName => "$firstName $lastName";
}
```

Trong file `lib/data/models/address_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

class AddressModel {
  final String id;
  final String city;
  final String ward;
  final String street;
  final String number;
  final bool isDefault;
  final DateTime? createdAt;
  final double? latitude;
  final double? longitude;

  AddressModel({
```

```
        required this.id,
        required this.city,
        required this.ward,
        required this.street,
        required this.number,
        required this.isDefault,
        this.createdAt,
        this.latitude,
        this.longitude,
    });

    factory AddressModel.fromFirestore(DocumentSnapshot doc) {
        final data = doc.data() as Map<String, dynamic>;

        return AddressModel(
            id: doc.id,
            city: data['city'] ?? '',
            ward: data['ward'] ?? '',
            street: data['street'] ?? '',
            number: data['number'] ?? '',
            isDefault: data['isDefault'] ?? false,
            createdAt: (data['createdAt'] as Timestamp?)?.toDate(),
            latitude: (data['latitude'] as num?)?.toDouble(),
            longitude: (data['longitude'] as num?)?.toDouble(),
        );
    }

    String get fullAddress => '$number $street, $ward, $city';
}
```

Trong file lib/data/services/customer_service.dart, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/customer_model.dart';

class CustomerService {
    final _firestore = FirebaseFirestore.instance;

    Future<List<CustomerModel>> getCustomers() async {
        final snapshot = await _firestore.collection('users').get();

        return snapshot.docs
            .map((doc) => CustomerModel.fromMap(doc.data()))
            .toList();
    }

    Future<int> getOrdersCount(String userId) async {
        final snapshot = await _firestore
            .collection('orders')

```

```
        .where('userId', isEqualTo: userId)
        .get();

    return snapshot.docs.length;
}

Future<List<Map<String, dynamic>>> getOrdersOfUser(String userId) async {
    final snapshot = await _firestore
        .collection('orders')
        .where('userId', isEqualTo: userId)
        .get();

    return snapshot.docs.map((e) => e.data()).toList();
}

Future<void> deleteCustomer(String id) async {
    await _firestore.collection('users').doc(id).delete();
}

Future<CustomerModel?> getById(String id) async {
    final doc = await _firestore.collection('users').doc(id).get();

    if (!doc.exists) return null;

    final data = doc.data()!;
    data['id'] = doc.id;

    return CustomerModel.fromMap(data);
}
}
```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```
import 'package:draf_project_web/views/brands/brands_page.dart';
import 'package:draf_project_web/views/coupons/coupons_page.dart';
import 'package:draf_project_web/views/customers/customers_page.dart';
import 'package:draf_project_web/views/orders/orders_page.dart';
import 'package:draf_project_web/views/products/product_list_page.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/categories/category_page.dart';
import '../controllers/auth_controller.dart';

import '../views/attributes/attribute_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
    with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
    final GlobalKey<NavigatorState> navigatorKey;
    final AuthController authController;
```

```
String _currentPath = "/login";
AppRouterDelegate(this.authController)
  : navigatorKey = GlobalKey<NavigatorState>() {
  authController.addListener(notifyListeners);
}
@override
String? get currentConfiguration => _currentPath;
@override
Widget build(BuildContext context) {
  if (!authController.isLoggedIn) {
    return Navigator(
      key: navigatorKey,
      pages: const [MaterialPage(child: LoginPage())],
      onPopPage: (route, result) => route.didPop(result),
    );
  }
  Widget page;
  switch (_currentPath) {
    case "/attributes":
      page = const AttributesPage();
      break;
    case "/orders":
      page = const OrderPage();
      break;
    case "/brands":
      page = const BrandsPage();
      break;
    case "/customers":
      page = const CustomersPage();
      break;
    case "/coupons":
      page = const CouponsPage();
      break;
    case "/products":
      page = const ProductListPage();
      break;

    case "/categories":
      page = const CategoriesPage();
      break;

    default:
      page = const Center(child: Text("Dashboard Page"));
  }
  return Navigator(
    key: navigatorKey,
    pages: [
      MaterialPage(
        child: AdminLayout(
          currentRoute: _currentPath, // truyền route hiện tại
```

```
        onNavigate: (path) {
          _currentPath = path;
          notifyListeners();
        },
        child: page,
      ),
    ),
  ],
  onPopPage: (route, result) => route.didPop(result),
);
}

@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

11. Thiết kế và phát triển nhóm chức năng quản lý thông tin đánh giá sản phẩm

Trong file `lib/views/product_review/all_review_screen.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'package:draf_project_web/controllers/product_review_controller.dart';

class AllReviewScreen extends StatelessWidget {
  const AllReviewScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (BuildContext context) => ReviewController(),
      child: Scaffold(
```

```
backgroundColor: const Color(0xFFFF8F9FA), // Nền xám nhạt hiện đại
body: Consumer<ReviewController>(
  builder:
    (
      BuildContext context,
      ReviewController controller,
      Widget? child,
    ) {
      return Padding(
        padding: const EdgeInsets.all(30),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            /// --- HEADER SECTION ---
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Text(
                      "Customer Reviews",
                      style: TextStyle(
                        fontSize: 32,
                        fontWeight: FontWeight.bold,
                        color: Color(0xFF2D3436),
                        letterSpacing: 1,
                      ),
                    ),
                    Text(
                      "Manage and moderate your product feedback",
                      style: TextStyle(
                        fontSize: 14,
                        color: Colors.grey.shade600,
                      ),
                    ),
                  ],
                ),
            ],
          ),
          // Hiển thị tổng số review cho sinh động
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 20,
              vertical: 10,
            ),
            decoration: BoxDecoration(
              color: Colors.blue.withOpacity(0.1),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Text(
              "Total: ${controller.filteredReviews.length}",
```

```
        style: const TextStyle(
          color: Colors.blue,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),

  const SizedBox(height: 30),

  /// --- FILTER & SEARCH BAR ---
  Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(15),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.03),
          blurRadius: 10,
          offset: const Offset(0, 4),
        ),
      ],
    ),
    child: Row(
      children: [
        // Search Input
        Expanded(
          flex: 3,
          child: TextField(
            onChanged: controller.search,
            decoration: InputDecoration(
              hintText:
                "Search by product, user or
content...",
              prefixIcon: const Icon(
                Icons.search,
                color: Colors.blue,
              ),
              filled: true,
              fillColor: Colors.grey.shade50,
              border: OutlineInputBorder(
                borderRadius: BorderRadius.circular(12),
                borderSide: BorderSide.none,
              ),
              contentPadding: const EdgeInsets.symmetric(
                vertical: 15,
              ),
            ),
          ),
        ),
      ],
    ),
  ),
```



```

    ),
  ),
  const SizedBox(width: 20),
  // Dropdown Filter
  Expanded(
    flex: 1,
    child: Container(
      padding: const EdgeInsets.symmetric(
        horizontal: 12,
      ),
      decoration: BoxDecoration(
        color: Colors.grey.shade50,
        borderRadius: BorderRadius.circular(12),
      ),
      child: DropdownButtonHideUnderline(
        child: DropdownButton<String>(
          value: controller.statusFilter,
          icon: const Icon(
            Icons.filter_list,
            color: Colors.blue,
          ),
          isExpanded: true,
          items: const [
            DropdownMenuItem(
              value: "all",
              child: Text("All Status"),
            ),
            DropdownMenuItem(
              value: "pending",
              child: Text("Pending"),
            ),
            DropdownMenuItem(
              value: "approved",
              child: Text("Approved"),
            ),
          ],
          onChanged: (String? value) {
            if (value != null) {
              controller.filterByStatus(value);
            }
          },
        ),
      ),
    ),
  ),
],
),
),
const SizedBox(height: 25),

```

```

/// --- DATA TABLE CARD ---
Expanded(
  child: Container(
    width: double.infinity,
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(15),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.03),
          blurRadius: 10,
          offset: const Offset(0, 4),
        ),
      ],
    ),
    child: controller.filteredReviews.isEmpty
      ? _buildEmptyState()
      : ClipRRect(
          borderRadius: BorderRadius.circular(15),
          child: SingleChildScrollView(
            scrollDirection: Axis.vertical,
            child: SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: DataTable(
                headingRowHeight: 60,
                dataRowHeight: 75,
                columnSpacing: 25,
                headingRowColor:
                  MaterialStateProperty.all(
                    Colors.blue.withOpacity(0.05),
                  ),
                columns: _buildTableColumns(),
                rows: List.generate(
                  controller.filteredReviews.length,
                  (index) => _buildDataRow(
                    context,
                    controller,
                    index,
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    ),
  );

```

```

        },
    ),
),
);
}

/// Widget hiển thị khi không có dữ liệu
Widget _buildEmptyState() {
    return Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
            Icon(Icons.feedback_outlined, size: 80, color: Colors.grey.shade300),
            const SizedBox(height: 16),
            Text(
                "No reviews found",
                style: TextStyle(
                    fontSize: 18,
                    color: Colors.grey.shade500,
                    fontWeight: FontWeight.w500,
                ),
            ),
        ],
    );
}

/// Định nghĩa các cột của bảng
List<DataColumn> _buildTableColumns() {
    const TextStyle headerStyle = TextStyle(
        fontWeight: FontWeight.bold,
        color: Color(0xFF444444),
    );
    return const [
        DataColumn(label: Text("SEQ", style: headerStyle)),
        DataColumn(label: Text("PRODUCT", style: headerStyle)),
        DataColumn(label: Text("REVIEW CONTENT", style: headerStyle)),
        DataColumn(label: Text("RATING", style: headerStyle)),
        DataColumn(label: Text("USER", style: headerStyle)),
        DataColumn(label: Text("STATUS", style: headerStyle)),
        DataColumn(label: Text("DATE", style: headerStyle)),
        DataColumn(label: Text("ACTIONS", style: headerStyle)),
    ];
}

/// Tạo từng dòng dữ liệu
DataRow _buildDataRow(
    BuildContext context,
    ReviewController controller,
    int index,
) {
    final review = controller.filteredReviews[index];

```

```

return DataRow(
  cells: [
    // Số thứ tự
    DataCell(
      Text("#${index + 1}", style: const TextStyle(color: Colors.grey)),
    ),

    // Sản phẩm (Ảnh + Tên)
    DataCell(
      Row(
        children: [
          Container(
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(8),
              border: Border.all(color: Colors.grey.shade200),
            ),
            child: ClipRRect(
              borderRadius: BorderRadius.circular(8),
              child: Image.network(
                review.productImage,
                width: 45,
                height: 45,
                fit: BoxFit.cover,
                errorBuilder: (context, error, stackTrace) =>
                  const Icon(Icons.image, size: 20),
              ),
            ),
          ),
          const SizedBox(width: 12),
          SizedBox(
            width: 140,
            child: Text(
              review.productName,
              style: const TextStyle(fontWeight: FontWeight.w600),
              overflow: TextOverflow.ellipsis,
            ),
          ),
        ],
      ),
    ),

    // Nội dung Review
    DataCell(
      SizedBox(
        width: 220,
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [

```

```

        Text(
            review.title,
            style: const TextStyle(
                fontWeight: FontWeight.bold,
                fontSize: 13,
            ),
            overflow: TextOverflow.ellipsis,
        ),
        Text(
            review.reviewText,
            style: TextStyle(color: Colors.grey.shade600, fontSize:
12),
            overflow: TextOverflow.ellipsis,
            maxLines: 1,
        ),
    ],
),
),
),
),

// Rating Star
DataCell(
    Container(
        padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
        decoration: BoxDecoration(
            color: Colors.amber.withOpacity(0.1),
            borderRadius: BorderRadius.circular(20),
        ),
        child: Row(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
                const Icon(Icons.star, color: Colors.amber, size: 16),
                const SizedBox(width: 4),
                Text(
                    review.rating.toString(),
                    style: const TextStyle(
                        fontWeight: FontWeight.bold,
                        color: Colors.orange,
                    ),
                ),
            ],
        ),
    ),
),
),

// User
DataCell(
    FutureBuilder<String>(
        future: controller.getCustomerName(review.userId),
        builder: (context, snapshot) {

```

```

        return Text(
            snapshot.data ?? "Loading...",
            style: const TextStyle(
                color: Colors.blueGrey,
                fontWeight: FontWeight.w500,
            ),
        );
    },
),
),

// Status Badge
DataCell(
    Container(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        decoration: BoxDecoration(
            color: review.isApproved
                ? const Color(0xFFE3F9E5)
                : const Color(0xFFFFF4E5),
            borderRadius: BorderRadius.circular(20),
        ),
        child: Text(
            review.isApproved ? "Approved" : "Pending",
            style: TextStyle(
                fontSize: 12,
                fontWeight: FontWeight.bold,
                color: review.isApproved
                    ? const Color(0xFF1F8B24)
                    : const Color(0xFFD97706),
            ),
        ),
    ),
),

// Date
DataCell(
    Text(
        "${review.updatedAt.day.toString().padLeft(2,
'0')}/${review.updatedAt.month.toString().padLeft(2,
'0')}/${review.updatedAt.year}",
        style: TextStyle(color: Colors.grey.shade600, fontSize: 13),
    ),
),

// Action Buttons
DataCell(
    Row(
        children: [
            if (!review.isApproved)
                _buildActionButton(

```

```

        icon: Icons.check_circle_outline,
        color: Colors.green,
        tooltip: "Approve Review",
        onTap: () => controller.approve(review),
      ),
      const SizedBox(width: 8),
      _buildActionButton(
        icon: Icons.delete_outline,
        color: Colors.redAccent,
        tooltip: "Delete Review",
        onTap: () => _showDeleteDialog(context, controller,
review.id),
      ),
    ],
  ),
],
);
}

/// Nút bấm thao tác nhỏ gọn
Widget _buildActionButton({
  required IconData icon,
  required Color color,
  required String tooltip,
  required VoidCallback onTap,
}) {
  return Tooltip(
    message: tooltip,
    child: InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(8),
      child: Container(
        padding: const EdgeInsets.all(8),
        decoration: BoxDecoration(
          color: color.withOpacity(0.1),
          borderRadius: BorderRadius.circular(8),
        ),
        child: Icon(icon, color: color, size: 20),
      ),
    ),
  );
}

/// Dialog xác nhận xóa chuyên nghiệp hơn
Future<void> _showDeleteDialog(
  BuildContext context,
  ReviewController controller,
  String id,
) async {

```

```
final bool? confirm = await showDialog<bool>(  
  context: context,  
  builder: (BuildContext context) => AlertDialog(  
    shape: RoundedRectangleBorder(borderRadius:   
BorderRadius.circular(15)),  
    title: const Row(  
      children: [  
        Icon(Icons.warning_amber_rounded, color: Colors.red),  
        SizedBox(width: 10),  
        Text("Confirm Delete"),  
      ],  
    ),  
    content: const Text(  
      "This action cannot be undone. Do you want to remove this review  
permanently?",  
    ),  
    actions: [  
      TextButton(  
        onPressed: () => Navigator.pop(context, false),  
        child: const Text("Cancel", style: TextStyle(color:   
Colors.grey)),  
      ),  
      ElevatedButton(  
        onPressed: () => Navigator.pop(context, true),  
        style: ElevatedButton.styleFrom(  
          backgroundColor: Colors.red,  
          shape: RoundedRectangleBorder(  
            borderRadius: BorderRadius.circular(8),  
          ),  
        ),  
        child: const Text(  
          "Delete Now",  
          style: TextStyle(color: Colors.white),  
        ),  
      ),  
    ],  
  ),  
);  
  
if (confirm == true) {  
  await controller.delete(id);  
}  
}
```

Trong file `lib/controllers/product_review_controller.dart`, soạn mã nguồn có nội dung như sau:


```
import 'dart:async';
import 'package:flutter/material.dart';
import '../data/models/product_review_model.dart';
import '../data/models/customer_model.dart';
import '../data/services/product_review_service.dart';
import '../data/services/customer_service.dart';

class ReviewController extends ChangeNotifier {
  final ReviewService reviewService = ReviewService();
  final CustomerService customerService = CustomerService();

  StreamSubscription? _subscription;

  List<ReviewModel> allReviews = [];
  List<ReviewModel> filteredReviews = [];

  Map<String, CustomerModel> customerCache = {};

  ReviewController() {
    _listenReviews();
  }

  void _listenReviews() {
    _subscription = reviewService.getAll().listen((data) {
      allReviews = data;
      filteredReviews = data;
      notifyListeners();
    });
  }

  /// SEARCH
  void search(String keyword) {
    if (keyword.isEmpty) {
      filteredReviews = allReviews;
    } else {
      filteredReviews = allReviews.where((e) {
        return e.title.toLowerCase().contains(keyword.toLowerCase()) ||
          e.productName.toLowerCase().contains(keyword.toLowerCase());
      }).toList();
    }
    notifyListeners();
  }

  /// GET CUSTOMER FULLNAME
  Future<String> getCustomerName(String userId) async {
    if (customerCache.containsKey(userId)) {
      return customerCache[userId]!.fullName;
    }
  }
}
```

```
final customer = await customerService.getById(userId);

if (customer != null) {
  customerCache[userId] = customer;
  return customer.fullName;
}

return "Unknown";
}

Future<void> delete(String id) async {
  await reviewService.delete(id);
}

@override
void dispose() {
  _subscription?.cancel();
  super.dispose();
}

String statusFilter = "all";

void filterByStatus(String value) {
  statusFilter = value;

  if (value == "all") {
    filteredReviews = allReviews;
  } else if (value == "pending") {
    filteredReviews = allReviews.where((e) => e.isApproved ==
false).toList();
  } else {
    filteredReviews = allReviews.where((e) => e.isApproved ==
true).toList();
  }

  notifyListeners();
}

Future<void> approve(ReviewModel review) async {
  await reviewService.approveAndUpdateProduct(review);
}
}
```

Trong file `lib/data/models/product_review_model.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';

class ReviewModel {
```

```
final String id;
final String productId;
final String productName;
final String productImage;

final String title;
final String userId;
final String userName;
final String? userProfileImage;

final double rating;
final String reviewText;
final List<String> mediaUrls;

final DateTime createdAt;
final DateTime updatedAt;

final bool isApproved;

ReviewModel({
  required this.id,
  required this.productId,
  required this.productName,
  required this.productImage,
  required this.userId,
  required this.userName,
  required this.title,
  this.userProfileImage,
  required this.rating,
  required this.reviewText,
  required this.mediaUrls,
  required this.createdAt,
  required this.updatedAt,
  required this.isApproved,
});

factory ReviewModel.fromSnapshot(DocumentSnapshot doc) {
  final data = doc.data() as Map<String, dynamic>;

  return ReviewModel(
    id: doc.id,
    productId: data['productId'],
    productName: data['productName'] ?? '',
    productImage: data['productImage'] ?? '',
    userId: data['userId'],
    userName: data['userName'] ?? '',
    title: data['title'] ?? '',
    userProfileImage: data['userProfileImage'],
    rating: (data['rating'] ?? 0).toDouble(),
    reviewText: data['reviewText'] ?? '',
```

```
mediaUrls: List<String>.from(data['mediaUrls'] ?? []),
createdAt: (data['createdAt'] as Timestamp)?.toDate() ??
DateTime.now(),
updatedAt: (data['updatedAt'] as Timestamp)?.toDate() ??
DateTime.now(),
isApproved: data['isApproved'] ?? false,
);
}
}
```

Trong file `lib/data/services/product_review_service.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/product_review_model.dart';

class ReviewService {
  final _db = FirebaseFirestore.instance;
  final String collection = "reviews";

  /// APPROVE REVIEW
  Future<void> approveAndUpdateProduct(ReviewModel review) async {
    final reviewRef = _db.collection("reviews").doc(review.id);
    final productRef = _db.collection("products").doc(review.productId);

    await _db.runTransaction((transaction) async {
      final reviewSnap = await transaction.get(reviewRef);
      final productSnap = await transaction.get(productRef);

      if (!reviewSnap.exists || !productSnap.exists) {
        throw Exception("Review or Product not found");
      }

      final reviewData = reviewSnap.data() as Map<String, dynamic>;

      /// Nếu đã approve rồi thì không làm nữa
      if (reviewData['isApproved'] == true) return;

      final productData = productSnap.data() as Map<String, dynamic>;

      double currentRating = (productData['rating'] ?? 0).toDouble();
      int ratingCount = productData['ratingCount'] ?? 0;
      int reviewsCount = productData['reviewsCount'] ?? 0;

      int fiveStar = productData['fiveStarCount'] ?? 0;
      int fourStar = productData['fourStarCount'] ?? 0;
      int threeStar = productData['threeStarCount'] ?? 0;
      int twoStar = productData['twoStarCount'] ?? 0;
      int oneStar = productData['oneStarCount'] ?? 0;
```

```
double newRatingValue = review.rating;

/// Update số sao tương ứng
switch (newRatingValue.round()) {
    case 5:
        fiveStar++;
        break;
    case 4:
        fourStar++;
        break;
    case 3:
        threeStar++;
        break;
    case 2:
        twoStar++;
        break;
    case 1:
        oneStar++;
        break;
}

/// Tính lại rating trung bình
double newAverage =
    ((currentRating * ratingCount) + newRatingValue) / (ratingCount +
1);

/// 1. Update review
transaction.update(reviewRef, {
    "isApproved": true,
    "updatedAt": FieldValue.serverTimestamp(),
});

/// 2. Update product
transaction.update(productRef, {
    "rating": double.parse(newAverage.toStringAsFixed(1)),
    "ratingCount": ratingCount + 1,
    "reviewsCount": reviewsCount + 1,
    "fiveStarCount": fiveStar,
    "fourStarCount": fourStar,
    "threeStarCount": threeStar,
    "twoStarCount": twoStar,
    "oneStarCount": oneStar,
    "updatedAt": FieldValue.serverTimestamp(),
});
});
}

/// DELETE REVIEW
Future<void> delete(String id) async {
```

```

        await _db.collection(collection).doc(id).delete();
    }

    Stream<List<ReviewModel>> getAll() {
        return _db
            .collection(collection)
            .orderBy("updatedAt", descending: true)
            .snapshots()
            .map(
                (snapshot) =>
                    snapshot.docs.map((e) => ReviewModel.fromSnapshot(e)).toList(),
            );
    }
}

```

Trong file `lib/routes/app_routes.dart`, cập nhật lại mã nguồn có nội dung như sau:

```

import 'package:draf_project_web/views/brands/brands_page.dart';
import 'package:draf_project_web/views/coupons/coupons_page.dart';
import 'package:draf_project_web/views/customers/customers_page.dart';
import 'package:draf_project_web/views/orders/orders_page.dart';
import
    'package:draf_project_web/views/product_review/all_review_screen.dart';
import 'package:draf_project_web/views/products/product_list_page.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/categories/category_page.dart';
import '../controllers/auth_controller.dart';

import '../views/attributes/attribute_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
    with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
    final GlobalKey<NavigatorState> navigatorKey;
    final AuthController authController;
    String _currentPath = "/login";
    AppRouterDelegate(this.authController)
        : navigatorKey = GlobalKey<NavigatorState>() {
        authController.addListener(notifyListeners);
    }
    @override
    String? get currentConfiguration => _currentPath;
    @override
    Widget build(BuildContext context) {
        if (!authController.isLoggedIn) {
            return Navigator(
                key: navigatorKey,
                pages: const [MaterialPage(child: LoginPage())],
                onPopPage: (route, result) => route.didPop(result),
            );
        }
    }
}

```

```
    );
  }
  Widget page;
  switch (_currentPath) {
    case "/attributes":
      page = const AttributesPage();
      break;
    case "/orders":
      page = const OrderPage();
      break;
    case "/brands":
      page = const BrandsPage();
      break;
    case "/customers":
      page = const CustomersPage();
      break;
    case "/reviews":
      page = const AllReviewScreen();
      break;
    case "/coupons":
      page = const CouponsPage();
      break;
    case "/products":
      page = const ProductListPage();
      break;

    case "/categories":
      page = const CategoriesPage();
      break;

    default:
      page = const Center(child: Text("Dashboard Page"));
  }
  return Navigator(
    key: navigatorKey,
    pages: [
      MaterialPage(
        child: AdminLayout(
          currentRoute: _currentPath, // truyền route hiện tại
          onNavigate: (path) {
            _currentPath = path;
            notifyListeners();
          },
          child: page,
        ),
      ),
    ],
    onPopPage: (route, result) => route.didPop(result),
  );
}
```

```
@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```

12. Thiết kế và phát triển nhóm chức năng màn hình bảng điều khiển

Trong file `lib/views/dashboard/dashboard_page.dart`, soạn mã nguồn có nội dung như sau:

```
import 'package:flutter/material.dart';
import 'package:fl_chart/fl_chart.dart';
import 'package:intl/intl.dart';

import '../controllers/order_controller.dart';
import '../data/models/order_model.dart';

class MyDashboard extends StatefulWidget {
  const MyDashboard({super.key});

  @override
  State<MyDashboard> createState() => _MyDashboardState();
}

class _MyDashboardState extends State<MyDashboard> {
  final OrderController controller = OrderController();

  bool isLoading = true;
  double totalSale = 0;
  double avgOrderValue = 0;
  int totalOrders = 0;
  int soldProducts = 0;

  Map<String, int> statusCount = {};
}
```



```
List<OrderModel> recentOrders = [];  
  
@override  
void initState() {  
  super.initState();  
  loadDashboard();  
}  
  
Future<void> loadDashboard() async {  
  await controller.fetchOrders();  
  final orders = controller.orders;  
  
  totalOrders = orders.length;  
  totalSale = orders.fold(0, (sum, order) => sum + order.totalAmount);  
  soldProducts = orders.fold(0, (sum, order) => sum + order.itemCount);  
  avgOrderValue = totalOrders == 0 ? 0 : totalSale / totalOrders;  
  
  statusCount.clear();  
  for (var order in orders) {  
    final status = order.orderStatus;  
    statusCount[status] = (statusCount[status] ?? 0) + 1;  
  }  
  
  recentOrders = orders.take(5).toList();  
  
  setState(() {  
    isLoading = false;  
  });  
}  
  
String money(double value) {  
  return NumberFormat("#,###").format(value);  
}  
  
Color getStatusColor(String status) {  
  switch (status.toLowerCase()) {  
    case 'created':  
      return Colors.blue;  
    case 'pending':  
      return Colors.orange;  
    case 'processing':  
      return Colors.purple;  
    case 'shipped':  
      return Colors.indigo;  
    case 'delivered':  
      return Colors.green;  
    case 'cancelled':  
      return Colors.red;  
    case 'returned':  
      return Colors.blueGrey;
```

```

        case 'refunded':
            return Colors.pink;
        default:
            return Colors.grey;
    }
}

@override
Widget build(BuildContext context) {
    if (isLoading) {
        return const Center(
            child: CircularProgressIndicator(strokeWidth: 2, color: Colors.blue),
        );
    }

    return Scaffold(
        backgroundColor: const Color(0xFFFF8F9FD),
        body: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                    /// TIÊU ĐỀ DASHBOARD
                    Row(
                        children: [
                            Container(
                                padding: const EdgeInsets.all(10),
                                decoration: BoxDecoration(
                                    color: Colors.blue.withOpacity(0.1),
                                    borderRadius: BorderRadius.circular(12),
                                ),
                                child: const Icon(
                                    Icons.analytics_rounded,
                                    color: Colors.blue,
                                    size: 28,
                                ),
                            ),
                            const SizedBox(width: 15),
                            const Text(
                                "Hệ Thống Quản Lý",
                                style: TextStyle(
                                    fontSize: 26,
                                    fontWeight: FontWeight.w800,
                                    color: Color(0xFF1A1C1E),
                                ),
                            ),
                        ],
                    ),
                    const SizedBox(height: 30),
                ],
            ),
        ),
    );
}

```

```
/// 4 THẺ THỐNG KÊ NẰM TRÊN 1 HÀNG NGANG
Row(
  children: [
    Expanded(
      child: statCard(
        "Tổng Doanh Thu",
        money(totalSale),
        Icons.monetization_on_rounded,
        [Colors.blue, Colors.blueAccent],
      ),
    ),
    const SizedBox(width: 15),
    Expanded(
      child: statCard(
        "Giá Trị TB",
        money(avgOrderValue),
        Icons.auto_graph_rounded,
        [Colors.purple, Colors.deepPurpleAccent],
      ),
    ),
    const SizedBox(width: 15),
    Expanded(
      child: statCard(
        "Tổng Đơn",
        totalOrders.toString(),
        Icons.shopping_bag_rounded,
        [Colors.orange, Colors.orangeAccent],
      ),
    ),
    const SizedBox(width: 15),
    Expanded(
      child: statCard(
        "Đã Bán",
        soldProducts.toString(),
        Icons.inventory_2_rounded,
        [Colors.teal, Colors.greenAccent],
      ),
    ),
  ],
),

const SizedBox(height: 30),

/// BỐ CỤC CHI TIẾT
Row(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    /// CỘT TRÁI
    Expanded(
```

```
flex: 3,
child: Column(
  children: [
    cardContainer(
      title: "Phân Tích Doanh Thu Tuần",
      child: SizedBox(
        height: 300,
        child: LineChart(mainLineData()),
      ),
    ),
    const SizedBox(height: 25),
    cardContainer(
      title: "Đơn Hàng Gần Đây",
      child: SizedBox(
        width: double.infinity,
        child: DataTable(
          headingRowColor: WidgetStateProperty.all(
            Colors.grey[50],
          ),
          horizontalMargin: 10,
          columnSpacing: 20,
          columns: const [
            DataColumn(
              label: Text(
                "Mã Đơn",
                style: TextStyle(fontWeight:
FontWeight.bold),
              ),
            ),
            DataColumn(
              label: Text(
                "Ngày",
                style: TextStyle(fontWeight:
FontWeight.bold),
              ),
            ),
            DataColumn(
              label: Text(
                "Số Lượng",
                style: TextStyle(fontWeight:
FontWeight.bold),
              ),
            ),
            DataColumn(
              label: Text(
                "Trạng Thái",
                style: TextStyle(fontWeight:
FontWeight.bold),
              ),
            ),
          ],
        ),
      ),
    ),
  ],
),
```

```

        DataColumn(
            label: Text(
                "Tổng Tiền",
                style: TextStyle(fontWeight:
FontWeight.bold),
            ),
        ),
    ],
    rows: recentOrders
        .map(
            (o) => DataRow(
                cells: [
                    DataCell(
                        Text("#${o.id.substring(0, 5)}..."),
                    ),
                    DataCell(
                        Text(
                            DateFormat(
                                'dd/MM/yyyy',
                            ).format(o.orderDate),
                        ),
                    ),
                    DataCell(
                        Center(
                            child:
Text(o.itemCount.toString()),
                        ),
                    ),
                    DataCell(statusChip(o.orderStatus)),
                    DataCell(
                        Text(
                            "${money(o.totalAmount)}đ",
                            style: const TextStyle(
                                fontWeight: FontWeight.bold,
                            ),
                        ),
                    ),
                ],
            ),
        )
        .toList(),
    ),
),
),
],
),
),
const SizedBox(width: 25),

```

```

    /// CỘT PHẢI
    Expanded(
      flex: 2,
      child: cardContainer(
        title: "Thống Kê Trạng Thái",
        child: Column(
          children: [
            SizedBox(
              height: 280,
              child: PieChart(
                PieChartData(
                  sectionsSpace: 4,
                  centerSpaceRadius: 50,
                  sections: buildStatusSections(),
                ),
              ),
            ),
            const SizedBox(height: 20),
            ...statusCount.entries
              .map((e) => statusRow(e.key, e.value))
              .toList(),
          ],
        ),
      ),
    ),
  ],
),
);
}

/// ===== CÁC COMPONENT GIAO DIỆN =====

Widget statCard(
  String title,
  String value,
  IconData icon,
  List<Color> colors,
) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: colors,
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ),
    ),
    borderRadius: BorderRadius.circular(16),
  );
}

```

```

        boxShadow: [
          BoxShadow(
            color: colors[0].withOpacity(0.3),
            blurRadius: 10,
            offset: const Offset(0, 4),
          ),
        ],
      ),
      child: Stack(
        clipBehavior: Clip.none,
        children: [
          Positioned(
            right: -10,
            bottom: -10,
            child: Icon(icon, size: 50, color:
Colors.white.withOpacity(0.15)),
          ),
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                title,
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(
                  color: Colors.white70,
                  fontSize: 13,
                  fontWeight: FontWeight.w500,
                ),
              ),
              const SizedBox(height: 8),
              FittedBox(
                child: Text(
                  value,
                  style: const TextStyle(
                    color: Colors.white,
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }

  Widget cardContainer({required String title, required Widget child}) {
    return Container(

```

```
padding: const EdgeInsets.all(24),
decoration: BoxDecoration(
  color: Colors.white,
  borderRadius: BorderRadius.circular(20),
  boxShadow: [
    BoxShadow(
      blurRadius: 15,
      color: Colors.black.withOpacity(0.04),
      offset: const Offset(0, 8),
    ),
  ],
),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      title,
      style: const TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
        color: Color(0xFF2D3238),
      ),
    ),
    const SizedBox(height: 20),
  ],
),
);
}

Widget statusChip(String status) {
  Color color = getStatusColor(status);
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: color.withOpacity(0.1),
      borderRadius: BorderRadius.circular(8),
      border: Border.all(color: color.withOpacity(0.2)),
    ),
    child: Text(
      status,
      style: TextStyle(
        color: color,
        fontSize: 11,
        fontWeight: FontWeight.bold,
      ),
    ),
  );
}
```



```
Widget statusRow(String status, int count) {
  Color color = getStatusColor(status);
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 6.0),
    child: Row(
      children: [
        Container(
          width: 10,
          height: 10,
          decoration: BoxDecoration(color: color, shape: BoxShape.circle),
        ),
        const SizedBox(width: 10),
        Expanded(
          child: Text(
            status,
            style: const TextStyle(color: Colors.black54, fontSize: 13),
          ),
        ),
        Text(
          count.toString(),
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
      ],
    ),
  );
}

/// ===== DATA & CHARTS =====

LineChartData mainLineData() {
  return LineChartData(
    gridData: FlGridData(
      show: true,
      drawVerticalLine: false,
      getDrawingHorizontalLine: (value) =>
        FlLine(color: Colors.grey[100], strokeWidth: 1),
    ),
    titlesData: FlTitlesData(
      rightTitles: AxisTitles(sideTitles: SideTitles(showTitles: false)),
      topTitles: AxisTitles(sideTitles: SideTitles(showTitles: false)),
      bottomTitles: AxisTitles(
        sideTitles: SideTitles(
          showTitles: true,
          getTitlesWidget: (value, meta) {
            const days = ["T2", "T3", "T4", "T5", "T6", "T7", "CN"];
            if (value.toInt() >= 0 && value.toInt() < days.length) {
              return Padding(
                padding: const EdgeInsets.only(top: 10),
                child: Text(
                  days[value.toInt()],

```

```

        style: TextStyle(color: Colors.grey[500], fontSize: 12),
      ),
    );
  }
  return const SizedBox();
},
),
),
),
borderData: FlBorderData(show: false),
lineBarsData: [
  LineChartBarData(
    spots: buildWeeklySpots(),
    isCurved: true,
    gradient: const LinearGradient(colors: [Colors.blue, Colors.cyan]),
    barWidth: 4,
    dotData: FlDotData(show: true),
    belowBarData: BarAreaData(
      show: true,
      gradient: LinearGradient(
        colors: [
          Colors.blue.withOpacity(0.15),
          Colors.blue.withOpacity(0.0),
        ],
      ),
    ),
  ),
],
);
}

List<FlSpot> buildWeeklySpots() {
  Map<int, double> weekdaySales = {1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7:
0};
  for (var order in controller.orders) {
    int day = order.orderDate.weekday;
    weekdaySales[day] = (weekdaySales[day] ?? 0) + order.totalAmount;
  }
  return List.generate(
    7,
    (index) => FlSpot(index.toDouble(), weekdaySales[index + 1!]),
  );
}

List<PieChartSectionData> buildStatusSections() {
  return statusCount.entries.map((e) {
    return PieChartSectionData(
      color: getStatusColor(e.key),
      value: e.value.toDouble(),
      title:

```

```

        '${((e.value / (totalOrders == 0 ? 1 : totalOrders)) *
100).toStringAsFixed(0)}%',
        radius: 60,
        titleStyle: const TextStyle(
          fontSize: 11,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      );
    }).toList();
  }
}

```

Trong terminal của project, sinh viên chạy lệnh sau để thêm vào thư viện cần thiết:

```
flutter pub add fl_chart intl
```

Trong file lib/routes/app_routes.dart, cập nhật lại mã nguồn có nội dung như sau:

```

import 'package:draf_project_web/views/customers/customers_page.dart';
import
'package:draf_project_web/views/product_review/all_review_screen.dart';
import 'package:flutter/material.dart';
import '../views/auth/login_page.dart';
import '../views/layout/admin_layout.dart';
import '../views/cagetories/category_page.dart';
import '../controllers/auth_controller.dart';
import '../views/brands/brands_page.dart';
import '../views/products/product_list_page.dart';
import '../views/attributes/attribute_page.dart';
import '../views/dashboard/dashboard_page.dart';

import '../views/coupons/coupons_page.dart';
import '../views/orders/orders_page.dart';

class AppRouterDelegate extends RouterDelegate<String>
  with ChangeNotifier, PopNavigatorRouterDelegateMixin<String> {
  final GlobalKey<NavigatorState> navigatorKey;
  final AuthController authController;
  String _currentPath = "/login";
  AppRouterDelegate(this.authController)
    : navigatorKey = GlobalKey<NavigatorState>() {
    authController.addListener(notifyListeners);
  }
  @override
  String? get currentConfiguration => _currentPath;
  @override
  Widget build(BuildContext context) {
    if (!authController.isLoggedIn) {
      return Navigator(
        key: navigatorKey,

```

```
        pages: const [MaterialPage(child: LoginPage())],
        onPopPage: (route, result) => route.didPop(result),
    );
}
Widget page;
switch (_currentPath) {
    case "/products":
        page = const ProductListPage();
        break;
    case "/attributes":
        page = const AttributesPage();
        break;
    case "/coupons":
        page = const CouponsPage();
        break;
    case "/brands":
        page = const BrandsPage();
        break;
    case "/orders":
        page = const OrderPage();
        break;
    case "/categories":
        page = const CategoriesPage();
        break;
    case "/customers":
        page = const CustomersPage();
        break;
    case "/reviews":
        page = const AllReviewScreen();
        break;

    case "/dashboard":
        page = const MyDashboard();
        break;
    default:
        page = const Center(child: Text("Dashboard Page"));
}
return Navigator(
    key: navigatorKey,
    pages: [
        MaterialPage(
            child: AdminLayout(
                currentRoute: _currentPath, // truyền route hiện tại
                onNavigate: (path) {
                    _currentPath = path;
                    notifyListeners();
                },
            ),
            child: page,
        ),
    ],
);
```

```
    ],
    onPopPage: (route, result) => route.didPop(result),
  );
}

@override
Future<void> setNewRoutePath(String configuration) async {
  _currentPath = configuration;
}
}

class AppRouteParser extends RouteInformationParser<String> {
  @override
  Future<String> parseRouteInformation(
    RouteInformation routeInformation,
  ) async {
    return routeInformation.location ?? "/login";
  }

  @override
  RouteInformation restoreRouteInformation(String configuration) {
    return RouteInformation(location: configuration);
  }
}
```